

OS1 kolokvijumi

kategorisani zadaci sa kolokvijuma 2006-2023 sa stranice predmeta

pripremili Luka Simić i Aleksa Marković

Jul 2024

Sadržaj

Predgovor	4
Ulaz/izlaz	5
1. zadatak, treći kolokvijum, jun 2022.	5
3. zadatak, kolokvijum, jul 2021.	6
3. zadatak, kolokvijum, avgust 2021.	7
3. zadatak, kolokvijum, oktobar 2020.	8
3. zadatak, kolokvijum, avgust 2020.	9
1. zadatak, prvi kolokvijum, mart 2019.	10
1. zadatak, prvi kolokvijum, maj 2019.	11
1. zadatak, prvi kolokvijum, jun 2019.	13
1. zadatak, prvi kolokvijum, mart 2018.	14
1. zadatak, prvi kolokvijum, april 2018.	15
1. zadatak, prvi kolokvijum, jun 2018.	16
1. zadatak, prvi kolokvijum, mart 2017.	17
1. zadatak, prvi kolokvijum, april 2017.	19
1. zadatak, prvi kolokvijum, jun 2017.	20
1. zadatak, prvi kolokvijum, mart 2016.	21
1. zadatak, prvi kolokvijum, maj 2016.	22
1. zadatak, prvi kolokvijum, septembar 2016.	24
1. zadatak, prvi kolokvijum, mart 2015.	24
1. zadatak, prvi kolokvijum, maj 2015.	26
1. zadatak, prvi kolokvijum, septembar 2015.	27
1. zadatak, prvi kolokvijum, mart 2014.	28
1. zadatak, prvi kolokvijum, april 2014.	29
1. zadatak, prvi kolokvijum, septembar 2014.	31
1. zadatak, prvi kolokvijum, mart 2013.	32
1. zadatak, prvi kolokvijum, april 2013.	33
1. zadatak, prvi kolokvijum, septembar 2013.	34
1. zadatak, prvi kolokvijum, mart 2012.	35
1. zadatak, prvi kolokvijum, maj 2012.	36
1. zadatak, prvi kolokvijum, septembar 2012.	37
1. zadatak, prvi kolokvijum, april 2011.	38
1. zadatak, prvi kolokvijum, maj 2011.	39
1. zadatak, prvi kolokvijum, septembar 2011.	40
1. zadatak, prvi kolokvijum, april 2010.	41
2. zadatak, prvi kolokvijum, april 2009.	42
2. zadatak, prvi kolokvijum, april 2008.	43
1. zadatak, prvi kolokvijum, april 2007.	44
2. zadatak, prvi kolokvijum, april 2006.	44

Ulaz/izlaz (blokovski uređaji)	46
1. zadatak, treći kolokvijum, jun 2023.	46
1. zadatak, treći kolokvijum, avgust 2023.	47
3. zadatak, kolokvijum, jun 2021.	49
1. zadatak, treći kolokvijum, jun 2019.	50
1. zadatak, treći kolokvijum, jun 2018.	52
1. zadatak, treći kolokvijum, jun 2017.	52
1. zadatak, treći kolokvijum, septembar 2016.	54
1. zadatak, treći kolokvijum, jun 2015.	55
1. zadatak, treći kolokvijum, septembar 2015.	56
1. zadatak, treći kolokvijum, jun 2014.	57
1. zadatak, treći kolokvijum, septembar 2014.	57
1. zadatak, treći kolokvijum, jun 2013.	58
1. zadatak, treći kolokvijum, jun 2012.	59
1. zadatak, treći kolokvijum, septembar 2011.	60
Komandna linija	62
2. zadatak, treći kolokvijum, jun 2024.	62
2. zadatak, treći kolokvijum, avgust 2023.	62
2. zadatak, treći kolokvijum, jun 2017.	63
2. zadatak, treći kolokvijum, septembar 2012.	64
Fajl sistem (interfejs)	66
1. zadatak, treći kolokvijum, jun 2024.	66
2. zadatak, treći kolokvijum, jun 2023.	66
2. zadatak, treći kolokvijum, jun 2022.	67
4. zadatak, kolokvijum, jun 2021.	69
4. zadatak, kolokvijum, jul 2020.	70
2. zadatak, treći kolokvijum, jun 2019.	71
2. zadatak, treći kolokvijum, jun 2018.	72
2. zadatak, treći kolokvijum, jun 2016.	74
2. zadatak, treći kolokvijum, septembar 2016.	75
2. zadatak, treći kolokvijum, jun 2015.	76
2. zadatak, treći kolokvijum, septembar 2015.	77
2. zadatak, treći kolokvijum, jun 2014.	77
2. zadatak, treći kolokvijum, septembar 2014.	78
2. zadatak, treći kolokvijum, jun 2013.	79
2. zadatak, treći kolokvijum, septembar 2013.	80
2. zadatak, treći kolokvijum, jun 2012.	81
2. zadatak, treći kolokvijum, jun 2011.	81
2. zadatak, treći kolokvijum, septembar 2011.	81
Fajl sistem (implementacija)	83
3. zadatak, treći kolokvijum, jun 2024.	83
3. zadatak, treći kolokvijum, jun 2023.	84
3. zadatak, treći kolokvijum, avgust 2023.	85
3. zadatak, treći kolokvijum, jun 2022.	86
4. zadatak, kolokvijum, jul 2021.	87
4. zadatak, kolokvijum, avgust 2021.	88
4. zadatak, kolokvijum, oktobar 2020.	89
4. zadatak, kolokvijum, jun 2020.	89
4. zadatak, kolokvijum, septembar 2020.	91
3. zadatak, treći kolokvijum, jun 2019.	91
3. zadatak, treći kolokvijum, jun 2018.	92
3. zadatak, treći kolokvijum, jun 2017.	93
3. zadatak, treći kolokvijum, jun 2016.	93
3. zadatak, treći kolokvijum, septembar 2016.	94

3. zadatak, treći kolokvijum, jun 2015.	94
3. zadatak, treći kolokvijum, septembar 2015.	96
3. zadatak, treći kolokvijum, jun 2014.	97
3. zadatak, treći kolokvijum, septembar 2014.	98
3. zadatak, treći kolokvijum, jun 2013.	99
3. zadatak, treći kolokvijum, septembar 2013.	99
3. zadatak, treći kolokvijum, jun 2012.	100
3. zadatak, treći kolokvijum, septembar 2012.	100
3. zadatak, treći kolokvijum, jun 2011.	101
3. zadatak, treći kolokvijum, septembar 2011.	102
Uvod u operativne sisteme	103
1. zadatak, prvi kolokvijum, april 2009.	103
1. zadatak, prvi kolokvijum, april 2008.	103
1. zadatak, prvi kolokvijum, april 2006.	103

Predgovor

Svrha ove zbirke jeste da objedini sve do sada dostupne kolokvijume iz Operativnih sistema 1 iz nekoliko razloga:

- Na kolokvijumima iz OS1 je dozvoljena literatura, i studenti često pristupaju tome tako što odštampaju kolokvijume i njihova rešenja pa se na kolokvijumu snalaze kroz taj odštampani materijal ako vide neki sličan zadatak. Ovom zbirkom postiže se organizacija takvih materijala radi lakšeg snalaženja na kolokvijumu i ušteda u papirima potrebnim za štampanje svih tih rokova.
- Zadaci su kategorisani po oblastima i sličnosti kako bi se lakše vežbali određeni tipovi zadataka na kolokvijumu.

Poglavlja zbirke idu istim redom kao kolokvijumsko gradivo na predmetu, konkretno:

1. Kompajler/linker, stranična organizacija, kontinualna alokacija, segmentna organizacija, segmentno-stranična organizacija, dinamičko učitavanje, preklopi
2. Prekidi, sistemski pozivi, interfejs niti, promena konteksta, sinhronizacija procesa, baferi, proizvođač/potrošač
3. Ulaz/izlaz, komandna linija, fajl sistem

Raspored ovih oblasti po kolokvijumima se retko menja, ali se takva situacija može desiti (usled predviđenih ili nepredviđenih okolnosti). U tom slučaju, proverite koje oblasti dolaze na kolokvijumu sa predmetnim asistentima. Bitno je napomenuti da se ovaj redosled u prošlosti menjao, pa ako se neki zadatak ranije pojavljivao na prvom kolokvijumu to ne mora da znači da je ta oblast i tekuće godine na prvom kolokvijumu. U suštini, **da li neki zadatak može doći na kolokvijumu određujete prvenstveno na osnovu toga iz koje je oblasti, a ne na osnovu toga na kom se kolokvijumu pojavio u prošlosti.**

Greške u formatiranju i kategorizaciji su sigurno prisutne. Ukoliko ih uočite, možete se javiti jednom od autora ili poslati *pull request* na repozitorijum projekta: <https://github.com/siwiki/OS-kolokvijumi>. Ispravljeni dokumenti će biti dostupni u [Releases](#). Svaka pomoć je dobrodošla.

Srećno na kolokvijumu.

Autori

Ulaz/izlaz

1. zadatak, treći kolokvijum, jun 2022.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima kontrolera tastature preko koga stižu znakovi otkucani na tastaturi:

```
typedef volatile unsigned REG;
REG* ioStatus =...;      // status register
char* ioData =...;       // data register
```

Kontroler tastature poseduje interni memorijski modul koji služi za prihvatanje jednog ili više znakova otkucanih na tastaturi (hardverski bafer). Kada se ovom baferu pojave znakovi (jedan ili više), kontroler generiše prekid. Tada se iz registra za podatke mogu čitati pristigli znakovi sukcesivnim operacijama čitanja, jedan po jedan, sve dok je bit spremnosti (*ready*) u razredu 0 statusnog registra postavljen na 1. Naredni nalet pristiglih znakova će ponovo generisati prekid.

Za smeštanje znakova učitanih sa tastature kernel koristi svoj (softverski) ograničeni bafer veličine 256 znakova. U ovaj bafer upisuju se znakovi učitani sa kontrolera tastature sve dok u njemu ima mesta; znakovi koji ne mogu da stanu u bafer se jednostavno odbacuju. Iz ovog bafera znakove uzimaju različiti uporedni tokovi kontrole (procesi) pozivom operacije `getc`; ukoliko u baferu nema znakova, pozivajući tok kontrole treba da se suspenduje dok znakova ne bude. Sinhronizacija se može obavljati semaforima čiji je interfejs isti kao u školskom jezgru. Pretpostaviti sledeće:

- prekidna rutina izvršava se međusobno isključivo sa operacijama na semaforu;
- operacija `signal` na semaforu može se pozivati iz prekidne rutine, jer ona ne radi nikakvu promenu konteksta;
- procesor maskira prekide pri obradi prekida; prekid sa kontrolera tastature se može eksplicitno maskirati pozivom `kbint_mask()`, a demaskirati pozivom `kbint_unmask()`.

Implementirati opisani podsistem: bafer kernela (operaciju `getc` i sve druge potrebne operacije) i prekidnu rutinu kontrolera tastature.

Rešenje

```
class KeyboardBuffer {
public:
    KeyboardBuffer() : head(0), tail(0), count(0), mutex(1), itemAvailable(0) {}
    char getc();
    void putc(char c);
private:
    static const size_t KB_SIZE = 256;
    char buffer[KB_SIZE];
    size_t head, tail, count;
    Semaphore mutex, itemAvailable;
};

static KeyboardBuffer* instance() {
    static KeyboardBuffer _instance;
    return &_instance;
}

char KeyboardBuffer::getc() {
    itemAvailable.wait();
    mutex.wait();
    kbint_mask();
    char c = buffer[head];
    head = (head + 1) % KB_SIZE;
    count--;
    kbint_unmask();
```

```

    mutex.signal();
    return c;
}
void KeyboardBuffer::putc(char c) {
    if (count < KB_SIZE) {
        buffer[tail] = c;
        tail = (tail + 1) % KB_SIZE;
        count++;
        itemAvailable.signal();
    }
}
interrupt void keyb_int() {
    while (*ioStatus & 1) {
        KeyboardBuffer::instance()->putc(*ioData);
    }
}

```

3. zadatak, kolokvijum, jul 2021.

- [Postavka](#)
- [Rešenje](#)

U računaru postoji ulazni uređaj koji prihvata pakete sa neke komunikacione veze. Paket je veličine `PACKET_SIZE` (u `sizeof(char)`). Kada paket stigne u interni hardverski bafer uređaja, uređaj generiše prekid na koji je vezana rutina `packetArrived` kernela. Kernel treba da prebaci paket u svoj interni memorijski kružni bafer `packetBuffer` koji ima mesta za `BUFFER_SIZE` paketa. Ostatak kernela te pakete iz kružnog bafera kernela isporučuje dalje kome treba (procesima). Kada prebaci paket u svoj kružni bafer, kernel treba to da signalizira uređaju upisom konstante `IO_COMPLETE` u upravljački registar uređaja, kako bi ovaj znao da može da prihvati naredni paket u svoj interni bafer. Ukoliko je kružni bafer kernela pun, kernel treba uređaju da javi da paket mora da se odbaci upisom konstante `IO_REJECT` u upravljački registar uređaja. Interni hardverski bafer uređaja je veličine jednog paketa i preslikan je u memorijski prostor počev od adrese `ioData`. Date su deklaracije pokazivača preko kojih se može pristupiti registrima kontrolera ulaznog uređaja, kao i deklaracije za kružni bafer:

```

typedef volatile unsigned int REG;
REG* ioCtrl = ...; // Input device control register
volatile char* ioData = ...; // Input device packet buffer

extern const size_t PACKET_SIZE, BUFFER_SIZE;
extern char packetBuffer[BUFFER_SIZE * PACKET_SIZE];
extern char *pbHead, *pbTail;

interrupt void packetArrived();

```

Glava `pbHead` ukazuje na prvi znak paketa u kružnom baferu koji je na redu za prenos u ostatak kernela, a rep `pbTail` ukazuje na prvi znak prvog slobodnog mesta za prijem novog paketa u kružni bafer. Tokom izvršavanja prekidnih rutina drugi prekidi su maskirani i nema uporednog izvršavanja ostalog koda kernela.

Realizovati prekidnu rutinu `packetArrived`.

Rešenje

```

interrupt void packetArrived() {
    if (pbTail == pbHead) { // Buffer full, reject packet
        *ioCtrl = IO_REJECT;
    } else {
        for (int i = 0; i < PACKET_SIZE; i++) pbTail[i] = ioData[i];
        *ioControl = IO_COMPLETE;
        if ((pbTail - packetBuffer) / PACKET_SIZE == BUFFER_SIZE - 1) {
            pbTail = packetBuffer;
        }
    }
}

```

```

        } else {
            pbTail = pbTail + PACKET_SIZE;
        }
    }
}

```

3. zadatak, kolokvijum, avgust 2021.

- [Postavka](#)
- [Rešenje](#)

U računaru postoji ulazni uređaj koji prihvata pakete sa neke komunikacione veze. Paket je veličine `PACKET_SIZE` (u `sizeof(char)`). Kada paket stigne u interni hardverski bafer uređaja, uređaj generiše prekid na koji je vezana rutina `packetArrived` kernela. Kernel treba da prebaci paket u svoj interni memorijski kružni bafer `packetBuffer` koji ima mesta za

`BUFFER_SIZE` paketa. Ovo prebacivanje kernel obavlja DMA prenosom. Ostatak kernela te pakete iz kružnog bafera kernela isporučuje dalje kome treba (procesima). Kada prebaci paket u svoj kružni bafer, kernel treba to da signalizira uređaju upisom konstante `IO_COMPLETE` u upravljački registar uređaja, kako bi ovaj znao da može da prihvati naredni paket u svoj interni bafer. Ukoliko je kružni bafer kernela pun ili pri DMA prenosu dođe do greške, kernel treba uređaju da javi da paket mora da se odbaci upisom konstante `IO_REJECT` u upravljački registar uređaja. Date su deklaracije pokazivača preko kojih se može pristupiti registrima DMA kontrolera i kontrolera ulaznog uređaja, kao i deklaracije za kružni bafer:

```

typedef unsigned int REG;
REG* dmaCtrl = ...; // DMA control register
REG* dmaStatus = ...; // DMA status register
REG* dmaAddress = ...; // DMA block address register
REG* dmaCount = ...; // DMA block size register
REG* ioCtrl = ...; // Input device control register
extern const size_t PACKET_SIZE, BUFFER_SIZE;
extern char packetBuffer[BUFFER_SIZE * PACKET_SIZE];
extern char *pbHead, *pbTail;

interrupt void packetArrived ();
interrupt void dmaComplete ();

```

Greška u DMA prenosu signalizira se postavljanjem bita u statusnom registru; ovaj bit maskira se konstantom `DMA_ERROR`. DMA prenos pokreće se upisom konstante `DMA_START` u upravljački registar DMA kontrolera. Kada DMA kontroler završi prenos, generiše prekid na koji je vezana rutina `dmaComplete` kernela. Po završetku prenosa ne treba ništa upisivati u upravljački registar DMA kontrolera. Glava `pbHead` ukazuje na prvi znak paketa u kružnom baferu koji je na redu za prenos u ostatak kernela, a rep `pbTail` ukazuje na prvi znak prvog slobodnog mesta za prijem novog paketa u kružni bafer. Tokom izvršavanja prekidnih rutina drugi prekidi su maskirani i nema uporednog izvršavanja ostalog koda kernela. Realizovati prekidne rutine `packetArrived` i `dmaComplete`.

Rešenje

```

interrupt void packetArrived () {
    if (pbTail == pbHead) { // Buffer full, reject packet
        *ioCtrl = IO_REJECT;
    } else {
        *dmaAddress = pbTail;
        *dmaCount = PACKET_SIZE;
        *dmaCtrl = DMA_START;
    }
}

interrupt void dmaComplete () {
    if (*dmaStatus & DMA_ERROR) {

```

```

    *ioControl = IO_REJECT;
} else {
    *ioControl = IO_COMPLETE;
    if ((pbTail - packetBuffer) / PACKET_SIZE == BUFFER_SIZE - 1)
        pbTail = packetBuffer;
    else
        pbTail = pbTail + PACKET_SIZE;
}
}

```

3. zadatak, kolokvijum, oktobar 2020.

- Postavka
- Rešenje

Date su deklaracije pokazivača preko kojih se može pristupiti registrima jednog DMA kontrolera:

```

typedef unsigned int REG;
REG* dmaCtrl = ...; // DMA control register
REG* dmaStatus = ...; // DMA status register
REG* dmaAddress = ...; // DMA block address register
REG* dmaCount = ...; // DMA block size register

```

U upravljačkom registru najniži bit je bit *Start* kojim se pokreće prenos jednog bloka preko DMA, a u statusnom registru najniži bit je bit završetka prenosa (*TransferComplete*), a bit do njega bit greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Zahtevi za ulaznim operacijama na nekom uređaju sa kog se prenos blokova vrši preko ovog DMA kontrolera vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```

struct IORequest {
    REG* buffer; // Data buffer (block)
    unsigned int size; // Buffer (blok) size
    int status; // Status of operation
    IORequest* next; // Next in the list
};

```

Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`. Kada u praznu listu kernel stavi prvi zahtev, pozvaće operaciju `transfer` koja treba da pokrene prenos za taj prvi zahtev i pokrene DMA kontroler upisom u bit *Start*. Kada se završi prenos zadat jednim zahtevom, potrebno je u polje `status` date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška), izbaciti obrađeni zahtev iz liste i pokrenuti prenos za sledeći zapis u listi, bez zaustavljanja DMA kontrolera. Ako zahteva u listi više nema, tek onda treba zaustaviti DMA kontroler resetovanjem bita *Start*. Kada bude stavljaao novi zahtev u listu, kernel će proveriti i videti da je ona bila prazna, pa ponovo pozvati operaciju `transfer` itd.

Potrebno je napisati kod operacije `transfer()`, zajedno sa odgovarajućom prekidnom rutinom `dmaInterrupt()` za prekid od DMA kontrolera.

```

void transfer ();
interrupt void dmaInterrupt ();

```

Rešenje

```

extern IORequest* ioHead;
IORequest* pending = 0;

int startDMA () {
    pending = ioHead;
    if (pending == 0) return 0;
    ioHead = ioHead->next;
    *dmaAddress = pending->buffer;
}

```



```

    *dmaCount = pending->size;
    return 1;
}

void transfer() {
    startDMA();
    *dmaCtrl = 1; // Start I/O
}

interrupt void dmaInterrupt () {
    if (pending==0) return; // Exception
    if (*dmaStatus&2) // Error in I/O
        pending->status = -1;
    else
        pending->status = 0;
    if (startDMA()==0) { // No more requests
        *dmaCtrl = 0;
        return;
    }
}

```

3. zadatak, kolokvijum, avgust 2020.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima kontrolera jednog ulaznog uređaja:

```

typedef unsigned int REG;
REG* ioCtrl = ...; // control register
REG* ioStatus = ...; // status register
REG* ioData = ...; // data register
const unsigned BlockSize = ...; // block size

```

Uređaj je blokovski, što znači da jedna ulazna operacija uvek prenosi jedan blok iste veličine `BlockSize` reči, ali jednu po jednu reč preko registra za podatke. U upravljačkom registru najniži bit je bit *Start* kojim se pokreće prenos. Za svaki nov prenos jednog bloka potrebno je upisati 1 u bit *Start*, a na kraju prenosa bloka u taj bit upisati 0. U statusnom registru najniži bit je bit *Ready* koji signalizira spremnost jedne reči u registru za podatke, a bit do njega bit greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). U slučaju greške u prenosu, uređaj generiše isti prekid kao i u slučaju spremnosti za prenos novog podatka.

Zahtevi za ulaznim operacijama prenosa blokova podataka na ovom uređaju vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```

struct IORequest {
    REG* buffer; // Data buffer (block)
    int status; // Status of operation
    IORequest* next; // Next in the list
};

```

Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`. Kada u praznu listu kernel stavi prvi zahtev, pozvaće operaciju `transfer()` koja treba da pokrene prenos za taj prvi zahtev. Kada se završi prenos bloka zadat jednim zahtevom, potrebno je u polje `status` date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška), izbaciti obrađeni zahtev iz liste i pokrenuti prenos za sledeći zapis u listi. Ako zahteva u listi više nema, ne treba uraditi više ništa (kada bude stavljao novi zahtev u listu, kernel će proveriti i videti da je ona bila prazna, pa pozvati ponovo operaciju `transfer()` itd.)

Potrebno je napisati kod operacije `transfer()`, zajedno sa odgovarajućom prekidnom rutinom `ioInterrupt()` za prekid od uređaja, pri čemu prenos treba vršiti tehnikom programiranog ulaza-izlaza uz korišćenje prekida.

```
void transfer ();
interrupt void ioInterrupt ();
```

Rešenje

```
static REG* ptr = 0; // pointer to current data item
static int count = 0; // counter

void startIO () { // Helper: start a new transfer
    if (ioHead==0) return;
    ptr = ioHead->buffer;
    count = BlockSize;
    *ioCtrl = 1; // Start I/O
}

void transfer () {
    startIO();
}

interrupt void ioInterrupt () {
    if (*ioStatus&2)
        ioHead->status = -1; // Error in I/O
    else { // Transfer the next data item
        *ptr++ = *ioData;
        if (--count)
            return;
        else
            ioHead->status = 0; // Transfer completed successfully
    }
    *ioCtrl = 0; // Stop the transfer
    ioHead = ioHead->next; // Remove the request from the list
    startIO(); // Start a new transfer
}
```

1. zadatak, prvi kolokvijum, mart 2019.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima kontrolera jednog ulaznog uređaja:

```
typedef unsigned long REG;
REG* ioCtrl = ...; // control register
REG* ioStatus = ...; // status register
REG* ioData = ...; // data register
REG* ioBlock = ...; // block address register
const unsigned int BLOCK_SIZE = ...;
const REG START_READING = ...;
```

Sa ovog ulaznog uređaja se jednom operacijom prenosi čitav blok podataka veličine `BLOCK_SIZE`. Adresa bloka na uređaju koji treba pročitati zadaje se upisom u registar na adresi `ioBlock`. Da bi se uređaju zahtevao prenos jednog bloka podataka, potrebno je najpre zadati adresu (broj) traženog bloka na uređaju, a potom u upravljački registar upisati vrednost predstavljenu simboličkom konstantom `START_READING`. Kontroler uređaja poseduje interni memorijski modul koji služi za prihvatanje celog pročitanih bloka podataka (bafer). Kada završi dohvatanje traženog bloka u ovaj svoj interni bafer, kontroler uređaja postavlja bit u razredu 0 svog statusnog registra na 1; ovaj signal *ne* generiše prekid procesoru. U slučaju greške, bit 1 postavlja takođe na 1. Kada je pročitani blok spreman u baferu, može se pročitati jedna po jedna reč tog bafera sukcesivnim čitanjem istog registra za podatke na adresi `ioData` (ovo obezbeđuje interni hardver kontrolera: svako naredno čitanje sa ove adrese inkrementira interni adresni registar za

adresiranje unutar bafera; upis `START_READING` u upravljački registar zapravo resetuje ovaj adresni registar). Zbog toga se sukcesivne reči mogu čitati sa ove adrese bez ikakvog čekanja (bez sinhronizacije), u sukcesivnim ciklusima čitanja.

Zahtevi za ulaznim operacijama na ovom uređaju vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer; // Data buffer (data block) in memory
    REG block;   // The block number (address)
    int status;  // Status of operation
    IORequest* next; // Next in the list
};
```

Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`, a na poslednji `ioTail`. Operacijom `transfer()` kernel stavlja jedan zahtev u ovu listu i po potrebi pokreće transfer na uređaju. Kada se završi prenos zadat jednim zahtevom, potrebno je u polje `status` date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška) i pokrenuti prenos za sledeći zapis u listi, a zahtev izbaciti iz liste. Napisati kod operacije `void transfer (IORequest* request)`.

Rešenje

```
void performIO () {
    while (ioHead!=0) {

        IORequest* ioPending = ioHead; // Take the first request,
        ioHead = ioHead->next; // remove it from the list,
        if (ioHead==0) ioTail = 0;

        *ioBlock = ioPending->block; // and read it from I/O
        *ioCtrl = START_READING;

        while (*ioStatus&1==0); // Wait for completion

        if (*ioStatus&2) // Error in I/O
            ioPending->status = -1;
        else {
            ioPending->status = 0;
            for (int i=0; i<BLOCK_SIZE; i++)
                ioPending->buffer[i] = *ioData;
        }
    }
}

void transfer (IORequest* req) {
    req->next = 0;
    if (!ioHead) {
        ioHead = ioTail = req;
        performIO();
    } else
        ioTail = ioTail->next = req;
}
```

1. zadatak, prvi kolokvijum, maj 2019.

- [Postavka](#)
- [Rešenje](#)

Na jedan isti kontroler vezana su dva ulazna uređaja. Kontroler može da obavlja uporedne prenose sa ova dva

uređaja preko dva logička „kanala“. Date su deklaracije pokazivača preko kojih se može pristupiti registrima ovog kontrolera:

```
typedef volatile unsigned int REG;
REG* ioCtrl = ...; // control register
REG* ioStatus = ...; // status register
REG* ioData = ...; // two data registers
```

U (samo jednom) upravljačkom registru dva najniža bita su biti *Start* kojim se pokreće prenos na prvom, odnosno drugom kanalu. Upravljački registar vezan je tako da se može i čitati. U (samo jednom) statusnom registru dva najniža bita su biti spremnosti (*Ready*), a dva bita do njih su biti greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Kada je uređaj na jednom kanalu obavio prenos jednog podatka i spreman je za sledeći, odgovarajući bit *Ready* se postavlja na 1, a kontroler *ne* generiše prekid. U slučaju greške u prenosu, uređaj postavlja *Ready* bit kao i u slučaju završetka prenosa jednog podatka. Na dve susedne adrese počev od adrese `ioData` nalaze se dva registra za podatke, za dva kanala.

Zahtevi za ulaznim operacijama prenosa blokova podataka vezani su u jednostruko ulančanu listu. Zahtev se može opslužiti na bilo kom slobodnom kanalu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer; // Data buffer (block)
    unsigned int size; // Buffer (blok) size
    int status; // Status of operation
    IORequest* next; // Next in the list
};
```

Polja u ovoj strukturi mogu se koristiti kao promenljive tokom prenosa (ne mora se očuvati njihova početna vrednost nakon prenosa). Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`. Kada u praznu listu kernel stavi prvi zahtev ili nekoliko zahteva odjednom, pozvaće operaciju `transfer()` koja treba da pokrene prenos za prvi zahtev (ili prva dva). Kada se završi prenos zadat jednim zahtevom, potrebno je u polje `status` date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška) i pokrenuti prenos na kanalu koji je završio prenos za sledeći zapis u listi. Ako zahteva u listi više nema, ne treba uraditi više ništa (kada bude stavljao novi zahtev u listu, kernel će proveriti i videti da je ona bila prazna, pa pozvati ponovo operaciju `transfer()` itd.). Zahtev koji je opslužen ne treba više da bude u listi (ali ne treba dealocirati/brisati samu strukturu zahteva).

Potrebno je napisati kod operacije `transfer()`, pri čemu prenos treba vršiti tehnikom programiranog ulaza-izlaza prozivanjem.

```
void transfer ();
```

Rešenje

```
static IORequest* pending[2] = {0,0}; // Pending requests for two channels
```

```
void startIO (int i) { // Helper: start a new transfer with channel i
    if (ioHead==0) {
        *ioCtrl &= ~(1<<i); // Stop channel i
        pending[i] = 0;
        return;
    }
    pending[i] = iohead;
    ioHead = ioHead->next; // Remove the request from the list
    pending[i]->next = 0;
    // Start I/O with channel i:
    *ioCtrl |= (1<<i);
    return;
}
```

```
void handleIO (int i) { // Helper: handle I/O with channel i
    if (!((*ioStatus)&(1<<i))) return;
```

```

if ((*ioStatus)&(1<<(2+i)))
    pending[i]->status = -1; // Error in I/O
else { // Transfer the next data item
    *(pending[i]->buffer)++ = ioData[i];
    if (--pending[i]->size)
        return;
    else {
        pending[i]->status = 0; // Transfer completed successfully
    }
}
startIO(i); // Initiate the next transfer with this channel
}

void transfer () {
    if (!ioHead) return;
    startIO(0);
    startIO(1);
    while (pending[0] || pending[1]) {
        if (pending[0]) handleIO(0);
        if (pending[1]) handleIO(1);
    }
}

```

1. zadatak, prvi kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva ulazno/izlazna uređaja:

```

typedef volatile unsigned int REG;
REG* io1Ctrl = ...; // Device 1 control register
REG* io1Status = ...; // Device 1 status register
REG* io1Data = ...; // Device 1 data register
REG* io2Ctrl = ...; // Device 2 control register
REG* io2Status = ...; // Device 2 status register
REG* io2Data = ...; // Device 2 data register

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Potrebno je napisati funkciju `transfer` koja najpre vrši ulaz bloka podataka zadate veličine sa prvog uređaja korišćenjem tehnike prozivanja (*polling*), a potom izlaz tog istog učitano bloka podataka na drugi uređaj korišćenjem prekida, i vraća kontrolu pozivaocu tek kada se oba prenosa završe.

Rešenje

```

static REG* ioPtr = 0;
static int ioCount = 0;
static int ioCompleted = 0;

void transfer (int count) {
    REG* buffer = new REG[count];

    // I/O 1
    ioPtr = buffer;
    ioCount = count;
    *io1Ctrl = 1; // Start I/O 1
    while (ioCount>0) {
        while (!(*io1Status&1)); // busy wait

```

```

    *ioPtr++ = *io1Data;
    ioCount--;
}
*io1Ctrl = 0; // Stop I/O 1

// I/O 2:
ioPtr = buffer;
ioCount = count;
ioCompleted = 0;
*io2Ctrl = 1; // Start I/O 2

// Wait for I/O 2 completion:
while (!ioCompleted);

delete [] buffer;
}

interrupt void io2Interrupt() {
    *io2Data = *ioPtr++;
    if (--ioCount == 0) {
        ioCompleted = 1;
        *io2Ctrl = 0; // Stop I/O 2
    }
}
}

```

1. zadatak, prvi kolokvijum, mart 2018.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva ulazna uređaja:

```

typedef unsigned REG;
REG* io1Ctrl = ...; // Device 1 control register
REG* io1Data = ...; // Device 1 data register
REG* io1Stat = ...; // Device 1 status register
REG* io2Ctrl = ...; // Device 2 control register
REG* io2Data = ...; // Device 2 data register
REG* io2Stat = ...; // Device 2 status register
REG* timer = ...; // Timer

```

U upravljačkim registrima bit 0 je bit *Start*. Spremnost novog ulaznog podatka prvi uređaj ne signalizira nikakvim signalom, ali je sigurno da je on spreman najkasnije 50 ms nakon preuzimanja prethodnog podatka (čitanja podatka iz registra), odnosno od postavljanja bita *Start*, ako se radi o prvom podatku. Spremnost novog ulaznog podatka drugi uređaj signalizira bitom spremnosti u razredu 0 svog statusnog registra, čije postavljanje ne generiše prekid procesoru.

Na magistralu računara vezan je i registar posebnog uređaja, vremenskog brojača (tajmera). Upisom celobrojne vrednosti n u ovaj registar vremenski brojač počinje merenje vremena od n milisekundi, nakon isteka tog vremena, generiše prekid procesoru.

Potrebno je napisati kod, uključujući i prekidnu rutinu od vremenskog brojača, koji vrši prenos po jednog bloka podataka sa svakog od dva uređaja uporedo. Prenos se obavlja pozivom sledeće funkcije iz koje se vraća kada su oba prenosa završena:

```

void transfer (unsigned* blk1, int count1, unsigned* blk2, int count2);

```

Rešenje

```

static unsigned *io1Ptr = 0, *io2Ptr = 0;
static int io1Count = 0, io2Count = 0;
static const unsigned timeout = 50;

void transfer (unsigned* blk1, int count1, unsigned* blk2, int count2) {
    // I/O 1:
    io1Ptr = blk1;
    io1Count = count1;
    *io1Ctrl = 1; // Start I/O 1
    *timer = timeout; // Start timer

    // I/O 2:
    io2Ptr = blk2;
    *io2Ctrl = 1; // Start I/O 2
    for (io2Count=count2; io2Count>0; io2Count--) {
        while (*io2Stat&1 == 0); // Busy-wait for I/O 2 to be ready
        *io2Ptr++ = *io2Data;
    }
    *io2Ctrl = 0; // Stop I/O 2

    // Busy wait for I/O 1 completion:
    while (io1Count);
}

interrupt void timerInterrupt () {
    *io1Ptr++ = *io1Data;
    if (--io1Count)
        *timer = timeout; // Restart timer
    else
        *io1Ctrl = 0; // Stop I/O 1
}

```

1. zadatak, prvi kolokvijum, april 2018.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima četiri uređaja; prva dva uređaja su ulazni, a druga dva su izlazni:

```

typedef unsigned int REG;
REG* io1Ctrl =...; // Device 1 control register
REG* io1Status =...; // Device 1 status register
REG* io1Data =...; // Device 1 data register
REG* io2Ctrl =...; // Device 2 control register
REG* io2Status =...; // Device 2 status register
REG* io2Data =...; // Device 2 data register
REG* io3Ctrl =...; // Device 3 control register
REG* io3Status =...; // Device 3 status register
REG* io3Data =...; // Device 3 data register
REG* io4Ctrl =...; // Device 4 control register
REG* io4Status =...; // Device 4 status register
REG* io4Data =...; // Device 4 data register

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Samo uređaj broj 2 generiše prekid kada je spreman.

Potrebno je napisati proceduru `transfer` i odgovarajuću prekidnu rutinu za prekid sa uređaja broj 2 koja učitava po jednu reč sa uređaja 1 i tu reč odmah prebacuje na uređaj 3, odnosno učitava po jednu reč sa uređaja 2 i tu reč odmah prebacuje na uređaj 4. Ova dva prenosa se obavljaju uporedno, sve dok se sa bilo kog ulaznog uređaja ne učitava vrednost 0. Prenose sa uređajima 1, 3 i 4 vršiti programiranim ulazom/izlazom sa prozivanjem (*polling*), a sa uređajem 2 korišćenjem prekida. Smatrati da je periferni uređaj 4 veoma brz, tj. da vrlo brzo (reda vremena izvršavanja nekoliko desetina instrukcija procesora) od prijema podatka biva spreman za prijem novog podatka.

Rešenje

```
const REG ESC = 0;
bool complete = false;

void transfer () {
    REG data = ESC;

    *io1Ctrl = 1;
    *io2Ctrl = 1;
    *io3Ctrl = 1;
    *io4Ctrl = 1;

    while (!complete) {
        while (*io1Status & 1 == 0);
        data = *io1Data;

        if (data == ESC) break;

        while (*io3Status & 1 == 0);
        *io3Data = data;
    }

    complete = true;
    *io1Ctrl = 0;
    *io2Ctrl = 0;
    *io3Ctrl = 0;
    *io4Ctrl = 0;
}

interrupt void device2 () {
    if (complete) return;
    REG data = *io2Data;
    if (data != ESC) {
        while (*io4Status & 1 == 0);
        *io4Data = data;
    } else {
        complete = true;
        *io1Ctrl = 0;
        *io2Ctrl = 0;
        *io3Ctrl = 0;
        *io4Ctrl = 0;
    }
}
```

1. zadatak, prvi kolokvijum, jun 2018.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima tri uređaja; prvi uređaja je ulazni, a druga dva su izlazni:

```
typedef unsigned int REG;
REG* io1Ctrl = ...; // Device 1 control register
REG* io1Status = ...; // Device 1 status register
REG* io1Data = ...; // Device 1 data register
REG* io2Ctrl = ...; // Device 2 control register
REG* io2Status = ...; // Device 2 status register
REG* io2Data = ...; // Device 2 data register
REG* io3Ctrl = ...; // Device 3 control register
REG* io3Status = ...; // Device 3 status register
REG* io3Data = ...; // Device 3 data register
```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Potrebno je napisati potprogram `transfer` koji učitava po jednu reč sa ulaznog uređaja i odmah tu učitane reč šalje na bilo koji od ova dva izlazna uređaja, koji god od njih je pre spreman. Ovaj prenos se obavlja sve dok se sa ulaznog uređaja ne učitava vrednost 0. Sve prenose vršiti programiranim ulazom/izlazom sa prozivanjem (*polling*).

Rešenje

```
const REG ESC = 0;

void transfer() {
    REG data = ESC;

    *io1Ctrl = 1;
    *io2Ctrl = 1;
    *io3Ctrl = 1;

    while (1) {
        while (*io1Status == 0);
        data = *io1Data;

        if (data == ESC) break;

        int done = 0;
        while (!done) {
            if (*io2Status != 0) {
                *io2Data = data;
                done = 1;
            } else if (*io3Status != 0) {
                *io3Data = data;
                done = 1;
            }
        }
    }

    *io1Ctrl = 0;
    *io2Ctrl = 0;
    *io3Ctrl = 0;
}
```

1. zadatak, prvi kolokvijum, mart 2017.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima kontrolera jednog izlaznog uređaja:

```
typedef unsigned int REG;
REG* ioCtrl = ...;      // control register
REG* ioStatus = ...;    // status register
REG* ioData = ...;      // data register
const unsigned int BLOCK_SIZE = ...;
const REG START_SENDING = ..., END_SENDING = ...;
```

Na ovaj izlazni uređaj se jednom operacijom prenosi čitav blok podataka veličine `BLOCK_SIZE`. Da bi se uređaju zadao prenos jednog bloka podataka, potrebno je najpre u upravljački registar upisati vrednost predstavljenu simboličkom konstantom `START_SENDING`. Potom treba jednu po jednu rem upisivati na istu adresu registra za podatke (`ioData`). Kontroler uređaja poseduje memorijski modul koji služi za prihvatanje celog bloka podataka (bafer), tako da se reči upisane redom na adresu registra za podatke upisuju redom u ovaj bafer (ovo obezbeđuje interni hardver kontrolera: svaki naredni upis na ovu adresu inkrementira interni adresni registar za adresiranje unutar bafera; upis `START_SENDING` u upravljački registar zapravo resetuje ovaj adresni registar). Zbog toga se sukcesivne reči mogu upisivati na ovu adresu bez ikakvog čekanja (bez sinhronizacije), u sukcesivnim ciklusima upisa. Kada se završi upis celog bloka podataka, potrebno je u upravljački registar upisati vrednost predstavljenu simboličkom konstantom `END_SENDING`. Ovim se kontroleru nalaže da započne izlaznu operaciju zadatog bloka podataka. Kada završi izlaznu operaciju prenosa celog bloka podataka, kontroler periferije postavlja bit u razredu 0 statusnog registra, ali ne generiše prekid procesoru. U statusnom registru bit 1 je tada bit greške u prenosu (*Error*).

Zahtevi za izlaznim operacijama na ovom uređaju vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer; // Data buffer (data block)
    int status;  // Status of operation
    IORequest* next; // Next in the list
};
```

Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`, a na poslednji `ioTail`. Operacijom `transfer()` kernel stavlja jedan zahtev u ovu listu i po potrebi pokreće transfer na uređaju. Kada se završi prenos zadat jednim zahtevom, potrebno je u polje `status` date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška) i pokrenuti prenos za sledeći zapis u listi, a zahtev izbaciti iz liste.

Napisati kod operacije `void transfer (IORequest* request)`.

Rešenje

```
void performIO () {
    while (ioHead!=0) {

        IORequest* ioPending = ioHead; // Take the first request,
        ioHead = ioHead->next; // remove it from the list,
        if (ioHead==0) ioTail = 0;

        *ioCtrl = START_SENDING; // and send it to I/O
        for (int i=0; i<BLOCK_SIZE; i++)
            *ioData = ioPending->buffer[i];
        *ioCtrl = END_SENDING;

        while (*ioStatus&1 == 0); // Wait for completion

        if (*ioStatus&2) // Error in I/O
            ioPending->status = -1;
        else
            ioPending->status = 0;
    }
}
```

```

}

void transfer (IORequest* req) {
    req->next = 0;
    if (!ioHead) {
        ioHead = ioTail = req;
        performIO();
    } else
        ioTail = ioTail->next = req;
}

```

1. zadatak, prvi kolokvijum, april 2017.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva izlazna uređaja:

```

typedef unsigned int REG;
REG* io1Ctrl = ...;    // Device 1 control register
REG* io1Data = ...;    // Device 1 data register
REG* io2Ctrl = ...;    // Device 2 control register
REG* io2Data = ...;    // Device 2 data register
REG* timer = ...;      // Timer

```

U upravljačkim registrima bit 0 je bit *Start*. Prenos svakog pojedinačnog podatka na svaki od ovih uređaja zahteva se upisom podatka u registar podataka tog uređaja. Spremnost registra za podatke za prihvatanje novog izlaznog podatka prvi uređaj ne signalizira nikakvim signalom, ali je sigurno da je on spreman za novi najkasnije 50 ms nakon zadatog prethodnog zahteva (upisa podatka u registar). Spremnost registra za podatke za prihvatanje novog izlaznog podatka drugi uređaj signalizira prekidom.

Na magistralu računara vezan je i registar posebnog uređaja, vremenskog brojača. Upisom celobrojne vrednosti n u ovaj registar vremenski brojač počinje merenje vremena od n milisekundi, nakon isteka tog vremena, generiše prekid procesoru.

Potrebno je napisati kod, uključujući i obe prekidne rutine (od drugog uređaja i tajmera), koji vrši prenos po jednog bloka podataka na svaki od dva uređaja uporedo. Prenos se obavlja pozivom sledeće funkcije iz koje se vraća kada su oba prenosa završena:

```
void transfer (unsigned* blk1, int count1, unsigned* blk2, int count2);
```

Rešenje

```

static unsigned *io1Ptr = 0, *io2Ptr = 0;
static int io1Count = 0, io2Count = 0;
static unsigned timeout = 50;
void transfer (unsigned* blk1, int count1, unsigned* blk2, int count2) {
    // I/O 1:
    io1Ptr = blk1;
    io1Count = count1;
    *io1Ctrl = 1; // Start I/O 1
    *io1Data = *io1Ptr++;
    *timer = timeout; // Start timer
    // I/O 2:
    io2Ptr = blk2;
    io2Count = count2;
    *io2Ctrl = 1; // Start I/O 2
    *io2Data = *io2Ptr++;

    // Busy wait for I/O completion:

```

```

    while (io1Count || io2Count);
}
interrupt void io2Interrupt() {
    if (--io2Count)
        *io2Data = *io1Ptr++; // New output request
    else
        *io2Ctrl = 0; // Stop I/O 2
}
interrupt void timerInterrupt () {
    if (--io1Count) {
        *io1Data = *io1Ptr++; // New output request
        *timer = timeout; // Restart timer
    } else
        *io1Ctrl = 0; // Stop I/O 1
}

```

1. zadatak, prvi kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima jednog kontrolera mrežne kartice, kao i jednog DMA kontrolera koji je direktno spregnut sa tom mrežnom karticom i može prenositi paket veličine PKT_SIZE primljen sa mreže u memoriju:

```

typedef volatile unsigned int REG;
REG* ioNetCtrl = ...; // Network device control register
REG* ioNetStatus = ...; // Network device status register
REG* ioNetData = ...; // Network device data register
REG* dmaCtrl = ...; // DMA control register
REG* dmaStatus = ...; // DMA status register
REG* dmaAddr = ...; // DMA buffer address register
REG* dmaCount = ...; // DMA buffer size register
const int BUF_SIZE = ..., PKT_SIZE = ...;
char buffer[BUF_SIZE][PKT_SIZE];
int bufHead, bufTail;

```

Kada stigne jedan paket sa mreže, mrežna kartica generiše prekid procesoru. Tada treba pokrenuti transfer tog paketa sa mrežne kartice u memoriju, korišćenjem DMA kontrolera. DMA kontroler se pokreće upisom konstante DMA_START u njegov upravljači registar. Paket veličine PKT_SIZE treba smestiti na mesto na koje ukazuje indeks bufTail u kružnom baferu buffer. Ako je bafer pun, što se vidi tako da je bufTail==bufHead, mrežnoj kartici treba javiti da primljeni paket treba da odbaci (tj. da prijemnoj strani treba da javi da prijem nije uspeo), što se postiže upisom konstante PKT_REJECT u upravljački registar mrežne kartice. Napisati kod prekidnih rutina za prekide sa mrežne kartice i DMA kontrolera.

Rešenje

```

interrupt void intNet () {
    if (bufTail==bufHead) {
        // Buffer full, reject the packet:
        *ioNetCtrl = PKT_REJECT;
        return;
    }
    *dmaAddr = &buffer[bufTail];
    *dmaCount = PKT_SIZE;
    *dmaCtrl = DMA_START;
}

```

```
interrupt void intDMA () {
    ++bufTail %= BUF_SIZE;
}
```

1. zadatak, prvi kolokvijum, mart 2016.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva DMA kontrolera:

```
typedef unsigned int REG;
REG* dma1Ctrl =...;    // DMA1 control register
REG* dma1Status =...;  // DMA1 status register
REG* dma2Ctrl =...;    // DMA2 control register
REG* dma2Status =...;  // DMA2 status register
```

Inicijalno, svaki DMA kontroler je u *pasivnom* stanju. Prvi upis u upravljački registar u pasivnom stanju tumači se kao upis adrese bloka podataka za prenos. Naredni upis u isti registar tumači se kao upis veličine bloka podataka za prenos. Naredni upis nenulte vrednosti u ovaj registar tumači se kao pokretanje prenosa, čime se kontroler prebacuje u *aktivno* stanje. Kada završi prenos, bilo ispravno ili u slu čaju greške, DMA kontroler zaustavlja prenos, generiše prekid procesoru i prebacuje se ponovo u pasivno stanje. U statusnom registru najniži bit je bit završetka transfera (*TransferComplete*), a bit do njega bit greške (*Error*). Završetak prenosa sa bilo kog DMA kontrolera postavlja njegov bit *TransferComplete* i generiše isti zahtev za prekid procesoru (signali završetka operacije sa dva DMA kontrolera vezani su na ulazni zahtev za prekid preko OR logičkog kola). Zahtevi za ulaznim operacijama na nekom uređaju sa kog se prenos blokova vrši preko bilo kog od ova DMA kontrolera vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer; // Data buffer (block)
    unsigned int size; // Buffer (blok) size
    int status; // Status of operation
    IORequest* next; // Next in the list
};
```

Na prvi zahtev u listi pokazuje globalni pokazivač *ioHead*, a na poslednji *ioTail*. Operacijom *transfer()* kernel stavlja jedan zahtev u ovu listu i po potrebi pokreće transfer na bilo kom trenutno slobodnom DMA kontroleru. Kada se završi prenos zadat jednim zahtevom na jednom DMA kontroleru, potrebno je u polje status date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška) i pokrenuti prenos za sledeći zapis u listi na tom DMA kontroleru, a zahtev izbaciti iz liste. Obratiti pažnju na to da oba DMA kontrolera mogu završiti prenos i generisati prekid istovremeno. Potrebno je napisati kod operacije *transfer()*, zajedno sa odgovarajućom prekidnom rutinom *dmaInterrupt()* za prekid od DMA kontrolera.

```
void transfer (IORequest* request);
interrupt void dmaInterrupt ();
```

Rešenje

```
IORequest *dma1Pending = 0, *dma2Pending = 0; // Currently pending requests
```

```
void startDMA1 () {
    if (ioHead!=0 && dma1Pending==0) {
        dma1Pending = ioHead; // Take the first request,
        ioHead = ioHead->next; // remove it from the list
        if (ioHead==0) ioTail = 0;
        *dma1Ctrl = dma1Pending->buffer; // and assign it to DMA1
        *dma1Ctrl = dma1Pending->size;
        *dma1Ctrl = 1; // Start I/O
    }
}
```

```

}

void startDMA2 () {
    if (ioHead!=0 && dma2Pending==0) {
        dma2Pending = ioHead; // Take the first request,
        ioHead = ioHead->next; // remove it from the list
        if (ioHead==0) ioTail = 0;
        *dma2Ctrl = dma2Pending->buffer; // and assign it to DMA2
        *dma2Ctrl = dma2Pending->size;
        *dma2Ctrl = 1; // Start I/O
    }
}

void transfer (IORequest* req) {
    req->next = 0;
    if (!ioHead) {
        ioHead = ioTail = req;
        startDMA1();
        startDMA2();
    } else
        ioTail = ioTail->next = req;
}

interrupt void dmaInterrupt () {
    if (*dma1Status&1) { // DMA1 transfer complete
        if (dma1Pending==0) return; // Exception
        if (*dma1Status&2) // Error in I/O
            dma1Pending->status = -1;
        else
            dma1Pending->status = 0;
        dma1Pending = 0;
        startDMA1();
    }
    if (*dma2Status&1) { // DMA2 transfer complete
        if (dma2Pending==0) return; // Exception
        if (*dma2Status&2) // Error in I/O
            dma2Pending->status = -1;
        else
            dma2Pending->status = 0;
        dma2Pending = 0;
        startDMA2();
    }
}

```

1. zadatak, prvi kolokvijum, maj 2016.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima kontrolera jednog izlaznog uređaja:

```

typedef unsigned int REG;
REG* ioCtrl = ...; // control register
REG* ioStatus = ...; // status register
REG* ioData = ...; // data register
const unsigned int BLOCK_SIZE = ...;
const REG START_SENDING = ..., END_SENDING = ...;

```

Na ovaj izlazni uređaj se jednom operacijom prenosi čitav blok podataka veličine `BLOCK_SIZE`. Da bi se uređaju zadao prenos jednog bloka podataka, potrebno je najpre u upravljački registar upisati vrednost predstavljenu simboličkom konstantom `START_SENDING`. Potom treba jednu po jednu rem upisivati na istu adresu registra za podatke (`ioData`). Kontroler uređaja poseduje memorijski modul koji služi za prihvatanje celog bloka podataka (bafer), tako da se reči upisane redom na adresu registra za podatke upisuju redom u ovaj bafer (ovo obezbeđuje interni hardver kontrolera: svaki naredni upis na ovu adresu inkrementira interni adresni registar za adresiranje unutar bafera; upis `START_SENDING` u upravljački registar zapravo resetuje ovaj adresni registar). Zbog toga se sukcesivne reči mogu upisivati na ovu adresu bez ikakvog čekanja (bez sinhronizacije), u sukcesivnim ciklusima upisa. Kada se završi upis celog bloka podataka, potrebno je u upravljački registar upisati vrednost predstavljenu simboličkom konstantom `END_SENDING`. Ovim se kontroleru nalaže da započne izlaznu operaciju zadanog bloka podataka.

Kada završi izlaznu operaciju prenosa celog bloka podataka, kontroler periferije generiše prekid procesoru. U statusnom registru najniži bit je tada bit greške u prenosu (*Error*).

Zahtevi za izlaznim operacijama na ovom uređaju vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer; // Data buffer (data block)
    int status; // Status of operation
    IORequest* next; // Next in the list
};
```

Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`, a na poslednji `ioTail`. Operacijom `transfer()` kernel stavlja jedan zahtev u ovu listu i po potrebi pokreće transfer na uređaju. Kada se završi prenos zadan jednim zahtevom, potrebno je u polje status date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška) i pokrenuti prenos za sledeći zapis u listi, a zahtev izbaciti iz liste. Potrebno je napisati kod operacije `transfer()`, zajedno sa odgovarajućom prekidnom rutinom `ioInterrupt()` za prekid od kontrolera uređaja.

```
void transfer (IORequest* request);
interrupt void ioInterrupt ();
```

Rešenje

```
IORequest* ioPending = 0; // Currently pending request
```

```
void performIO () {
    if (ioHead!=0 && ioPending==0) {
        ioPending = ioHead; // Take the first request,
        ioHead = ioHead->next; // remove it from the list,
        if (ioHead==0) ioTail = 0;
        *ioCtrl = START_SENDING; // and send it to I/O
        for (int i=0; i<BLOCK_SIZE; i++)
            *ioData = ioPending->buffer[i];
        *ioCtrl = END_SENDING;
    }
}
```

```
void transfer (IORequest* req) {
    req->next = 0;
    if (!ioHead) {
        ioHead = ioTail = req;
        performIO();
    } else
        ioTail = ioTail->next = req;
}
```

```
interrupt void ioInterrupt () {
    if (ioPending==0) return; // Exception
```

```

if (*ioStatus&1) // Error in I/O
    ioPending->status = -1;
else
    ioPending->status = 0;
ioPending = 0;
performIO();
}

```

1. zadatak, prvi kolokvijum, septembar 2016.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima jednog kontrolera izlaznog uređaja:

```

typedef unsigned int REG;
REG* ioCtrl = ...; // Output device control register
REG* ioData = ...; // Output device data register
const int BUFSIZE = ..., DELAY = ...;
REG buffer[BUFSIZE];

```

U upravljačkom registru najniži bit je bit *Start* kojim se pokreće uređaj. Statusni registar nije programski dostupan, niti kontroler signalizira spremnost na bilo koji način (npr. prekidom), ali je poznato da će, nakon zadavanja prenosa jednog podatka, kontroler sigurno završiti operaciju i biti spreman za novu nakon *DELAY* vremena.

Na jeziku C napisati kod operacije `transfer()` koja vrši prenos bloka podataka iz bafera `buffer` na dati izlazni uređaj. Ova operacija poziva se iz konteksta kernel niti. U kernelu je dostupna operacija `sleep(int)` koja uspavljuje (suspenduje) pozivajuću nit na zadato vreme. Sve eventualne greške ignorisati.

Rešenje

```

void transfer () {
    // Start the controller:
    *ioCtrl = 1;

    // Perform the output transfer:
    for (int i=0; i<BUFSIZE; i++) {
        sleep(DELAY); // Sleep-wait
        *ioData = buffer[i]; // Write data
    }

    // Stop the controller:
    *ioCtrl = 0;
}

```

1. zadatak, prvi kolokvijum, mart 2015.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima kontrolera jednog ulazno-izlaznog uređaja:

```

typedef unsigned int REG;
REG* ioCtrl = ...; // control register
REG* ioStatus = ...; // status register
REG* ioData = ...; // data register

```

U upravljačkom registru najniži bit je bit *Start* kojim se pokreće prenos. a bit do njega definiše smer prenosa podataka (0-ulaz. 1-izlaz). U statusnom registru najniži bit je bit spremnosti (*Ready*), a bit do njega bit greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). U slučaju greške u prenosu. uređaj generiše isti prekid kao i u slučaju spremnosti za prenos novog podatka.

Zahtevi za ulaznim i izlaznim operacijama prenosa blokova podataka na ovom uređaju vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer; // Data buffer (block)
    unsigned int size; // Buffer (blok) size
    int dir; // Transfer direction: 0-input, 1-output
    int status; // Status of operation
    IORequest* next; // Next in the list
};
```

Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`. Kada u praznu listu kernel stavi prvi zahtev, pozvaće operaciju `transfer()` koja treba da pokrene prenos za taj prvi zahtev. Kada se završi prenos zadat jednim zahtevom, potrebno je u polje status date strukture preneti status završene operacije (0 – ispravno završeno do kraja. -1 – greška). izbaciti obrađeni zahtev iz liste i pokrenuti prenos za sledeći zapis u listi. Ako zahteva u listi više nema, ne treba uraditi više ništa (kada bude stavljaao novi zahtev u listu, kernel će proveriti i videti da je ona bila prazna, pa pozvati ponovo operaciju `transfer()` itd.)

Potrebno je napisati kod operacije `transfer()`. zajedno sa odgovarajućom prekidnom rutinom `ioInterrupt()` za prekid od uređaja, pri čemu prenos treba vršiti tehnikom programiranog ulaza-izlaza uz korišćenje prekida.

```
void transfer ();
interrupt void ioInterrupt ();
```

Rešenje

```
static int dir = 0; // current transfer direction
static REG* ptr = 0; // pointer to current data item
static int count = 0; // counter

void startIO () { // Helper: start a new transfer
    ptr = ioHead->buffer;
    count = ioHead->size;
    dir = ioHead->dir;
    *ioCtrl = 1 | (dir<<1); // Start I/O
}

void transfer () {
    if (ioHead==0) return; // Exception - no requests
    startIO();
}

interrupt void ioInterrupt () {
    if (*ioStatus&2)
        ioHead->status = -1; // Error in I/O
    else { // Transfer the next data item
        if (dir)
            *ioData = *ptr++;
        else
            *ptr++ = *ioData;
        if (--count)
            return;
        else
            ioHead->status = 0; // Transfer completed successfully
    }

    ioHead = ioHead->next; // Remove the request from the list
    if (ioHead==0)
        *ioCtrl = 0; // No more requests
}
```

```

else
    startIO(); // Start a new transfer
}

```

1. zadatak, prvi kolokvijum, maj 2015.

- [Postavka](#)
- [Rešenje](#)

Na jedan isti ulazno-izlazni kontroler vezana su dva ulazno-izlazna uređaja. Kontroler može da obavlja uporedne prenose sa ova dva uređaja preko dva logička „kanala“. Date su deklaracije pokazivača preko kojih se može pristupiti registrima ovog kontrolera:

```

typedef unsigned int REG;
REG* ioCtrl = ...; // control register
REG* ioStatus = ...; // status register
REG* ioData = ...; // two data registers

```

U (samo jednom) upravljačkom registru dva najniža bita su biti *Start* kojim se pokreće prenos na prvom, odnosno drugom kanalu, a dva naredna bita definišu smer prenosa podataka na ova dva kanala (0-ulaz, 1-izlaz). Upravljački registar vezan je tako da se može i čitati. U (samo jednom) statusnom registru dva najniža bita su biti spremnosti (*Ready*), a dva bita do njih su biti greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Kada je uređaj na jednom kanalu obavio prenos jednog podatka i spreman je za sledeći, odgovarajući bit *Ready* se postavlja na 1, a kontroler generiše prekid (isti prekid za oba kanala). U slučaju greške u prenosu, uređaj generiše isti prekid kao i u slučaju završetka prenosa jednog podatka. Na dve susedne adrese počev od adrese `ioData` nalaze se dva registra za podatke, za dva kanala.

Zahtevi za ulaznim i izlaznim operacijama prenosa blokova podataka vezani su u jednostruko ulančanu listu. Zahtev se može opslužiti na bilo kom slobodnom kanalu. Zahtev ima sledeću strukturu:

```

struct IORequest {
    REG* buffer; // Data buffer (block)
    unsigned int size; // Buffer (blok) size
    int dir; // Transfer direction: 0-input, 1-output
    int status; // Status of operation
    IORequest* next; // Next in the list
};

```

Polja u ovoj strukturi mogu se koristiti kao promenljive tokom prenosa (ne mora se očuvati njihova početna vrednost nakon prenosa). Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`. Kada u praznu listu kernel stavi prvi zahtev ili nekoliko zahteva odjednom, pozvaće operaciju `transfer()` koja treba da pokrene prenos za prvi zahtev (ili prva dva). Kada se završi prenos zadat jednim zahtevom, potrebno je u polje `status` date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška) i pokrenuti prenos na kanalu koji je završio prenos za sledeći zapis u listi. Ako zahteva u listi više nema, ne treba uraditi više ništa (kada bude stavljao novi zahtev u listu, kernel će proveriti i videti da je ona bila prazna, pa pozvati ponovo operaciju `transfer()` itd.). Zahtev koji je opslužen ne treba više da bude u listi (ali ne treba brisati samu strukturu zahteva).

Potrebno je napisati kod operacije `transfer()`, zajedno sa odgovarajućom prekidnom rutinom `ioInterrupt()` za prekid od kontrolera, pri čemu prenos treba vršiti tehnikom programiranog ulaza-izlaza uz korišćenje prekida.

```

void transfer ();
interrupt void ioInterrupt ();

```

Rešenje

```

static IORequest* pending[2] = {0,0}; // Pending requests for two channels

void startIO (int i) { // Helper: start a new transfer with channel i
    if (ioHead==0) {
        *ioCtrl &= ~(1<<i); // Stop channel i
        return;
    }
}

```

```

}
pending[i] = iohead;
ioHead = ioHead->next; // Remove the request from the list
// Start I/O with channel i:
if (pending[i]->dir)
    *ioCtrl |= 1<<(2+i);
else
    *ioCtrl &= ~(1<<(2+i));
*ioCtrl |= (1<<i);
}

void handleInterrupt (int i) { // Helper: handle interrupt from channel i
    if ((*ioStatus)&(1<<(2+i)))
        pending[i]->status = -1; // Error in I/O
    else { // Transfer the next data item
        if (pending[i]->dir)
            ioData[i] = *(pending[i]->buffer)++;
        else
            *(pending[i]->buffer)++ = ioData[i];
        if (--pending[i]->size)
            return;
        else
            pending[i]->status = 0; // Transfer completed successfully
    }
    startIO(i); // Initiate the next transfer with this channel
}

void transfer () {
    startIO(0);
    startIO(1);
}

interrupt void ioInterrupt () {
    if (*ioStatus & 1)
        handleInterrupt(0);
    else
        handleInterrupt(1);
}

```

1. zadatak, prvi kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima jednog kontrolera ulaznog uređaja i jednog DMA kontrolera koji vrši prenos na jedan izlazni uređaj:

```

typedef unsigned int REG;
REG* ioCtrl = ...; // Input device control register
REG* ioStatus = ...; // Input device status register
REG* ioData = ...; // Input device data register
REG* dmaCtrl = ...; // DMA control register
REG* dmaStatus = ...; // DMA status register
REG* dmaAddr = ...; // DMA buffer address register
REG* dmaCount = ...; // DMA buffer size register
const int BUFSIZE;
REG buffer[2][BUFSIZE];

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Na jeziku C napisati kod operacije `transfer()` i prekidne rutine `dmaInterrupt()` za prekid sa DMA koje vrše prenos blokova podataka sa datog ulaznog uređaja tehnikom prozivanja (*polling*) na dati izlazni uređaj korišćenjem DMA na sledeći način. Data su dva memorijska bafera za smeštanje blokova podataka, svaki veličine `BUFSIZE` (nizovi `buffer[2]`). Prvi blok podataka treba učitavati u prvi niz (`buffer[0]`), reč po reč, tehnikom prozivanja. Kada se ovaj bafer napuni, treba pokrenuti izlaznu operaciju za podatke iz ovog bafera preko DMA kontrolera, a ulaznu operaciju nastaviti uporednim učitavanjem u drugi niz (`buffer[1]`), čime ova dva bafera zamenjuju uloge. Kada se i ovo učitavanje završi, treba ponovo učitavati u prvi, a izlaz vršiti iz drugog bafera, odnosno zameniti uloge bafera itd. u nedogled. Pretpostaviti da izlazni prenos jednog celog bloka podataka preko DMA traje znatno kraće nego ulazni prenos jednog bloka, tako da će izlaz iz jednog bafera sigurno biti završen pre nego što se uporedni ulaz u drugi bafer završi. Sve eventualne greške ignorisati.

Rešenje

```
void transfer () {
    int curBuffer = 0;
    // Start input controller:
    *ioCtrl = 1;

    while () {

        // Perform input transfer:
        for (int i=0; i<BUFSIZE; i++) {
            while (!(*ioStatus&1)); // Busy wait
            buffer[curBuffer][i] = *ioData; // Read data
        }

        // Start DMA output:
        *dmaAddr = buffer[curBuffer];
        *dmaCount = BUFSIZE;
        *dmaCtrl = 1;
        // Swap the buffers:
        curBuffer = 1-curBuffer;
    }
}

interrupt void dmaInterrupt () {
    *dmaCtrl = 0;
}
```

1. zadatak, prvi kolokvijum, mart 2014.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima jednog DMA kontrolera:

```
typedef unsigned int REG;
REG* dmaCtrl =...; // DMA control register
REG* dmaStatus =...; // DMA status register
REG* dmaAddress =...; // DMA block address register
REG* dmaCount =...; // DMA block size register
```

U upravljačkom registru najniži bit je bit *Start* kojim se pokreće prenos jednog bloka preko DMA, a u statusnom registru najniži bit je bit završetka prenosa (*TransferComplete*), a bit do njega bit greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Zahtevi za ulaznim operacijama na nekom uređaju sa kog se prenos blokova vrši preko ovog DMA kontrolera vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer; // Data buffer (block)
    unsigned int size; // Buffer (blok) size
    int status; // Status of operation
    IORequest* next; // Next in the list
};
```

Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`. Kada u praznu listu kernel stavi prvi zahtev, pozvaće operaciju `transfer()` koja treba da pokrene prenos za taj prvi zahtev. Kada se završi prenos zadat jednim zahtevom, potrebno je u polje `status` date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška), izbaciti obrađeni zahtev iz liste i pokrenuti prenos za sledeći zapis u listi. Ako zahteva u listi više nema, ne treba uraditi više ništa (kada bude stavljao novi zahtev u listu, kernel će proveriti i videti da je ona bila prazna, pa pozvati ponovo operaciju `transfer()` itd.) Potrebno je napisati kod operacije `transfer()`, zajedno sa odgovarajućom prekidnom rutinom `dmaInterrupt()` za prekid od DMA kontrolera.

```
void transfer ();
interrupt void dmaInterrupt ();
```

Rešenje

```
void transfer () {
    if (ioHead==0) return; // Exception
    *dmaAddress = ioHead->buffer;
    *dmaCount = ioHead->size;
    *dmaCtrl = 1; // Start I/O
}

interrupt void dmaInterrupt () {
    if (ioHead==0) return; // Exception
    if (*dmaStatus&2) // Error in I/O
        ioHead->status = -1;
    else
        ioHead->status = 0;
    ioHead = ioHead->next; // Remove the request from the list
    if (ioHead==0) { // No more requests
        *dmaCtrl = 0;
        return;
    }
    // Start a new transfer
    *dmaAddress = ioHead->buffer;
    *dmaCount = ioHead->size;
    *dmaCtrl = 1; // Start I/O
}
```

1. zadatak, prvi kolokvijum, april 2014.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva DMA kontrolera:

```
typedef unsigned int REG;
REG* dma1Ctrl =...; // DMA1 control register
REG* dma1Status =...; // DMA1 status register
REG* dma1Address =...; // DMA1 block address register
REG* dma1Count =...; // DMA1 block size register
```

```
REG* dma2Ctrl =...;    // DMA2 control register
REG* dma2Status =...;  // DMA2 status register
REG* dma2Address =...; // DMA2 block address register
REG* dma2Count =...;   // DMA2 block size register
```

U upravljačkom registru najniži bit je bit *Start* kojim se pokreće prenos jednog bloka preko DMA, a u statusnom registru najniži bit je bit završetka prenosa (*TransferComplete*), a bit do njega bit greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Kada DMA kontroler završi zadati prenos, on se automatski zaustavlja (nije ga potrebno zaustavljati upisom u upravljački registar). Završetak prenosa sa bilo kog DMA kontrolera generiše isti zahtev za prekid procesoru (signali završetka operacije sa dva DMA kontrolera vezani su na ulazni zahtev za prekid preko OR logičkog kola).

Zahtevi za ulaznim operacijama na nekom uređaju sa kog se prenos blokova vrši preko bilo kog od ova DMA kontrolera vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer; // Data buffer (block)
    unsigned int size; // Buffer (blok) size
    int status; // Status of operation
    IORequest* next; // Next in the list
};
```

Na prvi zahtev u listi pokazuje globalni pokazivač `ioHead`. Kada kernel u listu stavi novi zahtev, pozvaće operaciju `transfer()` koja treba da pokrene prenos za taj zahtev na bilo kom trenutno slobodnom DMA kontroleru (u slučaju da su oba kontrolera zauzeta ne treba ništa uraditi). Zahtev koji se dodeli nekom od DMA kontrolera na obradu izbacuje se iz liste. Kada se završi prenos zadat jednim zahtevom na jednom DMA kontroleru, potrebno je u polje `status` date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška) i pokrenuti prenos za sledeći zapis u listi na tom DMA kontroleru, i potom izbaciti zahtev iz liste. Obratiti pažnju na to da oba DMA kontrolera mogu završiti prenos i generisati prekid istovremeno. Ako zahteva u listi više nema, ne treba uraditi više ništa (kada bude stavljaao novi zahtev u listu, kernel će proveriti i videti da je ona bila prazna, pa pozvati ponovo operaciju `transfer()` itd.) Potrebno je napisati kod operacije `transfer()`, zajedno sa odgovarajućom prekidnom rutinom `dmaInterrupt()` za prekid od DMA kontrolera.

```
void transfer ();
interrupt void dmaInterrupt ();
```

Rešenje

```
IORequest *dma1Pending = 0, *dma2Pending = 0; // Currently pending requests
```

```
void startDMA1 () {
    if (ioHead!=0 && dma1Pending==0) {
        dma1Pending = ioHead; // Take the first request,
        ioHead = ioHead->next; // remove it from the list
        *dma1Address = dma1Pending->buffer; // and assign it to DMA1
        *dma1Count = dma1Pending->size;
        *dma1Ctrl = 1; // Start I/O
    }
}
```

```
void startDMA2 () {
    if (ioHead!=0 && dma2Pending == 0) {
        dma2Pending = ioHead; // Take the first request,
        ioHead = ioHead->next; // remove it from the list
        *dma2Address = dma2Pending->buffer; // and assign it to DMA2
        *dma2Count = dma2Pending->size;
        *dma2Ctrl = 1; // Start I/O
    }
}
```

```

void transfer () {
    startDMA1();
    startDMA2();
}

interrupt void dmaInterrupt () {
    if (dma1Status&1) { // DMA1 completed
        if (dma1Pending==0) return; // Exception
        if (*dma1Status&2) // Error in I/O
            dma1Pending->status = -1;
        else
            dma1Pending->status = 0;
        dma1Pending = 0;
        startDMA1();
    }
    if (dma2Status&1) { // DMA2 completed
        if (dma2Pending==0) return; // Exception
        if (*dma2Status&2) // Error in I/O
            dma2Pending->status = -1;
        else
            dma2Pending->status = 0;
        dma2Pending = 0;
        startDMA2();
    }
}

```

1. zadatak, prvi kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima jednog ulaznog uređaja i registru posebnog uređaja – vremenskog brojača:

```

typedef unsigned int REG;
REG* ioCtrl = ...; // Device control register
REG* ioData = ...; // Device data register
REG* timer = ...; // Timer

```

Učitavanje svakog pojedinačnog podatka sa ovog ulaznog uređaja zahteva se posebnim upisom vredosti 1 u najniži bit upravljačkog registra ovog uređaja. Spremnost ulaznog podatka u registru za podatke uređaj ne signalizira nikakvim signalom, ali je sigurno da je ulazni podatak spreman u registru podataka najkasnije 50 ms nakon zadatog zahteva (upisa u kontrolni registar).

Na magistralu računara vezan je i registar posebnog uređaja, vremenskog brojača. Upisom celobrojne vrednosti n u ovaj registar, vremenski brojač počinje merenje vremena od n milisekundi, nakon isteka tog vremena, generiše prekid procesoru.

Na jeziku C napisati kod operacije `transfer()` zajedno sa odgovarajućom prekidnom rutinom za prekid od vremenskog brojača `timerInterrupt()`, koja obavlja učitavanje bloka podataka zadate dužine na zadatu adresu u memoriji sa datog ulaznog uređaja.

```

void transfer (REG* buffer, unsigned int count);
interrupt void timerInterrupt ();

```

Rešenje

```

static const unsigned int timeout = 50; // 50 ms
static int completed = 0;

```

```

static REG* ptr = 0;
static unsigned int count = 0;

void transfer (REG* buffer, unsigned int cnt) {
    // Initialize transfer
    completed = 0;
    ptr = buffer;
    count = cnt;

    // Start transfer:
    *ioCtrl = 1; // Input request
    *timer = timeout; // Start timer

    while (!completed); // Busy wait for transfer completion
}

interrupt void timerInterrupt () {
    *ptr++ = *ioData; // Read data
    if (--count) {
        *ioCtrl = 1; // New input request
        *timer = timeout; // Restart timer
    } else // Completed
        completed = 1;
}

```

1. zadatak, prvi kolokvijum, mart 2013.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva uazno/izlazna uređaja:

```

typedef unsigned int REG;
REG* io1Ctrl = ...; // Device 1 control register
REG* io1Status = ...; // Device 1 status register
REG* io1Data = ...; // Device 1 data register
REG* io2Ctrl = ...; // Device 2 control register
REG* io2Status = ...; // Device 2 status register
REG* io2Data = ...; // Device 2 data register

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Potrebno je napisati kod koji najpre vrši ulaz bloka podataka zadate veličine sa prvog uređaja korišćenjem prekida, a potom izlaz tog istog učitano bloka podataka na drugi uređaj tehnikom prozivanja (*polling*).

Rešenje

```

static REG* ioPtr = 0;
static int ioCount = 0;
static int ioCompleted = 0;

void transfer (int count) {
    REG* buffer = new REG[count];

    // I/O 1:
    ioPtr = buffer;
    ioCount = count;
    ioCompleted = 0;
}

```



```

*io1Ctrl = 1; // Start I/O 1

// Wait for I/O 1 completion:
while (!ioCompleted);

// I/O 2
ioPtr = buffer;
ioCount = count;
*io2Ctrl = 1; // Start I/O 2
while (ioCount>0) {
    while (!(*io2Status&1)); // busy wait
    *io2Data = *ioPtr++;
    ioCount--;
}
*io2Ctrl = 0; // Stop I/O 2

delete [] buffer;
}

interrupt void io1Interrupt() {
    *ioPtr++ = *io1Data;
    if (--ioCount == 0) {
        ioCompleted = 1;
        *io1Ctrl = 0; // Stop I/O 1
    }
}
}

```

1. zadatak, prvi kolokvijum, april 2013.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima tri uređaja; prva dva uređaja su ulazni, a treći je izlazni:

```

typedef unsigned int REG;
REG* io1Ctrl = ...; // Device 1 control register
REG* io1Status = ...; // Device 1 status register
REG* io1Data = ...; // Device 1 data register
REG* io2Ctrl = ...; // Device 2 control register
REG* io2Status = ...; // Device 2 status register
REG* io2Data = ...; // Device 2 data register
REG* io3Ctrl = ...; // Device 3 control register
REG* io3Status = ...; // Device 3 status register
REG* io3Data = ...; // Device 3 data register

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Potrebno je napisati program koji učitava po jednu reč sa bilo kog od dva ulazna uređaja (na kom god se pre pojavi spreman podatak) i odmah tu učitavanu reč šalje na izlazni uređaj. Ovaj prenos se obavlja sve dok se sa ulaznog uređaja ne učitava vrednost 0. Sve prenose vršiti programiranim ulazom/izlazom sa prozivanjem (*polling*).

Rešenje

```

const REG ESC = 0;

void main () {
    REG data = ESC;

```

```

*io1Ctrl = 1;
*io2Ctrl = 1;
*io3Ctrl = 1;

while (1) {
    while (*io1Status==0 && *io2Status==0);
    if (*io1Status) data = *io1Data;
    else data = *io2Data;
    if (data==ESC) break;
    while (*io3Status==0);
    *io3Data = data;
}

*io1Ctrl = 0;
*io2Ctrl = 0;
*io3Ctrl = 0;
}

```

1. zadatak, prvi kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima jednog ulaznog uređaja:

```

typedef unsigned int REG;
REG* ioCtrl = ...; // Device control register
REG* ioStatus = ...; // Device status register
REG* ioData = ...; // Device data register

```

U upravljačkom registru najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnom registru najniži bit je bit spremnosti (*Ready*), a bit do njega bit greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Zahtevi za ulaznim operacijama na tom uređaju vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```

struct IORequest {
    REG* buffer; // Data buffer (block)
    unsigned int size; // Buffer (blok) size
    void (*callback)(IORequest*); // Call-back function
    int status; // Status of operation
    IORequest* next; // Next in the list
};

```

Kada se završi prenos zadat jednim zahtevom, potrebno je pozvati funkciju na koju ukazuje pokazivač `callback` u tom zahtevu, sa argumentom koji ukazuje na taj zahtev. Ovu funkciju implementira onaj ko je zahtev postavio i služi da mu signalizira da je zahtev obrađen. U polju `status` date strukture treba preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška). Obađeni zahtev ne treba brisati iz liste (to je odgovornost onog ko je zahtev postavio).

Potrebno je napisati kod operacije `transfer()`, zajedno sa odgovarajućom prekidnom rutinom za prekid od uređaja `ioInterrupt()`, koja obavlja sve prenose zadate zahtevima u listi na čiji prvi zapis ukazuje argument `ioHead` tehnikom programiranog ulaza korišćenjem prekida.

```

void transfer (IORequest* ioHead);
interrupt void ioInterrupt ();

```

Rešenje

```

static int completed = 0;
static int status = 0;
static REG* ptr = 0;
static unsigned int count = 0;

void transfer (IORequest* ioHead) {
    while (ioHead) {
        completed = 0; // initialize transfer
        status = 0;
        ptr = ioHead->buffer;
        count = ioHead->size;
        *ioCtrl = 1; // Start I/O
        while (!completed); // wait for I/O to complete
        ioHead->status=status; // set status
        ioHead->callBack(ioHead); // signal completion
        ioHead = ioHead->next; // take next
    }
}

interrupt void ioInterrupt () {
    if (*ioStatus&2) { // Error in I/O
        completed = 1;
        status = -1;
        *ioCtrl = 0; // Stop I/O 2
        return;
    }
    *ptr++ = *ioData; // input data
    if (--count == 0) { // transfer completed
        completed=1; // signal completion
        *ioCtrl = 0; // stop I/O
    }
}

```

1. zadatak, prvi kolokvijum, mart 2012.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva izlazna uređaja:

```

typedef unsigned int REG;
REG* io1Ctrl =...; // Device 1 control register
REG* io1Status =...; // Device 1 status register
REG* io1Data =...; // Device 1 data register
REG* io2Ctrl =...; // Device 2 control register
REG* io2Status =...; // Device 2 status register
REG* io2Data =...; // Device 2 data register

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Potrebno je napisati kod koji vrši prenos po jednog bloka podataka na svaki od dva uređaja uporedo, na prvi uređaj tehnikom prozivanja (*polling*), a na drugi korišćenjem prekida. Transfer se obavlja pozivom sledeće funkcije iz koje se vraća kada su oba prenosa završena:

```

void transfer (unsigned* blk1, int count1, unsigned* blk2, int count2);

```

Rešenje

```

static unsigned* io2Ptr = 0;
static int io2Count = 0;
static int io2Completed = 0;

void transfer (unsigned* blk1, int count1, unsigned* blk2, int count2) {
    // I/O 2:
    io2Ptr = blk2;
    io2Count = count2;
    io2Completed = 0;
    *io2Ctrl = 1; // Start I/O 2

    // I/O 1
    *io1Ctrl = 1; // Start I/O 1
    while (count1>0) {
        while (!(*io1Status&1)); // busy wait
        *io1Data = *blk1++;
        count1--;
    }
    *io1Ctrl = 0; // Stop I/O 1

    // Wait for I/O 2 completion:
    while (!io2Completed);
}

interrupt void io2Interrupt() {
    *io2Data = *io2Ptr++;
    if (--io2Count == 0) {
        io2Completed = 1;
        *io2Ctrl = 0; // Stop I/O 2
    }
}

```

1. zadatak, prvi kolokvijum, maj 2012.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima jednog DMA uređaja:

```

typedef unsigned int REG;
REG* dmaCtrl = ...; // control register
REG* dmaStatus = ...; // status register
REG* dmaBlkAddr = ...; // data block address register
REG* dmaBlkSize = ...; // data block size register

```

U upravljačkom registru najniži bit je bit *Start* kojim se pokreće prenos preko DMA, a u statusnom registru najniži bit je bit spremnosti (*Ready*) čija vrednost 1 ukazuje da je DMA spreman za novi prenos podatka (inicijalno je tako postavljen). Postavljanje bita spremnosti kada DMA završi zadati prenos generiše signal zahteva za prekid procesoru. Zahtevi za izlaznim operacijama na nekom izlaznom uređaju vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```

struct OutputRequest {
    char* buffer; // Buffer with data (block)
    unsigned int size; // Buffer (blok) size
    void (*callback)(OutputRequest*); // Call-back function
    OutputRequest* next; // Next in the list
};

```

Kada se završi prenos zadat jednim zahtevom, potrebno je pozvati funkciju na koju ukazuje pokazivač `callBack` u tom zahtevu, sa argumentom koji ukazuje na taj zahtev. Ovu funkciju implementira onaj ko je zahtev postavio i služi da mu signalizira da je zahtev obrađen. Odrađeni zahtev ne treba brisati iz liste (to je odgovornost onog ko je zahtev postavio).

Potrebno je napisati kod operacije `transfer()`, zajedno sa odgovarajućom prekidnom rutinom `dmaInterrupt()`, koja obavlja sve prenose zadate zahtevima u listi na čiji prvi zapis ukazuje argument `ioHead`.

```
void transfer (OutputRequest* ioHead);
interrupt void dmaInterrupt ();
```

Rešenje

```
static int dmaCompleted = 0;

void transfer (OutputRequest* ioHead) {
    while (ioHead) {
        dmaCompleted = 0; // initialize transfer
        *dmaBlkAddress = ioHead->buffer;
        *dmaBlkSize = ioHead->size;
        *dmaCtrl = 1; // start transfer
        while (!dmaCompleted); // wait for DMA to complete
        ioHead->callBack(ioHead); // signal completion
        ioHead = ioHead->next; // take next
    }
}

interrupt void dmaInterrupt () {
    dmaCompleted = 1;
    *dmaCtrl = 0;
}
```

1. zadatak, prvi kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva ulazna uređaja:

```
typedef unsigned int REG;
REG* io1Ctrl = ...; // Device 1 control register
REG* io1Status = ...; // Device 1 status register
REG* io1Data = ...; // Device 1 data register
REG* io2Ctrl = ...; // Device 2 control register
REG* io2Status = ...; // Device 2 status register
REG* io2Data = ...; // Device 2 data register
```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*), a bit do njega bit greške (*Error*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Potrebno je napisati kod koji vrši prenos po jednog bloka podataka sa svakog od dva uređaja uporedo, sa prvog uređaja tehnikom prozivanja (*polling*), a sa drugog korišćenjem prekida. U slučaju greške na bilo kom uređaju, prenos sa tog uređaja treba prekinuti, a sa drugog uređaja treba nastaviti (osim ukoliko je i kod njega došlo do greške). Transfer se obavlja pozivom sledeće funkcije iz koje se vraća kada su oba prenosa završena:

```
int transfer (unsigned* blk1, int count1, unsigned* blk2, int count2);
```

Povratna vrednost ove funkcije treba da bude ceo broj čija binarna predstava u najniža dva bita ima sledeće značenje: bit 0 ukazuje na grešku u prenosu sa prvog uređaja (0-ispravno, 1- greška), a bit 1 na grešku u prenosu sa drugog uređaja.

Rešenje

```

static unsigned* io2Ptr = 0;
static int io2Count = 0;
static int io2Completed = 0;

int transfer (unsigned* blk1, int count1, unsigned* blk2, int count2) {
    // I/O 2:
    io2Ptr = blk2;
    io2Count = count2;
    io2Completed = 0;
    int status = 0;
    *io2Ctrl = 1; // Start I/O 2

    // I/O 1
    *io1Ctrl = 1; // Start I/O 1
    while (count1>0) {
        while (!(*io1Status&1)); // Busy wait
        if (*io1Status&2) { // Error in I/O 1
            status |= 1;
            break;
        }
        *blk1++ = *io1Data;
        count1--;
    }
    *io1Ctrl = 0; // Stop I/O 1

    // Wait for I/O 2 completion:
    while (!io2Completed);
    if(io2Completed<0)status |= 2;
    return status;
}

interrupt void io2Interrupt() {
    if (*io2Status&2) { // Error in I/O 2
        io2Completed = -1;
        *io2Ctrl = 0; // Stop I/O 2
        return;
    }
    *io2Ptr++ = *io2Data;
    if (--io2Count == 0) {
        io2Completed = 1;
        *io2Ctrl = 0; // Stop I/O 2
    }
}

```

1. zadatak, prvi kolokvijum, april 2011.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva izlazna uređaja:

```

typedef unsigned int REG;
REG* io1 Ctrl =...; // Device 1 control register
REG* io1 Status =...; // Device 1 status register
REG* io1 Data =...; // Device 1 data register
REG* io2Ctrl =...; // Device 2 control register

```

```
REG* io2Status =...; // Device 2 status register
REG* io2Data =...;   // Device 2 data register
```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Biti spremnosti statusnih registara oba uređaja vezani su preko logičkog ILI kola na isti signal zahteva za prekid, tako da se isti prekid generiše ako je bilo koji (ili oba) uređaja postavio svoj bit spremnosti.

Potrebno je napisati kod koji može da izvrši prenos datog bloka podataka korišćenjem oba uređaja uporedo, tako da se na svaki uređaj prenese po pola datog bloka (ili približno pola, ako je broj reči neparan). Ovaj prenos pokreće se pozivom dole date funkcije `transfer()`.

```
int flag1 = 0, flag2 = 0; // I/O completed
unsigned int index1, count1;
unsigned int index2, count2;
REG* buf1;
REG* buf2;

void transfer (REG* buffer, unsigned int count) {
    count1 = count/2;
    count2 = count1+count%2;
    buf1 = buffer;
    buf2 = buffer+count1;
    index1 = index2 = 0;
    flag1 = flag2 = 0;
    *io1Ctrl = 1;
    *io1Ctrl2 = 1;
}

int isIOCompleted () {
    return flag1 && flag2;
}

interrupt void ioInterrupt();
```

Napisati kod prekidne rutine `ioInterrupt()`.

Rešenje

```
interrupt void ioInterrupt() {
    if (*ioStatus1) {
        *io1Data = buf1[index1++];
        if (--count1 == 0) {
            flag1 = 1;
            *iosCtrl1 = 0;
        }
    }
    if (*ioStatus2) {
        *io2Data = buf2[index2++];
        if (--count2 == 0) {
            flag2 = 1;
            *iosCtrl2 = 0;
        }
    }
}
```

1. zadatak, prvi kolokvijum, maj 2011.

- [Postavka](#)

- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva ulazna uređaja:

```
typedef unsigned int REG;
REG* io1 Ctrl =...;    // Device 1 control register
REG* io1 Status =...;  // Device 1 status register
REG* io1 Data =...;    // Device 1 data register
REG* io2Ctrl =...;     // Device 2 control register
REG* io2Status =...;   // Device 2 status register
REG* io2Data =...;     // Device 2 data register
```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Potrebno je napisati kod funkcije `transfer()` koji vrši prenos datog bloka podataka korišćenjem oba uređaja uporedo, tako da se sa svakog uređaja prenese po pola datog bloka (ili približno pola, ako je broj reči neparan), programiranim ulazom/izlazom sa prozivanjem (*polling*), uz maksimalni paralelizam rada uređaja i prenosa delova bloka.

```
void transfer (REG* buffer, unsigned int count) ;
```

Rešenje

```
void transfer (REG* buffer, unsigned int count) {
    unsigned int index1, count1;
    unsigned int index2, count2;
    REG* buf1;
    REG* buf2;

    count1 = count/2;
    count2 = count1+count%2;
    buf1 = buffer;
    buf2 = buffer+count1;
    index1 = index2 = 0;
    *io1Ctrl = 1;
    *io1Ctrl2 = 1;

    while ((count1!=0) || (count2!=0)) {
        if ((count1!=0) && *ioStatus1) {
            buf1[index1++] = *io1Data;
            if (--count1 == 0) *iosCtrl1 = 0;
        }
        if ((count2!=0) && *ioStatus2) {
            buf2[index2++] = *io2Data;
            if (--count2 == 0) *iosCtrl2 = 0;
        }
    }
}
```

1. zadatak, prvi kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva uređaja; prvi uređaj je ulazni, drugi izlazni:

```
typedef unsigned int REG;
REG* io1 Ctrl =...;    // Device 1 control register
```



```

REG* io1 Status =...; // Device 1 status register
REG* io1 Data =...;   // Device 1 data register
REG* io2Ctrl =...;    // Device 2 control register
REG* io2Status =...;  // Device 2 status register
REG* io2Data =...;    // Device 2 data register

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće uređaj, a u statusnim registrima najniži bit je bit spremnosti (*Ready*). Svi registri su veličine jedne mašinske reči (tip `unsigned int`).

Potrebno je napisati program koji učitava po jednu reč sa ulaznog urešaja i odmah tu učitane reč šalje na izlazni uređaj. Ovaj prenos se ponavlja sve dok se sa ulaznog uređaja ne učitava vrednost 0. Oba prenosa vršiti programiranim ulazom/izlazom za prozivanje (*polling*).

Rešenje

```

const REG ESC = 0;

void main () {
    *io1Ctrl = 1;
    *io2Ctrl = 1;

    while (1) {
        while (*io1Status==0);
        REG data = *io1Data;
        if (data==ESC) break;
        while (*io2Status==0);
        *io2Data = data;
    }

    *io1Ctrl = 0;
    *io2Ctrl = 0;
}

```

1. zadatak, prvi kolokvijum, april 2010.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima dva DMA kontrolera:

```

typedef unsigned int REG;
REG* dma1Ctrl =...; // DMA1 control register
REG* dma1Status =...; // DMA1 status register
REG* dma1Addr =...; // DMA1 block address register
REG* dma1Count =...; // DMA1 block size register
REG* dma2Ctrl =...; // DMA2 control register
REG* dma2Status =...; // DMA2 status register
REG* dma2Addr =...; // DMA2 block address register
REG* dma2Count =...; // DMA2 block size register

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće DMA kontroler, a u statusnim registrima najniži bit je bit spremnosti (*Ready*) čija vrednost 1 ukazuje da je DMA kontroler završio prenos. Svaki DMA kontroler upravlja svojom izlaznom periferijom sa kojom direktno komunicira, tako da procesor ne treba (i ne može) direktno da pristupa kontrolerima periferija. Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Jezgro operativnog sistema postavlja zahteve za prenos blokova podataka iz memorije na bilo koju od dve periferije uvezivanjem sledećih struktura sa zahtevima u listu:

```

struct DMAReq {
    DMAReq* next; // Next request in the list
    unsigned int addr; // Block address
}

```

```

    unsigned int size; // Block size
};
DMAReq* head;
DMAReq* tail;

```

Na jeziku C napisati potprogram koji uzima jedan po jedan postavljeni zahtev iz liste i zadaje ih bilo kom slobodnom DMA kontroleru. Čekanje da se pojavi prvi zahtev u praznoj listi i da se oslobodi DMA kontroler treba obavljati uposlono, odnosno prozivanjem (*busy waiting, polling*).

Rešenje

```

void DMADriver() {
    while (1){
        DMAReq* req;
        // Busy wait for a request:
        while (head==0);
        // Take a request:
        req = head;
        head=head->next;
        if (head==0) tail==0;
        req->next=0;
        // Busy wait for a free DMA:
        while ((*dma1Status&1)==0 && (*dma2Status&1)==0);
        // Start DMA:
        if (*dma1Status&1) {
            *dma1Addr=req->addr;
            *dma1Count=req->size;
            *dma1Ctrl=1;
        } else
        if (*dma2Status&1) {
            *dma2Addr=req->addr;
            *dma2Count=req->size;
            *dma2Ctrl=1;
        }
        free(req);
    }
}

```

2. zadatak, prvi kolokvijum, april 2009.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima nekih perifernih uređaja, pošto su oni inicijalizovani adresama tih registara:

```

typedef unsigned int REG;
REG* ioCtrl1 =...; // PER1 control register
REG* ioStatus1 =...; // PER1 status register
REG* ioData1 =...; // PER1 data register
REG* ioCtrl2 =...; // PER2 control register
REG* ioStatus2 =...; // PER2 status register
REG* ioData2 =...; // PER2 data register
REG* ioCtrl3 =...; // PER3 control register
REG* ioStatus3 =...; // PER3 status register
REG* ioData3 =...; // PER3 data register

```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće periferija, a u statusnim registrima najniži bit je bit spremnosti (*Ready*) čija vrednost 1 ukazuje da je kontroler periferije spreman za prenos podatka preko svog

registra za podatke. PER1 i PER2 su ulazne, a PER3 je izlazna periferija. Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Date su deklaracije bloka podataka koji treba prenositi:

```
const int N = ...;
unsigned int buf[N];
```

Na jeziku C napisati program koji uporedo sa PER1 i PER2 učitava blok od ukupno N reči, i na PER3 upisuje taj isti blok od N reči, korišćenjem mehanizma prozivanja (engl. *polling*), bez nametanja naizmeničnosti prenosa po jednog podatka (npr. da se uvek jedan podatak prebaci sa PER1, pa onda sa PER2 i tako naizmenično), već podatak sa datom periferijom treba preneti kad god je ona spremna, ne čekajući ostale. Broj podataka koji se čitaju sa pojedinačnih periferija PER1 i PER2 nije unapred poznat, već zavisi od brzine rada periferija.

Rešenje

```
void main () {
    int i12=0, i3=0;
    *ioCtrl1=1; *ioCtrl2=1; *ioCtrl3=1; // Start
    while (i3<N) {
        if (i12<N && *ioStatus1&1) buf[i12++]=*ioData1;
        if (i12<N && *ioStatus2&1) buf[i12++]=*ioData2;
        if (i3<i12 && *ioStatus3&1) *ioData3=buf[i3++];
    }
    *ioCtrl1=0; *ioCtrl2=0; *ioCtrl3=0; // Stop them all
}
```

2. zadatak, prvi kolokvijum, april 2008.

- [Postavka](#)
- [Rešenje](#)

Date su deklaracije pokazivača preko kojih se može pristupiti registrima nekih periferijskih uređaja, pošto su oni inicijalizovani adresama tih registara:

```
typedef unsigned int REG;
REG* ioCtrl1 =...; // PER1 control register
REG* ioStatus1 =...; // PER1 status register
REG* ioData1 =...; // PER1 data register
REG* ioCtrl2 =...; // PER2 control register
REG* ioStatus2 =...; // PER2 status register
REG* ioData2 =...; // PER2 data register
```

U upravljačkim registrima najniži bit je bit *Start* kojim se pokreće periferija, a u statusnim registrima najniži bit je bit spremnosti (*Ready*) čija vrednost 1 ukazuje da je kontroler periferije spreman za prenos podatka preko svog registra za podatke. PER1 je ulazna, a PER2 izlazna periferija. Svi registri su veličine jedne mašinske reči (tip `unsigned int`). Date su deklaracije dva bloka podataka koje treba preneti:

```
const int N = ...;
unsigned int buf1[N], buf2[N];
```

Na jeziku C napisati program koji uporedo sa PER1 učitava blok od N reči, a na PER2 upisuje blok od N reči, korišćenjem mehanizma prozivanja (engl. *polling*), bez nametanja naizmeničnosti prenosa po jednog podatka (npr. da se uvek jedan podatak prebaci sa PER1, pa onda sa PER2 i tako naizmenično), već podatak sa datom periferijom treba preneti kad god je ona spremna, ne čekajući na onu drugu.

Rešenje

```
void main () {
    int i1=0, i2=0;
    *ioCtrl1=1; *ioCtrl2=1; // Start them both
    while (i1<N || i2<N) {
        if (i1<N && *ioStatus1&1) buf1[i1++]=*ioData1;
```

```

    if (i2<N && *ioStatus2&1) *ioData2=buf2[i2++];
}
*ioCtrl1=0; *ioCtrl2=0; // Stop them both
}

```

1. zadatak, prvi kolokvijum, april 2007.

- [Postavka](#)
- [Rešenje](#)

Predložiti interfejs `DMACtrl` i specifikovati ga apstraktnom klasom na jeziku C++. Ovaj interfejs treba da delovima operativnog sistema koji upravljaju uređajima obezbedi uniforman pristup različitim vrstama DMA kontrolera, tako što im na isti način zadaje operacije prenosa bloka podataka iz ili u memoriju, kao i način obaveštavanja o završenom prenosu i eventualnoj grešci. Obaveštavanje o završenoj operaciji može da bude prozivanjem (*polling*) ili prekidom sa zadatim ulazom.

Rešenje

```

enum Status{OK, Error};

class DMA {
public:
    //ulazni parametri: adresa bloka u memoriji, adresa registra podataka
    //uređaja, adresa statusnog registra, maska za statusni registar, smjer
    //operacije i velicina bloka
    virtual void start(void* mem, void* devData, void* devStatus, int
StatusMask, int memToIO, int vel);

    //rezultat: logicka vrijednost koja pokazuje da li je operacija završena
    virtual int is_finished();

    //zadaje kontroleru broj prekidne rutine preko koje treba da obavjesti
    //sistem o kraju operacije
    virtual void setIntNo(int interruptNo);

    //vraca status poslednje završene operacije
    virtual Status getStatus()
}

```

2. zadatak, prvi kolokvijum, april 2006.

- [Postavka](#)
- [Rešenje](#)

Na nekom računaru sa multiprogramskim operativnim sistemom neki kontroler periferije nema vezan svoj signal završetka operacije na ulaze za prekid procesora, ali je poznato vreme završetka operacije zadate tom uređaju u najgorem slučaju (maksimalno vreme završetka operacije). Za obradu ulazno/izlaznih operacija zadatih tom uređaju brine se poseban proces tog operativnog sistema. Kakvu uslugu treba operativni sistem da obezbedi, a ovaj proces da koristi, da bi se izbegla tehnika kojom se može detektovati završetak operacije čitanjem statusnog registra i ispitivanjem bita završetka operacije (*polling*), tj. da bi se izbeglo uposlano čekanje (*busy waiting*)? Precizno opisati kako ovaj proces treba da koristi ovu uslugu.

Rešenje

Operativni sistem treba da nudi uslugu da neki proces može da se uspava i da se probuti posle tačno određenog vremena (ili malo kasnije od tog vremena). Npr operativni sistem treba da obezbedi primitivu `void sleep(int time)`, koja obezbedjuje da se proces uspava time milisekundi (odnosno stavi u red uspavanih, engl. *Sleep*), a da se posle toga može probuditi (odnosno proces vrati u red spremnih, engl. *Ready*). Proces za obradu ulazno/izlazne operacije treba da radi sledeće:

```
const int maxTime = ...; // response time (worst case)

while (!end){
    initialize_IO_operation();
    sleep(maxTime);
    process_IO_result();
}
```

Ulaz/izlaz (blokovski uređaji)

1. zadatak, treći kolokvijum, jun 2023.

- [Postavka](#)
- [Rešenje](#)

U ulazno-izlaznom podsistemu nekog operativnog sistema zahtevi za operacijama sa diskom predstavljaju se instancama strukture `DiskRequest`. Jedan zahtev odnosi se na prenos `blockCount` susednih blokova počev od bloka broj `startBlockNo`, a prenos se vrši sa baferom `buffer` i u smeru koji zadaje polje `dir`.

```
struct DiskRequest {
    uint32 blockCount;
    uint32 startBlockNo;
    uint32* buffer;
    enum Dir : uint32 {in = 0, out = 1};
    Dir dir;
    ... // Other details irrelevant here
};
```

Za svaki fizički uređaj formira se poseban red `RequestQueue` ovakvih zahteva. Svaki takav red opslužuje po jedna interna nit jezgra koja izvršava funkciju `diskDriver` prikazanu dole. Ova funkcija uzima jedan po jedan zahtev iz reda i svaki zahtev zadaje drajveru uređaja `dd` pozivom njegove operacije `startTransfer`. Potom se blokira na semaforu `semComplete` koji drajver uređaja treba da signalizira kada zahtev bude opslužen, a onda dalje obaveštava podnosioca zahteva (izostavljeni detalji).

```
void diskDriver(RequestQueue* rque, IBlockDeviceDriver* dd) {
    while (true) {
        // Blocks until a request arrives
        DiskRequest* req = rque->getRequest();
        dd->startTransfer(req);
        semComplete->wait();
        ... // Notify the request initiator
    }
}
```

Interfejs drajvera uređaja `IBlockDeviceDriver` dat je dole. Kada se drajver instalira, jezgro poziva njegovu operaciju `init` za inicijalizaciju samog drajvera, zadajući semafor koji treba signalizirati nakon svakog opsluženog zahteva (`semComplete` pomenut gore).

```
class IBlockDeviceDriver {
public:
    virtual int init(Semaphore* complete) = 0;
    virtual void startTransfer(DiskRequest*) = 0;
};
```

Tokom inicijalizacije u svojoj operaciji `init` drajver može da poziva sledeće operacije jezgra:

- `uint32* requestDMAChannel()`: zahteva od jezgra zauzimanje jednog slobodnog DMA kontrolera (kanala) za svoje potrebe; jezgro dodeljuje takav kontroler na upotrebu drajveru, ukoliko ga ima, i vraća adresu početka regiona memorijski mapiranih upravljačkih registara dodeljenog DMA kontrolera; ukoliko slobodnog DMA kontrolera nema, vraća `null`;
- `uint32 requestIVTEntry(void (*transferComplete)(void*), void* ptr)`: od jezgra traži zauzeće i inicijalizaciju jednog slobodnog ulaza u IVT; ukoliko ne uspe, vraća negativnu vrednost; ukoliko uspe, inicijalizuje taj ulaz u IVT adresom funkcije zadate prvim parametrom i vraća taj broj ulaza; jezgro obezbeđuje da se prekidna rutina poziva sa parametrom `ptr` (prekidna rutina na koju je usmeren jedan ulaz u IVT poziva se uvek sa istim tim parametrom).

Ukoliko inicijalizacija ne uspe, funkcija `init` treba da vrati negativnu vrednost, 0 za uspeh. Svaki DMA kontroler obavlja prenos sa samo jednim diskom i ima dva 32-bitna upravljačka registra u svom regionu memorijski mapiranih registara. Prvi registar nalazi se na pomeraju nula od početka regiona, a prilikom inicijalizacije DMA kontrolera u

ovaj registar potrebno je upisati broj ulaza u IVT koji će ovaj DMA kontroler koristiti prilikom prekida koji generiše kada završi svaku zadatu operaciju. Drugi registar nalazi se na pomeraju 4 (adresibilna jedinica je bajt, adrese su 32-bitne) od početka regiona, a jedna operacija prenosa DMA kontroleru se zadaje sukcesivnim upisima u ovaj isti upravljački registar na sledeći način (nije potrebno čekati na dozvolu za sledeći upis, ovi upisi mogu da idu odmah jedan iza drugog): najpre je u ovaj registar potrebno upisati adresu bafera, zatim broj prvog bloka na disku, zatim broj susednih blokova koje treba preneti i konačno smer prenosa (0 ili 1). Ovaj poslednji upis ujedno i pokreće prenos. Po završetku prenosa DMA kontroler generiše prekid.

Realizovati u potpunosti klasu `DiskDeviceDriver` koja implementira interfejs `IBlockDeviceDriver`, a koja realizuje drajver za prenos sa disk uređajem korišćenjem DMA kontrolera. Jedan objekat ove klase treba da bude zadužen za jedan fizički disk uređaj.

Rešenje

```
class DiskDeviceDriver : public IBlockDeviceDriver {
public:
    virtual int init(Semaphore* complete);
    virtual void startTransfer(DiskRequest*);
private:
    uint32* dmaCtrlRegion;
    Semaphore* semComplete;
    friend void transferComplete(void*);
};

int DiskDeviceDriver::init(Semaphore* sc) {
    semComplete = sc;
    dmaCtrlRegion = requestDMAChannel();
    if (!dmaCtrlRegion) return -1;
    uint32 ivte = requestIVTEntry(transferComplete, this);
    if (ivte < 0) return -1;
    *(dmaCtrlRegion + 0) = ivte;
    return 0;
}

void DiskDeviceDriver::startTransfer(DiskRequest* req) {
    *(dmaCtrlRegion + 1) = req->buffer;
    *(dmaCtrlRegion + 1) = req->startBlockNo;
    *(dmaCtrlRegion + 1) = req->blockCount;
    *(dmaCtrlRegion + 1) = req->dir;
}

interrupt void transferComplete(void* obj) {
    ((DiskDeviceDriver*)obj)->semComplete->signal();
}
```

1. zadatak, treći kolokvijum, avgust 2023.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu implementira se keš blokova sa blokovskih uređaja kodom koji je dat u nastavku. Za svaki uređaj sa datim identifikatorom pravi se jedan objekat klase `BlockIOCache`, inicijalizovan tim identifikatorom, koji predstavlja keš blokova sa tog uređaja. Keš je kapaciteta `CACHESIZE` blokova veličine `BLKSIZE`. Keš je interno organizovan kao heš mapa map sa `MAPSIZE` ulaza. Svaki ulaz niza map sadrži glavu liste keširanih blokova koji se preslikavaju u taj ulaz. Funkcija `hash` je heš funkcija koja preslikava broj bloka u ulaz u nizu map. Glava liste, kao i pokazivač na sledeći element u listi čuvaju se kao indeksi elementa niza `entries` koji sadrži keširane blokove; vrednost -1 označava kraj (`null`). Svaki element niza `entries` je struktura tipa `CacheEntry` u kojoj je polje `blkNo` broj bloka koji je keširan u tom elementu, polje `next` ukazuje na sledeći element liste, a polje `buf` je sadržaj samog bloka.

Na početku složene operacije sa uređajem, kod koji koristi keš najpre zahteva da potrebni blok bude učitao pozivom funkcije `getBlock` koja vraća pokazivač na niz bajtova u baferu – učitanoj bloku. Pošto više ovakvih složenih operacija može biti pokrenuto uporedo, blok iz keša može biti izbačen (zamenjen drugim) samo ako ga više niko ne koristi, što se realizuje brojanjem referenci u polju `refCounter` strukture `CacheEntry`. Prikazana je implementacija funkcije `getBlock` koja treba da obezbedi da je traženi blok u kešu, odnosno učitao ga ako nije.

Potrebno je implementirati pomoćnu funkciju `getFreeEntry` koja treba da vrati indeks slobodnog ulaza u nizu `entries` u koji se može učitati traženi blok u keš. Inicijalno je keš prazan i svi ulazi u njemu su slobodni. Ova funkcija treba redom da zauzima elemente niza `entries`, sve dok ima slobodnih. Kada slobodnih ulaza više nema, ona treba da izbacila blok (snimi ga na disk) u prvom ulazu koji je na redu po FIFO redosledu (najstariji učitao), ali samo pod uslovom da se blok u tom ulazu ne koristi (tj. njegov `refCounter` je nula). Ako to nije zadovoljeno, treba da proba sa sledećim i tako u krug. Ako nema mesta u kešu jer nijedan blok ne može da se izbacila, treba vratiti -1. Ukoliko je potrebno dodati ili izmeniti članove ove klase, precizno navesti kako to treba uraditi. Na raspolaganju je i funkcija koja učitava blok, odnosno upisuje blok na datu uređaj:

```
void ioRead(int device, BlkNo blk, Byte* buffer);
void ioWrite(int device, BlkNo blk, Byte* buffer);

typedef unsigned char Byte; // Unit of memory
typedef long BlkNo; // Device block number
const unsigned BLKSIZE = ...; // Block size in Bytes

class BlockIOCache {
public:
    BlockIOCache(int device);
    Byte* getBlock(BlkNo blk);
    ...
protected:
    static int hash(BlkNo);
    int getFreeBlock();
private:
    static const unsigned CACHESIZE = ...; // Cache size in blocks
    static const unsigned MAPSIZE = ...; // Hash map size in entries
    struct CacheEntry {
        BlkNo blkNo;
        int next;
        int refCounter;
        Byte buf[BLKSIZE];
    };
    int dev;
    int map[MAPSIZE]; // Hash map
    CacheEntry entries[CACHESIZE]; // Cache
    ...
};

Byte* BlockIOCache::getBlock(BlkNo blk) {
    // Find the requested block in the cache and return it if present:
    int entry = hash(blk);
    for (int i = map[entry]; i != -1; i = entries[i].next)
        if (entries[i].blkNo == blk) {
            entries[i].refCounter++;
            return entries[i].buf;
        }
    // The block is not in the cache, find a free slot to load it:
    int free = getFreeEntry();
    if (free == -1) return 0; // Error: cannot find space
    // Load the requested block:
```



```

    entries[free].blkNo = blk;
    entries[free].refCounter = 1;
    entries[free].next = map[entry];
    map[entry] = free;
    ioRead(dev, blk, entries[free].buf);
    return entries[free].buf;
}

```

Rešenje

U klasu BlockIOCache treba uvesti privatne podatke članove numOfBlocks i hand, oba tipa int i inicijalizovana na 0.

```

int BlockIOCache::getFreeEntry() {
    if (numOfBlocks < CACHESIZE) return numOfBlocks++;
    int oldHand = hand;
    do {
        if (entries[hand].refCounter == 0) {
            ioWrite(dev, entries[hand].blkNo, entries[hand].buf);
            int ret = hand;
            hand = (hand + 1) % CACHESIZE;
            return ret;
        }
        hand = (hand + 1) % CACHESIZE;
    } while (hand != oldHand);
    return -1; // Cannot find a block to evict
}

```

3. zadatak, kolokvijum, jun 2021.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu implementira se keš blokova sa blokovskih uređaja („diskova“) kodom koji je dat u nastavku. Za svaki uređaj sa datim identifikatorom pravi se jedan objekat klase BlockIOCache, inicijalizovan tim identifikatorom, koji predstavlja keš blokova sa tog uređaja. Keš je kapaciteta CACHESIZE blokova veličine BLKSIZE. Keš je interno organizovan kao heš mapa map sa MAPSIZE ulaza. Svaki ulaz niza map sadrži glavu liste keširanih blokova koji se preslikavaju u taj ulaz. Funkcija hash je heš funkcija koja preslikava broj bloka u ulaz u nizu map. Glava liste, kao i pokazivač na sledeći element u listi čuvaju se kao indeksi elementa niza entries koji sadrži keširane blokove; vrednost -1 označava kraj (null). Svaki element niza entries je struktura tipa CacheEntry u kojoj je polje blkNo broj bloka koji je keširan u tom elementu, polje next ukazuje na sledeći element liste, a polje buf je sadržaj samog bloka.

Na početku složene operacije sa uređajem, kod koji koristi keš najpre traži da je potrební blok učitán pozivom funkcije getBlock koja vraća pokazivač na niz bajtova u baferu – učitánom bloku. Pošto više ovakvih složenih operacija može biti ugnježdjeno, blok iz keša može biti izbačen (zamenjen drugim) samo ako ga više niko ne koristi, što se realizuje brojanjem referenci u polju refCounter strukture CacheEntry.

Član freeHead je glava liste slobodnih elemenata u nizu entries (-1 ako slobodnih nema). Funkcija evict izbacuje jedan keširani blok iz punog keša, ukoliko takav može da nađe, oslobađa njegov ulaz i stavlja ga u listu slobodnih.

Implementirati funkciju getBlock koja treba da obezbedi da je traženi blok u kešu, odnosno učitá ga ako nije. Ako nema mesta u kešu jer nijedan blok ne može da se izbací, treba vratiti null. Ostale članice date klase su implementirane, a na raspolaganju je i funkcija koja učitava blok na dati uređaj:

```

void ioRead(int device, BlkNo blk, Byte* buffer);

typedef unsigned char Byte; // Unit of memory
typedef long long BlkNo; // Device block number
const unsigned BLKSIZE = ...; // Block size in Bytes
class BlockIOCache {

```

```

public:
    BlockIOCache(int device);
    Byte* getBlock(BlkNo blk);
    ...
protected:
    static int hash(BlkNo);
    void evict();
private:
    static const unsigned CACHESIZE = ...; // Cache size in blocks
    static const unsigned MAPSIZE = ...; // Hash map size in entries
    struct CacheEntry {
        BlkNo blkNo;
        int next;
        int refCounter;
        Byte buf[BLKSIZE];
    };
    int dev;
    int map[MAPSIZE]; // Hash map
    CacheEntry entries[CACHESIZE]; // Cache
    int freeHead; // Free entries list head
};
    
```

Rešenje

```

Byte* BlockIOCache::getBlock(BlkNo blk) {
    // Find the requested block in the cache and return it if present:
    int entry = hash(blk);
    for (int i = map[entry]; i != -1; i = entries[i].next) {
        if (entries[i].blkNo==blk) {
            entries[i].refCounter++;
            return entries[i].buf;
        }
    }
    // The block is not in the cache, find a free slot to load it:
    if (freeHead == -1) evict();
    if (freeHead == -1) return 0; // Error: cannot find space
    int free = freeHead;
    freeHead = entries[free].next;
    // Load the requested block:
    entries[free].blkNo = blk;
    entries[free].refCounter = 1;
    entries[free].next = map[entry];
    map[entry] = free;
    ioRead(dev, blk, entries[free].buf);
    return entries[free].buf;
}
    
```

1. zadatak, treći kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu implementira se keš blokova za ulazne, blokovski orijentisane uređaje sa direktnim pristupom; pošto su uređaji ulazni, blokovi se mogu samo čitati. Za svaki takav uređaj sa datim identifikatorom pravi se jedan objekat klase `BlockIOCache`, inicijalizovan tim identifikatorom, koji predstavlja keš blokova sa tog uređaja. Keš čuva najviše `CACHESIZE` blokova veličine `BLKSIZE` u nizu `cache`. Svaki ulaz i u nizu `cacheMap` sadrži broj bloka na uređaju koji se nalazi u ulazu i keša. Kada učitava blokove, keš najpre redom popunjava svoje ulaze, dok ima neiskorišćenih

ulaza; podatak član `numOfBlocks` govori o tome koliko je ulaza zauzeto (redom, prvih, od ulaza broj 0). Kada popuni sve ulaze, traženi blok koji nije u kešu učitava se na mesto bloka koji je najdavnije učitao (zamenjuje se taj blok učitanim blokom).

Na raspolaganju je operacija `ioRead` koja sa datog uređaja učitava blok sa zadatim brojem u bafer zadat poslednjim argumentom. Implementirati funkciju `BlockIOCache::read` koja treba da iz bloka za datim brojem `blk` niz bajtova počev od pozicije `offset` i dužine `sz` prepíše u bafer `buffer` koji je alocirao pozivalac. Pretpostaviti da su ovi argumenti ispravni i konzistentni (pozivalac je proverio njihovu ispravnost) i da se eventualne greške u operaciji `ioRead` obrađuju u njoj ili negde drugde (ignorirati ih). (Definicija klase `BlockIOCache` nije kompletna i može se dopunjavati po potrebi.)

```
typedef char byte; // Unit of memory
typedef long long BlkNo; // Device block number
void ioRead (int device, BlkNo blk, byte* buffer);

class BlockIOCache {
public:
    BlockIOCache (int device) : dev(device), numOfBlocks(0) {}
    void read (BlkNo blk, byte* buffer, size_t offset, size_t sz);

private:
    static const unsigned BLKSIZE = ...; // Block size in Bytes
    static const unsigned CACHESIZE = ...; // Cache size in blocks

    int dev;
    byte cache [CACHESIZE][BLKSIZE]; // Cache
    BlkNo cacheMap [CACHESIZE]; // Contents of the cache (block numbers)
    int numOfBlocks; // Number of used entries (blocks in the cache)
};
```

Rešenje

U klasi `BlockIOCache` potreban je još sledeći nestatički podatak član:

```
int BlockIOCache::toReplace = 0;

void BlockIOCache::read(BlkNo blk, byte* buffer, size_t offset, size_t sz)
{
    // Search for the requested block in the cache:
    int entry = -1;
    for (int i=0; i<this->numOfBlocks && entry<0; i++)
        if (this->cacheMap[i]==blk) entry = i; // Block found
    if (entry<0) {
        // The block is not in the cache, load it to the cache:
        if (this->numOfBlocks<CACHESIZE)
            entry = this->numOfBlocks++; // Load it to a free slot
        else {
            entry = this->toReplace++; // Replace the least recently loaded block
            this->toReplace %= CACHESIZE;
        }
        this->cacheMap[entry] = blk;
        ioRead(this->dev, blk, this->cache[entry]);
    }
    // Copy the extract to the buffer and return:
    for (size_t j=0; j<sz && offset+j<BLKSIZE; j++)
        buffer[j] = this->cache[entry][offset+j];
}
```

1. zadatak, treći kolokvijum, jun 2018.

- [Postavka](#)
- [Rešenje](#)

Dat je neki sekvencijalni, znakovno orijentisani ulazni uređaj (ulazni tok) sa koga se znak učitava sledećom funkcijom:

```
char getchar();
```

Od ovog uređaja napraviti apstrakciju sekvencijalnog, blokovski orijentisanog ulaznog uređaja, sa koga se blok veličine `BlockSize` učitava na zadatu adresu funkcijom:

```
void readBlock(char* addr);
```

Ignorirati sve greške.

Rešenje

```
void readBlock (char* addr) {
    for (int i=0; i<BlockSize; i++)
        addr[i] = getchar();
}
```

1. zadatak, treći kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu implementira se keš blokova sa blokovskih uređaja („diskova“) kodom koji je dat u nastavku. Za svaki uređaj sa datim identifikatorom pravi se jedan objekat klase `BlockIOCache`, inicijalizovan tim identifikatorom, koji predstavlja keš blokova sa tog uređaja. Keš čuva najviše `CACHESIZE` blokova veličine `BLKSIZE` u nizu `cache`. Svaki ulaz *i* u nizu `cacheMap` sadrži broj bloka na disku koji se nalazi u ulazu *i* keša. Flegovi u ulazu *i* niza `flags` ukazuju na to da je u ulazu *i* validan sadržaj (da ulaz nije prazan, odnosno već je bio korišćen i u njega je učitana sadržaj nekog bloka) – fleg `F_VALID`, odnosno da je sadržaj „zaprljan“, tj. menjan i treba ga upisati na disk – fleg `F_DIRTY`.

Kod koji koristi ovaj keš koristi ga na sledeći način za neku složenu operaciju sa uređajem:

```
Byte* buffer = diskCache->getBlock(blk);
...
buffer[...] = ...;
...
diskCache->writeBlock(buffer);
...
diskCache->releaseBlock(buffer);
```

Na početku ovakve složene operacije sa uređajem, ovaj kod najpre traži da keš obezbedi da je potrebni blok učitana pozivom funkcije `getDiskBlock` koja vraća pokazivač na niz bajtova u baferu – učitanoj bloku. Potom ovaj kod može da čita ili upisuje u sadržaj tog bafera. Ako je upisivao, mora da pozove funkciju `writeBlock`, kako bi keš znao da dati blok treba da upiše na uređaj. Kada završi celu operaciju, kod poziva funkciju `releaseBlock`, kako bi kešu stavio do znanja da on više ne koristi taj blok. Pošto više ovakvih složenih operacija može biti ugnježdjeno, blok iz keša može biti izbačen (zamenjen drugim) samo ako ga više niko ne koristi, što se realizuje brojanjem referenci u nizu `refCounter`.

Implementirati funkcije `getBlock` i `releaseBlock`. Funkcija `getBlock` treba da obezbedi da je traženi blok u kešu, odnosno učitava ga ako nije; ako ga treba učitati, to može biti urađeno u bilo koji ulaz keša koji sadrži blok koji nije validan ili nije više korišćen; ako takvog nema, treba vratiti 0 (greška, nema mesta u kešu). Na raspolaganju su sledeće dve funkcije koje vrše učitavanje, odnosno upis (respektivno) bloka na dati uređaj:

```
void ioRead(int device, BlkNo blk, Byte* buffer);
void ioWrite(int device, BlkNo blk, Byte* buffer);
typedef char Byte; // Unit of memory
typedef long long BlkNo; // Device block number
```

```

class BlockIOCache {
public:
    BlockIOCache (int device);

    Byte* getBlock (BlkNo blk);
    void writeBlock (Byte* buffer);
    void releaseBlock (Byte* buffer);

private:
    static const unsigned BLKSIZE = ...; // Block size in Bytes
    static const unsigned CACHESIZE = ...; // Cache size in blocks
    static const short F_VALID = 0x01; // Valid bit mask
    static const short F_DIRTY = 0x02; // Dirty bit mask

    int dev;
    Byte cache [CACHESIZE][BLKSIZE]; // Cache
    BlkNo cacheMap [CACHESIZE]; // Contents of the cache (block numbers)
    unsigned refCounter [CACHESIZE]; // Reference counters
    short flags [CACHESIZE]; // Valid, Dirty flags
};

BlockIOCache::BlockIOCache (int device) : dev(device) {
    for (int i=0; i<CACHESIZE; i++)
        this->cacheMap[i] = this->refCounter[i] = this->flags[i] = 0;
}

void BlockIOCache::writeBlock (Byte* buffer) {
    int i = (buffer-this->cache[0])/BLKSIZE;
    if (i<0 || i>=CACHESIZE) return; // Exception
    this->flags[i] |= F_DIRTY;
}

```

Rešenje

```

Byte* BlockIOCache::getBlock (BlkNo blk) {
    // Find the requested block in the cache and return it if present:
    int i, free = -1;
    for (i=0; i<CACHESIZE; i++) {
        if (this->flags[i]&F_VALID && this->cacheMap[i]==blk) {
            this->refCounter[i]++;
            return &this->cache[i];
        }
        if ((this->flags[i]&F_VALID==0 || this->refCounter[i]==0)
            && free==-1) free=i;
    }
    // The block is not in the cache, load it to the 'free' slot:
    if (free==-1) return 0; // A problem: there is no free space in the cache
    // Load the requested block:
    this->cacheMap[free] = blk;
    this->flags[free] = F_VALID;
    this->refCounter[free] = 1;
    ioRead(this->dev, blk, this->cache[free]);
    return this->cache[free];
}

void BlockIOCache::releaseBlock (Byte* buffer) {

```

```

int i = (buffer->this->cache[0])/BLKSIZE;
if (i<0 || i>=CACHESIZE) return; // Exception
// If the block is dirty, write it to the device:
if (this->flags[i]&F_VALID && this->flags[i]&F_DIRTY)
    ioWrite(this->dev, this->cacheMap[i], this->cache[i]);
this->flags[i] &= ~F_DIRTY;
this->refCounter[i]--;
}

```

1. zadatak, treći kolokvijum, septembar 2016.

- [Postavka](#)
- [Rešenje](#)

Dat je proceduralni interfejs prema nekom blokovski orijentisanom ulaznom uređaju sa direktnim pristupom:

```

extern const int BlockSize;
extern int BlockIOHandle;
long getSize(BlockIOHandle handle);
int readBlock(BlockIOHandle handle, long blockNo, char* addr);

```

Uređaj se identifikuje „ručkom“ tipa `BlockIOHandle`, a blok je veličine `BlockSize` znakova. Operacija `getSize` vraća ukupnu veličinu sadržaja (podataka) na uređaju (u znakovima), a operacija `readBlock` učitava blok sa zadatim brojem u bafer na zadatoj adresi u memoriji i vraća 0 u slučaju uspeha. Obe operacije vraćaju negativnu vrednost u slučaju greške, uključujući i pokušaj čitanja bloka preko granice veličine sadržaja.

Korišćenjem ovog interfejsa implementirati sledeći objektno orijentisani interfejs prema ovom uređaju, koji od njega čini apstrakciju ulaznog toka, odnosno znakovno orijentisanog ulaznog uređaja sa direktnim pristupom:

```

class IOStream {
public:
    IOStream (BlockIOHandle d);
    int seek (long offset);
    int getChar (char& c);
};

```

Operacija `seek` postavlja poziciju „kurzora“ za čitanje na zadatu poziciju (pomerač počev od znaka na poziciji 0), a operacije `getChar` čita sledeći znak sa tekuće pozicije kurzora u izlazni argument `c` i pomera kurzor za jedno mesto unapred. U slučaju bilo kakve greške, uključujući i pomeranje kurzora preko veličine sadržaja ili čitanje znaka kada je kurzor stigao do kraja sadržaja, operacije treba da vrate negativnu vrednost, a nulu u slučaju uspeha.

Rešenje

```

class IOStream {
public:
    IOStream (BlockIOHandle d) : dev(d), cursor(-1), curBlock(-1)
    { seek(0); }

    int seek (long offset);
    int getChar (char& c);
protected:
    int loadBlock(); // Helper; loads the block corresponding to the cursor
private:
    BlockIOHandle dev;
    char buffer[BlockSize];
    long curBlock, cursor;
};

int IOStream::loadBlock () {
    long blockNo = cursor/BlockSize;

```

```

    if (curBlock==blockNo) return 0;
    if (readBlock(dev,blockNo,buffer)<0) {
        cursor = -1;
        return -1;
    }
    curBlock = blockNo;
    return 0;
}

int IOStream::seek (int offset) {
    cursor = offset;
    if (cursor<0 || cursor>=getSize(dev)) {
        cursor = -1;
        return -1;
    }
    return 0;
}

int IOStream::getchar (char& c) {
    if (cursor<0 || cursor>=getSize(dev)) {
        cursor = -1;
        return -1;
    }
    if (loadBlock()<0) return -1;
    c = buffer[(cursor++)%BlockSize];
    return 0;
}

```

1. zadatak, treći kolokvijum, jun 2015.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu svaki drajver blokovski orijentisanog uređaja („diska“) registruje sledeću strukturu (tabelu) koja sadrži pokazivače na funkcije koje implementiraju odgovarajuće operacije sa tim uređajem:

```

typedef ... Byte; // Unit of memory
typedef ... BlkNo; // Disk block number

typedef int (*DiskOperation)(BlkNo block, Byte* buffer);

struct DiskOperationsTable {
    int isValid;
    DiskOperation readBlock, writeBlock;
    DiskOperationsTable () : isValid(0), readBlock(0), writeBlock(0) {}
};

```

Sistem organizuje tabelu registrovanih drajvera za priključene uređaje kao niz ovih struktura, s tim da polje `isValid==1` označava da je dati element niza zauzet (validan, postavljen, odnosno disk je registrovan), a 0 da je ulaz slobodan (disk nije registrovan):

```

const int MaxNumOfDisks; // Maximal number of registered disk devices
DiskOperationsTable disks[MaxNumOfDisks];

```

Sistem preslikava simbolička imena dodeljena priključenim uređajima, u obliku slova abecede, brojevima ulaza u tabeli `disks` (u opsegu od 0 do `MaxNumOfDisks-1`).

1. Realizovati funkcije:

```
int readBlock(int diskNo, BlkNo block, Byte* buffer);
int writeBlock(int diskNo, BlkNo block, Byte* buffer);
```

koje treba da pozovu odgovarajuću implementaciju operacije drajvera (polimorfno, dinamičkim vezivanjem) za zadati uređaj. (Ove funkcije poziva interna kernel nit kada opslužuje zahteve za operacijama sa diskovima, da bi inicijalizovala prenos na odgovarajućem uređaju.)

2. Realizovati funkciju koja registruje operacije drajvera za dati disk:

```
int registerDriver(int diskNo, DiskOperation read, DiskOperation write);
```

U slučaju greške, sve ovde navedene funkcije vraćaju negativnu vrednost, a u slučaju uspeha vraćaju 0.

Rešenje

```
int readBlock(int diskNo, BlkNo block, Byte* buffer) {
    if (diskNo<0 || diskNo>=MaxNumOfDisks) return -1; // Error
    if (disks[diskNo].isValid==0) return -2; // Error
    if (disks[diskNo].readBlock==0) return -3; // Error
    return (disks[diskNo].readBlock)(block,buffer);
}

int writeBlock(int diskNo, BlkNo block, Byte* buffer) {
    if (diskNo<0 || diskNo>=MaxNumOfDisks) return -1; // Error
    if (disks[diskNo].isValid==0) return -2; // Error
    if (disks[diskNo].writeBlock==0) return -3; // Error
    return (disks[diskNo].writeBlock)(block,buffer);
}

int registerDriver(int diskNo, DiskOperation read, DiskOperation write) {
    if (diskNo<0 || diskNo>=MaxNumOfDisks) return -1; // Error
    if (disks[diskNo].isValid) return -2; // Error
    disks[diskNo].isValid = 1;
    disks[diskNo].readBlock = read;
    disks[diskNo].writeBlock = write;
}
```

1. zadatak, treći kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

Na neki sekvencijalni, blokovski orijentisani izlazni uređaj identifikovan „ručkom“ `handle` se blok znakova veličine `BlockSize` ispisuje sa zadate adrese u memoriji funkcijom:

```
void writeBlock(IOHandle handle, char* addr);
```

Od ovog uređaja napraviti apstrakciju sekvencijalnog, znakovno orijentisanog izlaznog uređaja (izlazni tok), odnosno realizovati funkciju koja ispisuje znak po znak na taj uređaj:

```
void putchar(IOHandle handle, char);
```

Ignorirati sve greške.

Rešenje

```
void putchar (IOHandle handle, char c) {
    static char buffer[BlockSize];
    static int cursor = 0;
    buffer[cursor++] = c;
    if (cursor==BlockSize) {
        writeBlock(handle,buffer);
    }
}
```



```

    cursor = 0;
}
}

```

1. zadatak, treći kolokvijum, jun 2014.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu svaki drajver blokovski orijentisanog uređaja („diska“) registruje sledeću strukturu (tabelu) koja sadrži pokazivače na funkcije koje implementiraju odgovarajuće operacije sa tim uređajem:

```

typedef ... Byte; // Unit of memory
typedef ... BlkNo; // Disk block number

struct DiskOperationsTable {
    int (*readBlock)(BlkNo block, Byte* buffer);
    int (*writeBlock)(BlkNo block, Byte* buffer);
};

```

Sistem organizuje tabelu registrovanih drajvera za priključene uređaje kao niz pokazivača na ove strukture:

```
const int MaxNumOfDisks; // Maximal number of registered disk devices
```

```
DiskOperationsTable* disks[MaxNumOfDisks];
```

Sistem preslikava simbolička imena dodeljena priključenim uređajima, u obliku slova abecede, brojevima ulaza u tabeli disks (u opsegu od 0 do MaxNumOfDisks-1). Ukoliko uređaj sa datim simboličkim imenom nije priključen, odgovarajući ulaz u ovoj tabeli je *null*.

Realizovati funkcije:

```

int readBlock(int diskNo, BlkNo block, Byte* buffer);
int writeBlock(int diskNo, BlkNo block, Byte* buffer);

```

koje treba da pozovu odgovarajuću implementaciju operacije drajvera (polimorfno, dinamičkim vezivanjem) za zadati uređaj. (Ove funkcije poziva interna kernel nit kada opslužuje zahteve za operacijama sa diskovima, da bi inicijalizovala prenos na odgovarajućem uređaju.) U slučaju greške, sve ovde navedene funkcije vraćaju negativnu vrednost, a u slučaju uspeha vraćaju 0.

Rešenje

```

int readBlock(int diskNo, BlkNo block, Byte* buffer) {
    if (diskNo<0 || diskNo>=MaxNumOfDisks) return -1; // Error
    if (disks[diskNo]==NULL) return -1; // Error
    return (disks[diskNo]->readBlock)(block,buffer);
}

int writeBlock(int diskNo, BlkNo block, Byte* buffer) {
    if (diskNo<0 || diskNo>=MaxNumOfDisks) return -1; // Error
    if (disks[diskNo]==NULL) return -1; // Error
    return (disks[diskNo]->writeBlock)(block,buffer);
}

```

1. zadatak, treći kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

Dat je neki sekvencijalni, blokovski orijentisani ulazni uređaj sa koga se blok znakova veličine `BlockSize` učitava na zadatu adresu funkcijom:

```
void readBlock(char* addr);
```

Od ovog uređaja napraviti apstrakciju sekvencijalnog, znakovno orijentisanog ulaznog uređaja (ulazni tok), odnosno realizovati funkciju koja učitava znak po znak sa tog uređaja:

```
char getchar();
```

Ignorisati sve greške.

Rešenje

```
char getchar () {
    static char buffer[BlockSize];
    static int cursor = BlockSize;
    if (cursor==BlockSize) {
        readBlock(buffer);
        cursor = 0;
    }
    return buffer[cursor++];
}
```

1. zadatak, treći kolokvijum, jun 2013.

- [Postavka](#)
- [Rešenje](#)

Neki sistem organizuje keš blokova sa diska na sledeći način. Keš čuva najviše `CACHESIZE` blokova veličine `BLKSIZE` u nizu `diskCache`. Svaki ulaz *i* u nizu `diskCacheMap` sadrži broj bloka na disku koji se nalazi u ulazu *i* keša. Vrednost 0 u nekom ulazu niza `diskCacheMap` označava da je taj ulaz prazan (u njega nije učitani blok). Data je sledeća implementacija keša:

```
typedef ... Byte; // Unit of memory
typedef ... BlkNo; // Disk block number
const int BLKSIZE = ...; // Disk block size in Bytes
const int CACHESIZE = ...; // Disk cache size in blocks

Byte diskCache [CACHESIZE][BLKSIZE]; // Disk cache
BlkNo diskCacheMap [CACHESIZE]; // The contents of disk cache. 0 for empty

Byte* getDiskBlock (BlkNo blk) {
    // Search for the requested block in the cache and return it if found:
    int hash = blk%CACHESIZE;
    int cursor = hash;
    for (int i=0; i<CACHESIZE; i++) {
        cursor = (hash+i)%CACHESIZE;
        if (diskCacheMap[cursor]==blk) return diskCache[cursor];
        if (diskCacheMap[cursor]==0) break;
    }
    // Not found.
    if (diskCacheMap[cursor]!=0) cursor=hash; // Cache full
    // If there is a block to evict, write it to the disk:
    if (diskCacheMap[cursor]!=0)
        diskWrite(diskCacheMap[cursor], diskCache[cursor]);
    // Load the requested block:
    diskCacheMap[cursor] = blk;
    diskRead(blk, diskCache[cursor]);
    return diskCache[cursor];
}
```

Operacije `diskRead` i `diskWrite` vrše sinhrono čitanje, odnosno upis datog bloka sa diska.

Funkcija `getDiskBlock` vraća pokazivač na deo memorije u kome se nalazi učitani traženi blok diska, pri čemu se taj blok učitava u keš (uz prethodno snimanje eventualno izbačenog bloka) ukoliko taj blok veo nije u kešu. Ovu funkciju koristi ostatak sistema za pristup blokovima diska. Ova operacija implementira keš kao heš (*hash*) tabelu, s tim da koliziju rešava otvorenim adresiranjem.

Modifikovati funkciju `getDiskBlock` tako da se svaki blok na disku uvek smešta u isti ulaz u kešu, određen istom navedenom heš funkcijom.

Rešenje

```
Byte* getDiskBlock (BlkNo blk) {
    // Find the requested block in the cache and return it if present:
    int hash = blk%CACHE_SIZE;
    if (diskCacheMap[hash]==blk) return diskCache[hash];
    // The block is not in the cache.
    // If there is a block to evict, write it to the disk:
    if (diskCacheMap[hash]!=0)
        diskWrite(diskCacheMap[hash],diskCache[hash]);
    // Load the requested block:
    diskCacheMap[hash] = blk;
    diskRead(blk,diskCache[hash]);
    return diskCache[hash];
}
```

1. zadatak, treći kolokvijum, jun 2012.

- [Postavka](#)
- [Rešenje](#)

Neki sistem organizuje keš blokova sa diska na sledeći način. Keš čuva najviše `CACHE_SIZE` blokova veličine `BLK_SIZE` u nizu `diskCache`. Svaki ulaz *i* u nizu `diskCacheMap` sadrži broj bloka na disku koji se nalazi u ulazu *i* keša. Vrednost 0 u nekom ulazu niza `diskCacheMap` označava da je taj ulaz prazan (u njega nije učitani blok). Data je sledeća implementacija keša:

```
typedef ... Byte; // Unit of memory
typedef ... BlkNo; // Disk block number
const int BLK_SIZE = ...; // Disk block size in Bytes
const int CACHE_SIZE = ...; // Disk cache size in blocks

BYTE diskCache [CACHE_SIZE][BLK_SIZE]; // Disk cache
BlkNo diskCacheMap [CACHE_SIZE]; // The contents of disk cache. 0 for empty

int diskCacheCursor = 0; // FIFO cursor for eviction/loading

Byte* getDiskBlock (BlkNo blk) {
    // Search for the requested block in the cache and return it if found:
    for (int i=0; i<CACHE_SIZE; i++) {
        if (diskCacheMap[i]==blk) return diskCache[i];
        if (diskCacheMap[i]==0) break;
    }
    // Not found.
    // If there is a block to evict, write it to the disk:
    if (diskCacheMap[diskCacheCursor]!=0)
        diskWrite(diskCacheMap[diskCacheCursor],diskCache[diskCacheCursor]);
    // Load the requested block:
    diskCacheMap[diskCacheCursor] = blk;
    diskRead(blk,diskCache[diskCacheCursor]);
    Byte* ret = diskCache[diskCacheCursor];
}
```

```

    diskCacheCursor = (diskCacheCursor+1)%CACHESIZE;
    return ret;
}

```

Operacije `diskRead` i `diskWrite` vrše sinhrono čitanje, odnosno upis datog bloka sa diska:

```

void diskRead(BlkNo block, Byte* toBuffer);
void diskWrite(BlkNo block, Byte* fromBuffer);

```

Ukoliko je keš pun, a treba učitati novi blok, iz keša se izbacuje blok iz ulaza na koji ukazuje kurzor `diskCacheCursor` koji se pomera u krug, tako da je izbacivanje (engl. *eviction*) po FIFO (*first-in-first-out*) principu.

Funkcija `getDiskBlock` vraća pokazivač na deo memorije u kome se nalazi učitani traženi blok diska, pri čemu se taj blok učitava u keš (uz prethodno snimanje eventualno izbačenog bloka) ukoliko taj blok veo nije u kešu. Ovu funkciju koristi ostatak sistema za pristup blokovima diska.

Ova implementacija keša je nedovoljno efikasna jer se neki blok sa diska može smestiti u bilo koji ulaz keša i zato pronalaženje bloka u punom kešu često zahteva obilaženje velikog dela niza `diskCacheMap`. Zbog toga treba promeniti implementaciju ovog keša tako da se `diskCacheMap` organizuje kao heš (*hash*) tabela. Disk blok broj `b` se smešta u ulaz `hash(b)=b mod CACHESIZE`, ukoliko je taj ulaz slobodan. U slučaju da nije (tj. u slučaju kolizije), taj blok se smešta u prvi naredni slobodan blok (u krug, po modulu `CACHESIZE`; rešavanje kolizije otvorenim adresiranjem).

Dati ovako izmenjenu funkciju `getDiskBlock`.

Rešenje

```

Byte* getDiskBlock (BlkNo blk) {
    // Search for the requested block in the cache and return it if found:
    int hash = blk%CACHESIZE;
    int cursor = hash;
    for (int i=0; i<CACHESIZE; i++) {
        cursor = (hash+i)%CACHESIZE;
        if (diskCacheMap[cursor]==blk) return diskCache[cursor];
        if (diskCacheMap[cursor]==0) break;
    }
    // Not found.
    if (diskCacheMap[cursor]!=0) cursor=hash; // Cache full
    // If there is a block to evict, write it to the disk:
    if (diskCacheMap[cursor]!=0)
        diskWrite(diskCacheMap[cursor],diskCache[cursor]);
    // Load the requested block:
    diskCacheMap[cursor] = blk;
    diskRead(blk,diskCache[cursor]);
    return diskCache[cursor];
}

```

1. zadatak, treći kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Dat je interfejs klase koja apstrahuje jedan sekvencijalni, blokovski, ulazni uređaj sa blokom veličine `BlockSize` znakova:

```

class BlockDevice {
public:
    void open (); // Open the device channel
    void close (); // Close the device channel
    void loadBlock (char* buffer); // Read next block to the given buffer
};

```

Implementirati klasu `CharDevice` sa dole datim interfejsom koja adaptira interfejs datog blokovskog ulaznog uređaja u interfejs sekvencijalnog, ali znakovnog ulaznog uređaja. Konstruktor i destruktor ove klase treba da otvaraju, odnosno zatvaraju dati blokovski uređaj.

```
class CharDevice {
public:
    CharDevice (BlockDevice* bd);
    ~CharDevice ();
    char getChar ();
};
```

Korišćenjem klase `CharDevice` ilustrovati primerom učitavanje niza znakova sa ulaznog uređaja sve dok se ne učitava znak `'\0'`.

Rešenje

```
class CharDevice {
public:
    CharDevice (BlockDevice* bd) : myDev(bd), cursor(BlockSize) {
        if (myDev) myDev->open();
    }

    ~CharDevice () { if (myDev) myDev->close(); }

    char getChar ();

protected:
    void load ();

private:
    BlockDevice* myDev;
    char buffer[BlockSize];
    int cursor;
};

void CharDevice::load () {
    if (myDev && cursor==BlockSize) {
        myDev->loadBlock(buffer);
        cursor=0;
    }
}

char CharDevice::getChar () {
    load();
    if (cursor<BlockSize) return buffer[cursor++];
    else return '\0'; // Exception;
}

void main () {
    BlockDevice bd = ...;
    CharDevice input(bd);
    for (char c=input.getchar(); c!='\0'; c=input.getchar())
        ...
}
```

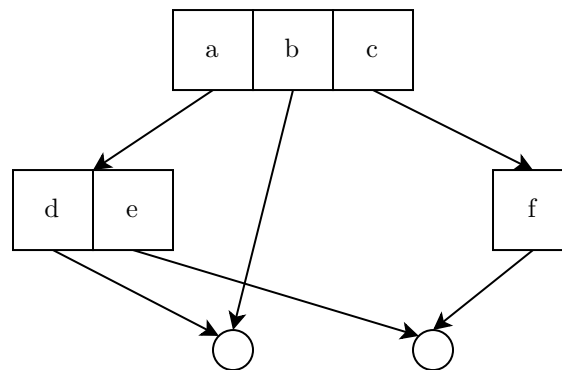
Komandna linija

2. zadatak, treći kolokvijum, jun 2024.

- [Postavka](#)
- [Rešenje](#)

Ispisati sekvencu Unix komandi kojima korisnik, kroz CLI, može da napravi strukturu (pod)direktorijuma (na slici predstavljeni pravougaonikima) i fajlova (na slici predstavljeni kružićima) koja je prikazana na datoj slici, unutar tekućeg direktorijuma koji je inicijalno prazan; „koreni“ direktorijum prikazan na slici predstavlja taj tekući direktorijum (u njemu treba formirati ulaze *a*, *b*, i *c*). Fajlove praviti tako da im se sadržaj učitava sa konzole (unos znaka Ctrl-D označava kraj ulaznog toka na konzoli); ne pisati unos koji se unosi na konzoli za sadržaj fajla. Sve strelice na slici predstavljaju tvrde veze (*hard links*). Na raspolaganju su sledeće komande:

- `mkdir <directory>`: pravi navedeni direktorijum sa podrazumevanim pravima pristupa; pretpostaviti da su ta podrazumevana prava pristupa dovoljna da tekući korisnik uradi sve potrebne komande;
- `cat`: ukoliko nije naveden ulazni fajl, ovaj sistemski program sadržaj koji daje na svoj standardni izlaz uzima sa svog standardnog ulaza;
- `ln <original_filename> <link_name>`: pravi tvrdnu vezu `link_name` ka fajlu `original_filename`;
- `cd <target_dir>`: menja tekući direktorijum na zadati.



Slika 1: Slika iz zadatka 2.

Rešenje

```
mkdir a
cat > b
mkdir c
ln b a/d
cat > a/e
ln a/e c/f
```

2. zadatak, treći kolokvijum, avgust 2023.

- [Postavka](#)
- [Rešenje](#)

Dat je podsetnik na neke osnovne Unix komande:

- `cat`: iz svakog fajla koji je naveden kao argument ove komande, redom kojim su oni navedeni, učitava znakove i ispisuje ih na standardni izlaz; ukoliko nema argumenata, znakove učitava sa standardnog ulaza (dok ne naiđe na znak EOF koji se na konzoli signalizira pritiskom tastera Ctrl+D).
- `ln <src_file> <dst_file>`: pravi novu tvrdnu vezu (ulaz u direktorijumu) za postojeći fajl `src_file` u određitu definisanom sa `dst_file`; opcija `-s` radi isto to, samo što pravi meku (simboličku) vezu.
- `rm`: briše fajl ili direktorijum ili vezu zadat kao argument.

Izvršavaju se sledeće komande po datom redosledu; napisati te komande ili odgovoriti na postavljeno pitanje:

1. U postojećem poddirektorijumu **a** tekućeg direktorijuma napraviti nov tekstualni fajl **doc** čiji će sadržaj biti učitani sa konzole: _____
2. U postojećem poddirektorijumu **b** tekućeg direktorijuma napraviti tvrdi vezu pod nazivom **hdoc** na fajl napravljen u koraku 1: _____
3. U postojećem poddirektorijumu **c** tekućeg direktorijuma napraviti meku vezu pod nazivom **sdoc** na fajl napravljen u koraku 2: _____
4. Obrisati **doc** u poddirektorijumu **a**: _____
5. Šta će nakon ovoga ispisati komanda **cat b/hdoc**? _____
6. Nakon izdate komande **rm b/hdoc**, šta će ispisati komanda **cat c/sdoc**? _____

Rešenje

1. **cat > a/doc**
2. **ln a/doc b/hdoc**
3. **ln -s b/hdoc c/sdoc**
4. **rm a/doc**
5. Komanda **cat b/hdoc** će ispisati uneti sadržaj učitani sa tastature i upisan u fajl u prvoj komandi.
6. Nakon komande **rm b/hdoc**, komanda **cat c/sdoc** će ispisati grešku (fajl ne postoji).

2. zadatak, treći kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

Sistemi bazirani na sistemu Unix podržavaju strukture direktorijuma tipa acikličnog usmerenog grafa (DAG) pomoću dve vrste referenci na fajl kao objekat u fajl sistemu:

- *soft (symbolic) link*: „meka (ili simbolička) veza“ predstavlja fajl posebnog tipa čiji sadržaj čuva proizvoljnu (apsolutnu ili relativnu) putanju do nekog drugog fajla, poput „prečice“ (*shortcut*); svaku operaciju (komandu) nad ovakvom vezom sistem preusmerava na referencirani fajl, osim komande **rm** za brisanje – brisanje simboličke veze briše samu vezu, ne i referencirani fajl; ukoliko referencirani fajl nestane ili se premesti, veza ostaje neažurna, „viseća“;
- *hard link*: „tvrda veza“ predstavlja jedan ulaz u direktorijumu koji referencira određeni fajl kao objekat (tj. njegov *inode*); na jedan fajl može ukazivati više tvrdih veza; komanda **rm** uklanja tvrdi vezu, a sam fajl se implicitno briše iz sistema kada nestane poslednja tvrda veza na njega.

Iz komandne linije mogu se izvršiti sledeće sistemske komande:

ls	prikazuje sadržaj tekućeg direktorijuma (<i>list</i>)
cd <dir>	menja tekući direktorijum (<i>change directory</i>)
ln <src> <dst>	za postojeći fajl sa zadatom stazom <src> kreira novu tvrdi vezu sa datom stazom <dst> (<i>link</i>)
ln -s <src> <dst>	za postojeći fajl sa zadatom stazom <src> kreira novu meku vezu sa datom stazom <dst>
rm <file>	briše ulaz sa zadatim imenom iz tekućeg direktorijuma; ukoliko je to poslednja tvrda veza na fajl, briše se i sam fajl.

Sve staze mogu biti apsolutne ili relativne. Zabeležena je sledeća sesija jednog korisnika:

```
$ cd /home/docs
$ ls
bar foo txt
$ cd /home/pics
$ ls
jane john chld
$ ln /home/docs/bar foo
$ rm john
$ cd /home/docs
$ rm bar
```

```
$ cd /home/pics
$ ls
```

1. Napisati izlaz poslednje komande.
2. Nakon prikazane sekvence zadate su sledeće komande:

```
$ ln -s /home/docs/foo bar
$ rm /home/docs/foo
$ rm foo
$ cd /home/docs
$ ls
```

Napisati izlaz poslednje komande.

Rešenje

1. jane chld foo
2. txt

2. zadatak, treći kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem podržava strukture direktorijuma tipa acikličnog usmerenog grafa (DAG). Iz komandne linije mogu se izvršiti sledeće sistemske komande:

dir	prikazuje sadržaj tekućeg direktorijuma
cd <dir>	menja tekući direktorijum
alias <file> <newname>	za postojeći fajl sa zadatom stazom <file> kreira novi ulaz u tekućem direktorijum pod datim novim imenom <newname> (<i>link</i>)
rm <file>	briše ulaz sa zadatim imenom iz tekućeg direktorijuma; ukoliko je to poslednja referenca na fajl, briše se i sam fajl.

Zabeležena je sledeća sesija jednog korisnika:

```
$ cd /home/docs
$ dir
bar <file>
foo <file>
txt <dir>
$ cd /home/pics
$ dir
jane <file>
john <file>
chld <dir>
$ alias /home/docs/bar foo
$ rm john
$ cd /home/docs
$ rm bar
$ cd /home/pics
$ dir
```

1. Napisati izlaz poslednje komande.
2. Nakon prikazane sekvence zadate su sledeće komande:

```
$ rm foo
$ cd /home/docs
$ dir
```

Napisati izlaz poslednje komande.

Rešenje

1. jane <file>
chld <dir>
foo <file>
2. foo <file>
txt <dir>

Fajl sistem (interfejs)

1. zadatak, treći kolokvijum, jun 2024.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem bibliotčnih funkcija *popen*, *dup2* i *fileno* implementirati funkciju *redirect* čiji je potpis dat i koja pokreće proces dete nad programom u fajlu sa putanjom zadatom argumentom *exe*, a onda preusmerava standardni izlaz pozivajućeg (svog) procesa na standardni ulaz tog procesa deteta; u slučaju neuspeha vraća -1, u slučaju uspeha vraća 0. Bibliotčna funkcija *fileno* vraća celobrojni deskriptor fajla (*fd*) koji odgovara zadatom znakovnom toku stream.

```
int fileno(FILE* stream);
int redirect(const char* exe);
```

Rešenje

```
int redirect(const char* exe) {
    FILE* out = popen(exe, "w");
    if (!out) return -1;
    int fd = fileno(out);
    dup2(fd, 1);
    return 0;
}
```

2. zadatak, treći kolokvijum, jun 2023.

- [Postavka](#)
- [Rešenje](#)

U sistemima nalik sistemu Unix cevovod (*pipe*) se formira sistemskim pozivom *pipe*:

```
#include <unistd.h>
int pipe (int pipefd[2]);
```

Ovaj sistemski poziv pravi cevovod i u *pipefd[0]* upisuje deskriptor fajla (*file descriptor*) odredišne strane otvorenog cevovoda (strana za čitanje), a u *pipefd[1]* upisuje deskriptor fajla izvorišne strane tog cevovoda (strana za upis). U slučaju greške, ovaj poziv vraća negativnu vrednost, a u slučaju uspeha vraća 0. Dati su i sistemski pozivi za čitanje i upis:

```
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, void *buf, size_t count);
```

Ovi pozivi prenose najviše *count* znakova sa fajlom koga identifikuje *fd* iz bafera ili u bafer na koga ukazuje *buf*, a vraćaju stvarno prenesen broj znakova.

Korišćenjem ovih, kao i drugih potrebnih sistemskih poziva i bibliotčnih funkcija (*fork*, *exit*, *close*, *putchar*, *fprintf*), napisati program koji, kada se nad njim pokrene proces, formira jedan cevovod kojim komunicira sa procesom detetom koga pokreće. Nakon toga, proces roditelj kroz cevovod procesu detetu šalje niz znakova dat svojim argumentom komandne linije *argv[1]* i završen znakom '\0', a proces dete dobijeni niz znakova ispisuje na svoj standardni izlaz. Nakon što uradi navedeno, svaki od procesa se gasi. Greške obraditi ispisom na *stderr* i gašenjem procesa.

Rešenje

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>

#define handle_error(msg) \
```

```

do { fprintf(stderr, "Error: %s\n", msg); exit(-1); } while(0)

int main(int argc, const char* argv[]) {
    if (argc != 2) handle_error("Missing argument.");
    int pipefd[2];
    if (pipe(pipefd) < 0) handle_error("Cannot create a pipe.");

    pid_t pid = fork();
    if (pid < 0) handle_error("Cannot create a child process.");
    if (pid > 0) {
        write(pipefd[1], argv[1], strlen(argv[1]));
    } else {
        char c;
        while (read(pipefd[0], &c, 1) > 0)
            putchar(c);
    }
    close(pipefd[0]);
    close(pipefd[1]);
    exit(0);
}

```

2. zadatak, treći kolokvijum, jun 2022.

- [Postavka](#)
- [Rešenje](#)

Neprazno binarno stablo čiji su čvorovi tipa `Node` zapisano je u binarni fajl na sledeći način (prikazan je pojednostavljen kod bez obrade grešaka): za svaki čvor najpre je zapisan njegov sadržaj tipa `NodeData`, zatim jedan `int` indikator koji kaže da li taj čvor ima svoje levo podstablo, zatim jedan `int` indikator koji kaže da li taj čvor ima svoje desno podstablo, a onda isto tako redom celo levo podstablo ako ga ima, pa celo desno podstablo ako ga ima. Korišćenjem istog POSIX API za fajlove napisati kod funkcije `readTree` za učitavanje i izgradnju stabla iz takvog fajla. Prvi parametar je staza do fajla, a drugi parametar je adresa pokazivača u koji treba upisati adresu korenog čvora formiranog i učitano g stabla. Obraditi greške vraćanjem negativne vrednosti. POSIX API funkcija

```
ssize_t read(int fd, void *buf, size_t count);
```

vraća broj stvarno učitanih bajtova (može biti manji od zahtevanog ako se stigne do kraja fajla) ili negativnu vrednost u slučaju greške.

```
#include <fcntl.h>
```

```

struct Node {
    NodeData data;
    Node *left, *right;
};

void writeSubtree(int fd, Node* node) {
    write(fd, node->data, sizeof(NodeData));
    int ind = (node->left != 0);
    write(fd, ind, sizeof(int));
    ind = (node->right != 0);
    write(fd, ind, sizeof(int));
    if (node->left) {
        writeSubtree(fd, node->left);
    }
    if (node->right) {
        writeSubtree(fd, node->right);
    }
}

void writeTree(const char* pathname, Node* root) {

```

```

    int fd = open(*pathname, O_WRONLY | O_CREATE | O_TRUNC);
    int r = writeSubtree(fd, root);
    close(fd);
}
int readTree(const char* pathname, Node** root);

```

Rešenje

```

#include <fcntl.h>
int readSubtree(int fd, Node** node) {
    static NodeData data;
    static int left, right;
    ssize_t cnt = read(fd, &data, sizeof(NodeData));
    if (cnt < sizeof(NodeData)) {
        return -2;
    }
    cnt = read(fd, &left, sizeof(int));
    if (cnt < sizeof(int)) {
        return -2;
    }
    cnt = read(fd, &right, sizeof(int));
    if (cnt < sizeof(int)) {
        return -2;
    }
    *node = new Node();
    if (*node == 0) {
        return -3;
    }
    (*node)->data = data;
    (*node)->left = (*node)->right = 0;
    int r = 0;
    if (left) {
        r = readSubtree(fd, &((*node)->left));
        if (r < 0) {
            return r;
        }
    }
    if (right) {
        r = readSubtree(fd, &((*node)->right));
        if (r < 0) {
            return r;
        }
    }
    return 0;
}
int readTree(const char* pathname, Node** root) {
    int fd = open(*pathname, O_RDONLY);
    if (fd < 0) {
        return -1;
    }
    int r = readSubtree(fd, root);
    close(fd);
    return r;
}

```

4. zadatak, kolokvijum, jun 2021.

- [Postavka](#)
- [Rešenje](#)

Na raspolaganju je POSIX API za fajlove:

```
int open(const char* path, int flags);
int close(int fd);
ssize_t read(int fd, void *buf, size_t count);
```

Tip `ssize_t` je označen celobrojni tip, isti kao `size_t`, samo što može da prihvati i vrednost -1. Funkcija `read` vraća stvarno učitano broj bajtova (0 ili više); ako je taj broj manji od `count`, stiglo se do kraja fajla. Funkcije `open`, `close` i `read` vraćaju -1 u slučaju greške.

Korišćenjem ovog interfejsa realizovati apstrakciju `IFStream` kao klasu čiji je interfejs dat dole. Ova klasa apstrahuje ulazni znakovni tok vezan za fajl. Staza do fajla zadaje se parametrom konstruktora. Zatvaranje fajla treba obezbediti destruktorem. Operacija `getc` vraća jedan znak učitano sa toka. Kada se došlo do kraja fajla, operacija `eof` vraća `true`. Ukoliko je u nekoj ranijoj operaciji sa tokom, uključujući i otvaranje, došlo do greške, funkcije `err` i `eof` vraćaju `true`. Ukoliko je tok u stanju greške ili se stiglo do njegovog kraja, operacija `getc` treba da vraća `'\n'`. Učitavanje treba da bude što je moguće efikasnije u smislu da se sa uređaja učitava najmanji mogući broj blokova za sekvencijalni pristup. Veličina bloka u bajtovima je `BLOCK_SIZE`, a veličina znakova u bajtovima je `CHAR_SIZE`.

```
class IFStream {
public:
    IFStream(const char* path);
    ~IFStream();
    bool eof() const;
    bool err() const;
    char getc();
private:
    static const size_t BLOCK_SIZE = ..., CHAR_SIZE = 2;
};
```

Ova apstrakcija može se koristiti ovako:

```
IFStream str("myfile.txt");
while (!str.err() && !str.eof()) {
    char c = str.getc();
    ...
}
```

Rešenje

```
#include <sys/stat.h>
#include <fcntl.h>

class IFStream {
public:
    IFStream(const char* path);
    ~IFStream() {
        if (fd >= 0) close(fd);
    }
    bool eof() const { return isEOF; }
    bool err() const { return isErr; }
    char getc();
protected:
    inline void fetch();
private:
    static const size_t BLOCK_SIZE = ..., CHAR_SIZE = 2;
    int fd;
```

```

    bool isEOF, isErr;
    char buffer[(BLOCK_SIZE + CHAR_SIZE - 1)/CHAR_SIZE];
    ssize_t curPos, size;
};

IFStream::IFStream(const char* path) : isEOF(false), isErr(false),
                                       curPos(-1), size(-1) {
    fd = open(path, O_RDONLY);
    if (fd == -1) {
        isErr = isEOF = true;
        return;
    }
    fetch();
}

inline void IFStream::fetch() {
    size = read(fd, buffer, BLOCK_SIZE);
    if (size == -1) isErr = true;
    if (size <= 0) isEOF = true;
    curPos = 0;
}

char IFStream::getc() {
    if (!isErr && !isEOF && curPos < size) {
        char c = buffer[curPos++];
        if (curPos >= size)
            if (size < BLOCK_SIZE) isEOF = true;
            else fetch();
        return c;
    }
    return '\\-1';
}

```

4. zadatak, kolokvijum, jul 2020.

- [Postavka](#)
- [Rešenje](#)

Dva procesa, pošiljalac i primalac, komuniciraju slanjem, odnosno prijemom „poruke“ preko fajla kao deljenog objekta. Kada pripremi poruku određene veličine u svom baferu, pošiljalac kreira fajl sa nazivom „*buffer.bin*“ u roditeljskom direktorijumu svog tekućeg direktorijuma, upiše poruku u taj fajl i zatvori fajl. Kada primalac „vidi“ ovaj fajl, on iz njega pročita poslatu poruku, a potom obriše taj fajl. Zbog ovakve sinhronizacije, pošiljalac treba da funkcioniše ovako: ukoliko pri pokušaju kreiranja navedenog fajla zaključi da taj fajl već postoji (jer ga primalac još uvek nije obrisao), pošiljalac se uspavljuje 5 sekundi, ostavljajući priliku primaocu da pročita i obriše fajl, a onda pokušava ponovo. Oba procesa izvršavaju se u ime istog korisnika. Napisati kod funkcije

```
int send(const void *buffer, size_t size);
```

procesa pošiljaoca koju on poziva kada želi da pošalje poruku datu u baferu zadate veličine. Ukoliko dođe do bilo koje greške koju ne može da obradi, ova funkcija treba da vrati -1, a u slučaju uspeha vraća 0. Na raspolaganju su sledeći standardni POSIX sistemski pozivi:

- `unsigned int sleep(unsigned int sleep_time_in_seconds)`: uspavljuje (suspenduje) pozivajući proces na zadato vreme (u sekundama);
- `int close (int fd)`: zatvara fajl sa zadatim deskriptorom;
- `int write (int fd, const void *buffer, size_t size)`: upisuje dati sadržaj u fajl sa zadatim deskriptorom;
- `int open(const char *pathname, int flags, mode_t mode)`: otvara fajl sa zadatom putanjom; argument *flags* mora da uključi jedno od sledećih prava pristupa: `O_RDONLY`, `O_WRONLY`, ili `O_RDWR`. Ako je

uključen i fleg `O_CREAT`, fajl će biti kreiran ukoliko ne postoji; ako je pritom uključen i `O_EXCL`, a fajl već postoji, funkcija će vratiti grešku (-1), a kod greške postavljen u globalnoj sistemskoj promenljivoj `errno` biće jednak `EEXIST`. Argument mode definiše prava pristupa za fajl koji se kreira samo u slučaju da je uključen `O_CREAT`, i to na sledeći način:

<code>S_IRWXU</code>	0x0700	user (file owner) has read, write and execute permission
<code>S_IRUSR</code>	0x0400	user has read permission
<code>S_IWUSR</code>	0x0200	user has write permission
<code>S_IXUSR</code>	0x0100	user has execute permission
<code>S_IRWXG</code>	0x0070	group has read, write and execute permission
<code>S_IRGRP</code>	0x0040	group has read permission
<code>S_IWGRP</code>	0x0020	group has write permission
<code>S_IXGRP</code>	0x0010	group has execute permission
<code>S_IRWXO</code>	0x0007	others have read, write and execute permission
<code>S_IROTH</code>	0x0004	others have read permission
<code>S_IWOTH</code>	0x0002	others have write permission
<code>S_IXOTH</code>	0x0001	others have execute permission

Rešenje

```
const char* fname = "../buffer.bin";
const unsigned int SLEEP_TIME = 5;

int send(const void* buffer, size_t size) {
    int fd;
    while (1) {
        fd = open(fname, O_WRONLY | O_CREAT | O_EXCL, S_IRUSR | S_IWUSR);
        if (fd >= 0) break;
        if (errno != EEXIST) return -1;
        sleep(SLEEP_TIME);
    }
    ret = write(fd, buffer, size) < 0 ? -1 : 0;
    ret |= close(fd);
    return ret;
}
```

2. zadatak, treći kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

U nekom binarnom fajlu zapisano je ogromno binarno stablo; stablo je potpuno balansirano, ima n nivoa i tačno $2^n - 1$ čvorova, tj. svaki čvor osim lista ima tačno dva deteta. U svakom čvoru nalazi se jedna jedinstvena vrednost tipa `int`, a stablo je sortirano (levi potomci su manji, a desni veći od svakog čvora). Stablo je zapisano u sadržaju fajla kao niz, tako da svaki element niza sadrži samo vrednost u odgovarajućem čvoru, dok su deca čvora koji odgovara elementu i implicitno određena i nalaze se u elementima $2i + 1$ i $2i + 2$ tog niza; koren je u elementu 0.

Korišćenjem dole datih sistemskih poziva standardnog C fajl interfejsa, implementirati funkciju

```
int binary_search(const char* filename, unsigned n, int x);
```

koja u binarnom fajlu sa datim imenom, u kome je zapisano stablo sa n nivoa u opisanom formatu, binarnom pretragom traži vrednost datu x i vraća 1 ako je pronađe, odnosno 0 ako je ne pronađe. Ignorisati sve potencijalne greške u sistemskim pozivima. Sledeće funkcije deklarisanе su u zaglavlju `stdio`:

- `std::FILE* std::fopen(const char* filename, const char* mode)`; otvara fajl sa zadatim imenom u zadatom modalitetu; za čitanje je modalitet „r“, za upis „w“, a za otvaranje fajla u binarnom režimu treba dodati sufiks „b“ na modalitet;
- `int std::fclose(FILE*)`; zatvara dati fajl;

- `std::size_t fread(void* buffer, std::size_t size, std::size_t count, std::FILE* stream);` iz datog fajla, počev od tekuće pozicije kurzora, u zadati bafer učitava najviše `count` objekata, svaki veličine `size` i pomera kurzor fajla iza pročitanih objekata; vraća broj stvarno pročitanih objekata (manje od traženog u slučaju greške ili nailaska na kraj fajla);
- `int fseek(std::FILE* stream, long offset, int origin);` pomera kurzor datog fajla na pomera (poziciju) zadatu parametrom `offset` (u bajtovima, odnosno jedinicama koje vraća operator `sizeof`), u odnosu na položaj zadat parametrom `origin`; za pomera u odnosu na početak fajla, ovaj parametar treba da bude jednak konstanti `SEEK_SET`.

Rešenje

```
#include <cstdio>
using namespace std;

int binary_search (const char* filename, unsigned n, int x) {
    FILE* f = fopen(filename, "rb");
    long nodeIndex = 0;
    int nodeValue;
    for (unsigned i=0; i<n; i++) {
        fseek(f, nodeIndex*sizeof(nodeValue), SEEK_SET);
        fread(&nodeValue, sizeof(nodeValue), 1, f);
        if (x==nodeValue) { fclose(f); return 1; }
        if (x<nodeValue)
            nodeIndex = 2*nodeIndex + 1;
        else
            nodeIndex = 2*nodeIndex + 2;
    }
    fclose(f);
    return 0;
}
```

2. zadatak, treći kolokvijum, jun 2018.

- [Postavka](#)
- [Rešenje](#)

Dole je dat izvod iz dokumentacije za API za fajlove u GNU sistemima. Date deklaracije su u `<unistd.h>` i `<fcntl.h>`. Korišćenjem samo dole datih funkcija, realizovati sledeću funkciju koja prepisuje ceo sadržaj datog ulaznog fajla proizvoljne veličine u dati izlazni fajl, korišćenjem svog bafera određene veličine. U slučaju bilo kakve greške, funkcija treba da vrati negativnu vrednost, u suprotnom treba da vrati 0.

```
int fcopy (const char *filenamefrom, const char *filenameto);

int open (const char *filename, int flags);
```

*Creates and returns a new file descriptor for the file named by **filename**. Initially, the file position indicator for the file is at the beginning of the file. The **flags** argument controls how the file is to be opened. This is a bit mask; you create the value by the bitwise OR of the appropriate parameters (using the `'|'` operator in C) – the following macros:*

- `O_RDONLY`: *Open the file for read access.*
- `O_WRONLY`: *Open the file for write access.*
- `O_RDWR`: *Open the file for both reading and writing.*
- `O_CREAT`: *If set, the file will be created if it doesn't already exist.*
- `O_APPEND`: *The bit that enables append mode for the file. If set, then all write operations write the data at the end of the file, extending it, regardless of the current file position.*
- `O_TRUNC`: *Truncate the file to zero length.*

*The normal return value from **open** is a non-negative integer file descriptor. In the case of an error, a value of -1 is returned instead.*


```
int close (int filedes);
```

Closes the file descriptor `filedes`. The normal return value is 0. In the case of an error, a value of -1 is returned.

```
ssize_t
```

This data type is used to represent the sizes of blocks that can be read or written in a single operation. It is similar to `size_t`, but must be a signed type.

```
ssize_t read (int filedes, void *buffer, size_t size);
```

Reads up to `size` bytes from the file with descriptor `filedes`, storing the results in the `buffer`. (This is not necessarily a character string, and no terminating null character is added.)

The return value is the number of bytes actually read. This might be less than `size`; for example, if there aren't that many bytes left in the file or if there aren't that many bytes immediately available. The exact behavior depends on what kind of file it is. Note that reading less than `size` bytes is not an error.

A value of zero indicates end-of-file (except if the value of the `size` argument is also zero). This is not considered an error. If you keep calling `read` while at end-of-file, it will keep returning zero and doing nothing else.

If `read` returns at least one character, there is no way you can tell whether end-of-file was reached. But if you did reach the end, the next `read` will return zero. In case of an error, `read` returns -1.

```
ssize_t write (int filedes, const void *buffer, size_t size);
```

Writes up to `size` bytes from `buffer` to the file with descriptor `filedes`. The data in `buffer` is not necessarily a character string and a null character is output like any other character. The return value is the number of bytes actually written. This may be `size`, but can always be smaller. Your program should always call `write` in a loop, iterating until all the data is written. In the case of an error, `write` returns -1.

Rešenje

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
int fcopy (const char *filenamefrom, const char *filenameeto) {
    static const int BufferSize = 1024;
    char buffer[BufferSize];

    int ffrom = open(filenamefrom, O_RDONLY);
    if (ffrom < 0) return ffrom;

    int fto = open(filenameeto, O_WRONLY | O_CREAT | O_APPEND | O_TRUNC);
    if (fto < 0) {
        close(ffrom);
        return fto;
    }

    do {
        ssize_t numRead = read(ffrom, buffer, BufferSize);
        if (numRead < 0) {
            close(ffrom);
            close(fto);
            return numRead;
        }
        ssize_t numWritten = 0;
        while (numWritten < numRead) {
            ssize_t nw = write(fto, &buffer[numWritten], numRead - numWritten);
            if (nw < 0) {
                close(ffrom);
                close(fto);
            }
        }
    } while (numRead > 0);
}
```

```

        return nw;
    }
    numWritten += nw;
}
while (numRead!=0);

close(ffrom);
close(ftp);
return 0;
}

```

2. zadatak, treći kolokvijum, jun 2016.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog fajl sistema svaki čvor u hijerarhijskoj strukturi direktorijuma i fajlova predstavljen je objektom klase `Node`. Operacija te klase:

```
Node* Node::getSubnode(const char* pStart, const char* pEnd);
```

vraća podčvor datog čvora `this` koji ima simboličko ime zadato nizom znakova koji počinje znakom na koga ukazuje `pStart`, a završava znakom ispred znaka na koga ukazuje `pEnd` (`pEnd` može ukazivati na `'\0'` ili `'/'`). Ukoliko dati čvor `this` nije direktorijum, ili u njemu ne postoji podčvor sa datim simboličkim imenom, ova funkcija vraća 0.

Za svaki proces se u polju `curDir` strukture PCB čuva pokazivač na čvor (tipa `Node*`) koji predstavlja tekući direktorijum datog procesa. Koreni direktorijum cele hijerarhije dostupan je kao statički pokazivač `Node::rootNode` tipa `Node*`.

Znak za razdvajanje (delimiter) u stazama, kao i znak za koreni direktorijum je kosa crta `'/'`. Implementirati sledeću funkciju koja se koristi u implementaciji ovog fajl sistema:

```
Node* Node::getNode (PCB* pcb, const char* path);
```

Ova funkcija vraća čvor određen stazom koja je zadatka drugim argumentom, pri čemu ta staza može biti zadata kao apsolutna (počinje znakom `'/'`), ili kao relativna (ne počinje znakom `'/'`) u odnosu na tekući direktorijum procesa čiji je PCB dat kao prvi argument.

Rešenje

```

Node* Node::getNode (PCB* pcb, const char* path) {
    static const char delimiter = '/';
    if (pcb==0 || path==0) return 0; // Exception!
    Node* node = 0;
    const char* pStart = path;
    if (*pStart==delimiter) {
        node = rootNode;
        pStart++;
    } else {
        node = pcb->curDir;
    }
    while (node && *pStart) {
        const char* pEnd = pStart+1;
        while (*pEnd && *pEnd!=delimiter) pEnd++;
        node = node->getSubnode(pStart,pEnd);
        pStart = (*pEnd)?(pEnd+1):pEnd;
    };
    return node;
}

```

2. zadatak, treći kolokvijum, septembar 2016.

- [Postavka](#)
- [Rešenje](#)

Dole je dat izvod iz dokumentacije za API prema znakovnim tokovima (*character streams*) vezanim za fajlove u GNU sistemima. Sve date deklaracije su u `<stdio.h>`. Korišćenjem samo dole datih funkcija, realizovati sledeću funkciju koja prepisuje ceo sadržaj datog ulaznog fajla u dati izlazni fajl, ali u obrnutom redosledu (poretku) znakova. U slučaju bilo kakve greške, funkcija treba da vrati negativnu vrednost, u suprotnom treba da vrati 0.

```
int fcopyreverse (const char *filenamefrom, const char *filenameto);
```

```
FILE* fopen (const char *filename, const char *opentype);
```

Opening a file with the `fopen` function creates a new stream and establishes a connection between the stream and a file. This may involve creating a new file. The `fopen` function opens a stream for I/O to the file `filename`, and returns a pointer to the stream. If the open fails, `fopen` returns a null pointer. The `opentype` argument is a string that controls how the file is opened and specifies attributes of the resulting stream. It must begin with one of the following sequences of characters:

- `'r'` - Open an existing file for reading only.
- `'w'` - Open the file for writing only. If the file already exists, it is truncated to zero length. Otherwise a new file is created.
- `'a'` - Open a file for append access; that is, writing at the end of file only. If the file already exists, its initial contents are unchanged and output to the stream is appended to the end of the file. Otherwise, a new, empty file is created.
- `'r+'` - Open an existing file for both reading and writing. The initial contents of the file are unchanged and the initial file position is at the beginning of the file.
- `'w+'` - Open a file for both reading and writing. If the file already exists, it is truncated to zero length. Otherwise, a new file is created.
- `'a+'` - Open or create file for both reading and appending. If the file exists, its initial contents are unchanged. Otherwise, a new file is created. The initial file position for reading is at the beginning of the file, but output is always appended to the end of the file.

```
int fclose (FILE *stream);
```

This function causes `stream` to be closed and the connection to the corresponding file to be broken. Any buffered output is written and any buffered input is discarded. The `fclose` function returns a value of 0 if the file was closed successfully, and `EOF` if an error was detected. It is important to check for errors when you call `fclose` to close an output stream, because real, everyday errors can be detected at this time. For example, when `fclose` writes the remaining buffered output, it might get an error because the disk is full. Even if you know the buffer is empty, errors can still occur when closing a file if you are using NFS.

```
int fseek (FILE *stream, long int offset, int whence);
```

The `fseek` function is used to change the file position of the stream `stream`. The value of `whence` must be one of the constants `SEEK_SET`, `SEEK_CUR`, or `SEEK_END`, to indicate whether the `offset` is relative to the beginning of the file, the current file position, or the end of the file, respectively. This function returns a value of zero if the operation was successful, and a nonzero value to indicate failure. A successful call also clears the end-of-file indicator of stream and discards any characters that were "pushed back" by the use of `ungetc`. `fseek` either flushes any buffered output before setting the file position or else remembers it so it will be written later in its proper place in the file.

```
int fgetc (FILE *stream);
```

This function reads the next character as an `unsigned char` from the stream `stream` and returns its value, converted to an `int`. If an end-of-file condition or read error occurs, `EOF` is returned instead.

```
int fputc (int c, FILE *stream);
```

The `fputc` function converts the character `c` to type `unsigned char`, and writes it to the `stream` stream. `EOF` is returned if a write error occurs; otherwise the character `c` is returned.

Rešenje

```
#include <stdio.h>

int fcopyreverse (const char* filenamefrom, const char* filenameeto) {
    File* from = fopen(filenamefrom, "r");
    if (from==NULL) return -1; // Error with opening input file
    File* to = fopen(filenameeto, "w");
    if (to==NULL) return -2; // Error with opening output file
    int ret = 0;
    ret = fseek(from, -1, SEEK_END);
    while (ret==0) {
        int c = fgetc(from);
        if (c==EOF) return -3; // Error with reading input file
        ret = fputc(c, to);
        if (ret==EOF) return -4; // Error with writing to output file
        ret = fseek(from, -2, SEEK_CUR);
    }
    ret = fclose(from);
    if (ret==EOF) return -5; // Error with closing input file
    ret = fclose(to);
    if (ret==EOF) return -6; // Error with closing output file
    return ret;
}
```

2. zadatak, treći kolokvijum, jun 2015.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje deljene (*shared*) i ekskluzivne (*exclusive*) ključeve za pristup fajlu. Za operacije čitanja (*read*, *r*) i izvršavanja (*execute*, *x*) fajla potreban je deljeni ključ, a za operaciju upisa (*write*, *w*) ekskluzivni ključ.

Tokom operacije otvaranja fajla, operativni sistem proverava da li je operacija koju je zahtevao proces dozvoljena (u smislu ključa) i zaključava fajl odgovarajućim ključem ukoliko jeste dozvoljena. Ovo se obavlja u sledećoj funkciji `lock`:

```
int lock(FCB* file, unsigned int op);
const int OP_RD = 4;
const int OP_WR = 2;
const int OP_EX = 1;
```

Zahtevana operacija se identifikuje jednim od tri bita *rwX* u argumentu *op*, s tim što je uvek tačno jedan bit postavljen na 1. Za potrebe maskiranja tih bita definisane su konstante `OP_RD`, `OP_WR` i `OP_EX`. Funkcija treba da vrati 1 ako je operacija dozvoljena, a 0 ako nije.

Pre poziva ove funkcije, `FCB` traženog fajla je već učitao u memoriju i na njega ukazuje prvi argument. U toj strukturi, pored ostalog, postoje i celobrojna polja `sharedLock` i `exclLock` za deljeni i ekskluzivni ključ nad datim fajlom (1-zaključan, 0-otključan). Realizovati funkciju `lock`.

Rešenje

```
int lock(FCB* f, unsigned int op) {
    if (f==0) return -1; // Exception!
    if ((op & OP_WR) && (!f->sharedLock && !f->exclLock))
        return f->exclLock = 1;
    if ((op & (OP_RD|OP_EX)) && !f->exclLock)
        return f->sharedLock = 1;
    return 0;
}
```

2. zadatak, treći kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

U nekom fajl sistemu postoje, između ostalih, i sledeći sistemski pozivi za osnovne operacije sa fajlom:

```
int fgetsize (int fhandle, unsigned long& size);
int fresize (int fhandle, unsigned long newsize);
int fmoveto (int fhandle, unsigned long offset);
int fwrite (int fhandle, byte* buffer, unsigned long size);
```

Operacija `fgetsize` u izlazni argument `size` upisuje trenutnu veličinu sadržaja fajla, a operacija `fresize` menja veličinu sadržaja fajla na novu zadatu veličinu, koja može biti i veća i manja od trenutne; ukoliko je manja, sadržaj na kraju se odseca, a ukoliko je veća, sadržaj koji se dodaje je nedefinisan. Operacija `fmoveto` pomera kurzor na zadatu poziciju bajta (numeracija bajtova sadržaja fajla počinje od 0). Sve ove operacije u slučaju greške vraćaju negativnu vrednost sa kodom greške.

Realizovati operaciju koja proširuje sadržaj fajla datim sadržajem (dodaje ga na kraj):

```
int append (int fhandle, byte* buffer, unsigned long size);
```

Rešenje

```
int append (int fhandle, byte* buffer, unsigned long sz) {
    int ret = 0;
    unsigned long oldSize = 0;
    ret = fgetsize(fhandle,oldSize);
    if (ret<0) return ret;
    unsigned long newSize = oldSize + sz;
    ret = fresize(fhandle,newSize);
    if (ret<0) return ret;
    ret = fmoveto(fhandle,oldSize);
    if (ret<0) return ret;
    ret = fwrite(fhandle,buffer,sz);
    return ret;
}
```

2. zadatak, treći kolokvijum, jun 2014.

- [Postavka](#)
- [Rešenje](#)

U nekom fajl sistemu primenjuje se zaštita pristupa fajlovima kao u sistemu Unix. U strukturi FCB postoje sledeća polja:

- `unsigned long int owner`: identifikator korisnika koji je vlasnik fajla;
- `unsigned long int group`: identifikator grupe korisnika kojoj je fajl pridružen;
- `unsigned int protection`: biti prava pristupa (relevantno je samo 9 najnižih bita, pri čemu najviša 3 su dodeljena vlasniku, naredna 3 grupi, a najniža 3 ostalima).

U strukturi UCB (*user control block*) koja predstavlja jednog registrovanog korisnika sistema takođe postoji polje `group` koje predstavlja identifikator grupe kojoj je taj korisnik pridružen. Prava pristupa za grupu kojoj je pridružen fajl odnose se na korisnike koji su pridruženi istoj toj grupi.

Na raspolaganju je i sledeća funkcija koja vraća pokazivač na odgovarajuću strukturu UCB za korisnika koji je identifikovan datim identifikatorom:

```
UCB* getUCB (unsigned long int uid);
```

Realizovati funkciju koja ispituje da li je tražena operacija dozvoljena datom korisniku za dati fajl. Operacija se identifikuje jednim od tri najniža bita *rwX*, sa značenjem kao u bitima prava pristupa za fajl, s tim što je uvek samo jedan bit postavljen na 1. Funkcija treba da vrati 1 ako je operacija dozvoljena, a 0 ako nije.

```
int isAllowed(FCB* file, unsigned long int uid, unsigned int op);
```

Rešenje

```
int isAllowed(FCB* f, unsigned long int uid, unsigned int op) {
    if (f==0) return 0; // Exception!
    unsigned int prot = f->protection;
    if (uid==f->owner)
        prot >= 6;
    else {
        UCB* usr = getUCB(uid);
        if (usr==0) return 0; // Exception!
        if (usr->group==f->group) prot >= 3;
    } else
        prot &= 7;
    return (op&prot)?1:0;
}
```

2. zadatak, treći kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

U nekom fajl sistemu postoje sledeći sistemski pozivi za osnovne operacije sa fajlom:

```
int open (const char *pathname, int flags, mode_t mode);
int close (int fhandle);
int read (int fhandle, byte* buffer, unsigned long size);
int write (int fhandle, byte* buffer, unsigned long size);
```

Sve ove operacije u slučaju greške vraćaju negativnu vrednost sa kodom greške. U slučaju uspeha, operacija otvaranja fajla vraća „ručku“ fajla (engl. *file handle*), a ostale vraćaju 0. Realizovati objektno orijentisani „omotač“ ovog interfejsa, odnosno implementirati apstrakciju fajla kao klasu sa sledećim interfejsom:

```
class File {
public:
    File (const char *pathname, int flags, mode_t mode) throw Exception;
    ~File () throw Exception;

    void read (byte* buffer, unsigned long size) throw Exception;
    void write (byte* buffer, unsigned long size) throw Exception;
};
```

Prilikom kreiranja objekta ove klase treba implicitno otvoriti fajl, a prilikom uništavanja objekta treba implicitno zatvoriti fajl. U slučaju greške, sve ove operacije treba da podignu izuzetak definisanog tipa `Exception`. Instance ovog tipa (klase) mogu se inicijalizovati celobrojnim kodom greške koju vraćaju sistemski pozivi.

Rešenje

```
class File {
public:
    File (const char *pathname, int flags, mode_t mode) throw Exception;
    ~File () throw Exception;

    void read (byte* buffer, unsigned long size) throw Exception;
    void write (byte* buffer, unsigned long size) throw Exception;

private:
    int fh;
};
```

```

File::File (const char *p, int f, mode_t m) throw Exception : fh(0) {
    int s = open(p,f,m);
    if (s<0) throw Exception(s);
    fh=s;
}

File::~File () throw Exception {
    int s = close(fh);
    if (s<0) throw Exception(s);
}

void File::read (byte* b, unsigned long sz) throw Exception {
    int s = read(fh,b,sz);
    if (s<0) throw Exception(s);
}

void File::write (byte* b, unsigned long sz) throw Exception {
    int s = write(fh,b,sz);
    if (s<0) throw Exception(s);
}

```

2. zadatak, treći kolokvijum, jun 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom interfejsu fajl sistema definisano je nekoliko funkcija za rad sa direktorijumima. Dat je izvod iz dokumentacije jedne od njih:

```
struct dirent * readdir (DIR *dirstream);
```

This function reads the next entry from the directory `dirstream`... If there are no more entries in the directory or an error is detected, `readdir` returns a null pointer.

Prevod: Ova funkcija čita sledeći ulaz u direktorijumu `dirstream`... Ako više nema ulaza u direktorijumu ili je došlo do greške, `readdir` vraća *null* pokazivač.

Napomena: simbol `.` označava tekući direktorijum.

Šta radi sledeći program?

```

#include <stddef.h>
#include <stdio.h>
#include <sys/types.h>
#include <dirent.h>

int main (void)
{
    DIR *dp;
    struct dirent *ep;

    dp = opendir (".");
    if (dp != NULL)
    {
        while (ep = readdir (dp))
            puts (ep->d_name);
        closedir (dp);
    }
    else

```

```
puts ("Couldn't open the directory.");

return 0;
}
```

Rešenje

Na standardni izlaz ispisuje sadržaj tekućeg direktorijuma, tačnije, nazive ulaza u tekućem direktorijumu (po jedan u redu).

2. zadatak, treći kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

Dat je izvod iz originalne dokumentacije za Unix/Linux sistemski poziv za otvaranje fajla:

```
int open(const char *pathname, int flags, mode_t mode);
```

The argument *flags* must include one of the following access modes: **O_RDONLY**, **O_WRONLY**, or **O_RDWR**. These request opening the file read-only, write-only, or read/write, respectively. In addition, zero or more file creation flags and file status flags can be bitwise-or'd in *flags*. The file creation flags are **O_CLOEXEC**, **O_CREAT**, **O_DIRECTORY**, **O_EXCL**, **O_NOCTTY**, **O_NOFOLLOW**, **O_TRUNC**, and **O_TTY_INIT**...

- **O_CREAT** If the file does not exist it will be created. The owner (user ID) of the file is set to the effective user ID of the process. The group ownership (group ID) is set to the effective group ID of the process...

mode specifies the permissions to use in case a new file is created. This argument must be supplied when **O_CREAT** is specified in *flags*.

The following symbolic constants are provided for *mode*:

S_IRWXU	0x0700	user (file owner) has read, write and execute permission
S_IRUSR	0x0400	user has read permission
S_IWUSR	0x0200	user has write permission
S_IXUSR	0x0100	user has execute permission
S_IRWXG	0x0070	group has read, write and execute permission
S_IRGRP	0x0040	group has read permission
S_IWGRP	0x0020	group has write permission
S_IXGRP	0x0010	group has execute permission
S_IRWXO	0x0007	others have read, write and execute permission
S_IROTH	0x0004	others have read permission
S_IWOTH	0x0002	others have write permission
S_IXOTH	0x0001	others have execute permission

1. Napisati kod za sistemski poziv koji će kreirati fajl „test.txt“ u tekućem direktorijumu pozivajućeg procesa (označava se sa `.`) i otvara ga za čitanje i upis, tako da vlasnik tog fajla ima prava na čitanje i upis, članovi grupe prava samo na čitanje, a ostali nikakva prava.
2. Ukoliko takav identičan poziv izvrši neki drugi proces (nakon što je poziv iz a) uspešno izvršen) koji se izvršava u ime drugog korisnika koji nije član iste grupe kao korisnik u čije ime je izvršen prvi poziv, da li će taj drugi poziv uspeti? Obrazložiti.

Rešenje

1. `open("./test.txt", O_CREAT|O_RDWR, S_IRUSR|S_IWUSR|S_IRGRP);`
2. Neće. Taj korisnik (u čije ime se izvršava drugi proces) je pripadnik „ostalih“, pošto nije ni vlasnik fajla niti pripadnik iste grupe, pa nema nikakva prava nad fajlom, a traži otvaranje tog (sada postojećeg) fajla sa mogućnošću čitanja i upisa.

2. zadatak, treći kolokvijum, jun 2012.

- [Postavka](#)
- [Rešenje](#)

U nekom fajl sistemu u sistemskom pozivu za otvaranje fajla proces navodi da li će fajl samo čitati ili ga i na bilo koji način menjati. U zavisnosti od toga, taj sistemski poziv zaključava fajl sa jednim od dve vrste ključa. Ako se fajl otvara samo za čitanje, fajl se zaključava deljenim ključem; ako se fajl otvara za izmenu, zaključava se ekskluzivnim ključem. Ukoliko poziv ne može da se izvrši zbog toga što ključ ne može da se dobije, poziv se otkazuje bez izmena u fajl sistemu i vraća se greška.

Procesi A, B, C i D izvršavaju sistemske pozive otvaranja i zatvaranja istog fajla u sledećem redosledu (neki proces izvršava poziv zatvaranja fajla samo ako ga je uspešno otvorio):

1. A: `open(READ)`
2. B: `open(WRITE)`
3. C: `open(READ)`
4. A: `close`
5. C: `close`
6. D: `open(WRITE)`

Koje od ovih operacija će se izvršiti uspešno, a koje neuspešno? Precizno obrazložiti odgovor!

Rešenje

Neuspešna je operacija 2), ostale su uspešne.

2. zadatak, treći kolokvijum, jun 2011.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu struktura PCB sadrži polje `open_files` koje je pokazivač na tabelu otvorenih fajlova tog procesa. U toj tabeli (zapravo nizu), svaki ulaz je struktura koja predstavlja deskriptor jednog otvorenog fajla. U toj strukturi postoji celobrojno polje `access` čija vrednost 1 označava da je dati proces otvorio dati fajl sa pravom na upis, a vrednost 0 označava da je dati proces otvorio dati fajl samo sa pravom na čitanje. Promenljiva tipa `FHANDLE` predstavlja indeks u tabeli otvorenih fajlova datog procesa u čijem je kontekstu otvoren fajl. Implementirati operaciju `check_access()` čiji je potpis `dat`, a koju poziva fajl podsistem u svakom sistemskom pozivu za pristup sadržaju fajla radi provere prava datog procesa na izvršavanje date operacije nad sadržajem fajla. Argument `p` je pokazivač na PCB procesa koji je izdao sistemski poziv, argument `f` je identifikator fajla, a argument `write` ima vrednost 0 ako sistemski poziv zahteva samo čitanje, odnosno vrednost 1 ako sistemski poziv zahteva upis u dati fajl.

```
int check_access (PCB* p, FHANDLE f, int write);
```

Rešenje

```
int check_access (PCB* p, FHANDLE f, int write) {
    return p->open_files[f].access || !write;
}
```

2. zadatak, treći kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Dat je deo API za neki fajl sistem:

```
typedef int FHANDLE; // File handle
FHANDLE fopen (char* fname, int accessFlags);
int fclose (FHANDLE);
int fread (FHANDLE, unsigned int offset, unsigned int size, void* buffer);
```

```
int fwrite (FHANDLE, unsigned int offset, unsigned int size, void* buffer);
unsigned int fsize (FHANDLE); // Returns the file size
```

Sve funkcije osim `fsize` vraćaju pozitivnu vrednost u slučaju uspeha, a negativnu vrednost sa kodom greške u slučaju neuspeha. Koncept kurzora nije ugrađen u ovaj fajl sistem, već funkcije za čitanje i upis u fajl zadaju pomeraj (*offset*) od koga se čita/upisuje.

Korišćenjem ovog API realizovati klasu koja apstrahuje fajl sa ugrađenim kurzorom i sledećim interfejsom:

```
class File {
public:
    File (char* fname, int accessFlags);
    ~File ();
    int read (void* buffer, unsigned int size);
    int write (void* buffer, unsigned int size);
    int seek (unsigned int offset); // Move the cursor to the given offset
};
```

Rešenje

```
class File {
public:
    File (char* fname, int accessFlags);
    ~File ();
    int read (void* buffer, int size);
    int write (void* buffer, int size);
    int seek (unsigned int offset); // Move the cursor to the given offset
private:
    FHANDLE fh;
    unsigned int cursor;
};

File::File (char* fname, int af) : cursor(0) {
    fh=fopen(fname,af);
}

File::~File () { fclose(fh); }

int File::read (void* buffer, unsigned int size) {
    int ret = fread(fh,cursor,size,buffer);
    seek(cursor+size);
    return ret;
}

int File::write (void* buffer, unsigned int size) {
    int ret = fwrite(fh,cursor,size,buffer);
    seek(cursor+size);
    return ret;
}

int File::seek (unsigned int offset) {
    unsigned int size = fsize(fh);
    if (offset>=size) cursor=size;
    else cursor=offset;
    return 1;
}
```

Fajl sistem (implementacija)

3. zadatak, treći kolokvijum, jun 2024.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog fajl sistema keš struktura FCB svih otvorenih fajlova i korišćenih direktorijuma (“globalna tabela otvorenih fajlova”) organizuje se u memoriji kao objekat klase `FCBCache`. U tom kešu jedan FCB čuva se u jednom ulazu predstavljenom strukturom `FCBEntry` čiji je deo definicije dat dole. Polje `id` čuva identifikator FCB-a koji je učitao u ovaj ulaz (npr. broj bloka na disku), polje `refCnt` je brojač referenci, a polje `fc` sam sadržaj FCB-a.

```
struct FCBEntry {
    FCBID id; // ID of the FCB stored in this entry
    unsigned long refCnt = 0; // Reference counter
    FCB fcb; // FCB of the node
    ...
};
```

Kada neki deo kernela želi da koristi neki FCB, recimo kada proces otvara fajl, poziva se operacija `FCBCache::request(FCBID)` koja pronalazi ulaz u kome se već nalazi traženi FCB ili ga učitava ukoliko on nije u kešu (tabeli). Tom prilikom se `refCnt` tog ulaza inkrementira ili postavlja na 1 ukoliko je dati FCB tek učitao. Slično, kada taj korisnik više ne koristi taj FCB, poziva operaciju `FCBCache::release(FCBID)` koja “oslobađa” taj FCB, ali on ostaje u kešu ukoliko ponovo bude potreban, dok se eventualno odatle ne izbacuje. Keš funkcioniše tako što nekorišćeni ulazi (oni kod kojih je `refCnt == 0`) ostaju u kešu osim ukoliko se ne zahteva prostor za učitavanje novog FCB-a, a u kešu više nema slobodnih ulaza. Tada se kao “žrtva za izbacivanje” bira onaj nekorišćeni ulaz koji je najdavnije bio korišćen – LRU (*least recently used*) politikom zamene.

Implementirati sledeće pomoćne nestatičke funkcije članice klase `FCBCache` koje se pozivaju iz implementacije navedenih operacija `FCBCache::request` i `FCBCache::release`:

- `void FCBCache::updateLRUonRequest(FCBEntry* f)`: ova operacija poziva se iz operacije `FCBCache::request` koja je pronašla odgovarajući ulaz `f`; ona treba da po potrebi ažurira evidenciju potrebnu za LRU algoritam zamene; pre poziva ove operacije `f->refCnt` je već ažuriran (inkrementiran);
- `void FCBCache::updateLRUonRelease(FCBEntry* f)`: ova operacija poziva se iz operacije `FCBCache::release` koja oslobađa odgovarajući ulaz `f`; ona treba da po potrebi ažurira evidenciju potrebnu za LRU algoritam zamene; pre poziva ove operacije `f->refCnt` je već ažuriran (dekrementiran);
- `FCBEntry* FCBCache::getLRUVictim()`: ova operacija poziva se iz operacije `FCBCache::request` po potrebi, ukoliko nema slobodnog mesta za učitavanje traženog FCB-a; ona treba da vrati ulaz koji je po LRU algoritmu odabran kao žrtva za zamenu i po potrebi ažurira evidenciju potrebnu za LRU algoritam zamene; ukoliko takvog nema, treba da vrati `null`.

Navešti proširenja klase `FCBCache` i strukture `FCBEntry` potrebnim članovima.

Rešenje

U strukturu `FCBEntry` treba dodati sledeće članove za ulančavanje u dvostruko ulančanu LRU listu ulaza koji se ne koriste:

```
FCBEntry *lruPrev = 0, *lruNext = 0; // For the LRU double-linked list
```

U klasu `FCBCache` treba dodati sledeće članove koji predstavljaju glavu i rep LRU liste nekorišćenih ulaza; lista je uređena po hronologiji korišćenja:

```
FCBEntry *lruHead = 0, *lruTail = 0; // LRU double-linked list
```

Pomoćna nestatička funkcija članica koja izbacuje dati ulaz iz LRU liste (ako je on u listi):

```
inline void FCBCache::removeFromLRU(FCBEntry* f) {
    // If not in the LRU list, return:
    if (!f->lruPrev && !f->lruTail && this->lruHead != f) return;
    // Else, remove it from the LRU list:
    if (f->lruNext) f->lruNext->lruPrev = f->lruPrev;
```

```

    else this->lruTail = f->lruPrev;
    if (f->lruPrev) f->lruPrev->lruNext = f->lruNext;
    else this->lruHead = f->lruNext;
    f->lruNext = f->lruTail = 0;
}

inline void FCBCache::updateLRUonRequest(FCBEntry* f) {
    if (f && f->refCnt == 1) this->removeFromLRU(f);
}

inline void FCBCache::updateLRUonRelease(FCBEntry* f) {
    if (f && f->refCnt == 0) {
        // Insert it at the head of the LRU list
        f->lruNext = this->head;
        f->lruPrev = 0;
        if (this->lruHead) this->lruHead->lruPrev = f;
        else this->lruTail = f;
        this->lruHead = f;
    }
}

FCBEntry* getLRUVictim () {
    FCBEntry* f = this->lruTail; // Get it from the tail of the LRU list
    if (f) this->removeFromLRU(f);
    return f;
}

```

3. zadatak, treći kolokvijum, jun 2023.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog FAT fajl sistema ceo FAT keširan je u memoriji u nizu `fat`:

```

extern uint32 fat[];
extern uint32 freeHead, freeCount;

```

Za ulančavanje se kao *null* vrednost u ulazu u FAT koristi 0 (blok broj 0 je rezervisan). U FCB fajla polje `head` sadrži redni broj prvog bloka sa sadržajem fajla i polje `size` koje sadrži veličinu sadržaja fajla u bajtovima. Slobodni blokovi su ulančani u jednostruku listu čija glava je u promenljivoj `freeHead`, dok ukupan broj slobodnih blokova čuva promenljiva `freeCount`. Implementirati sledeću funkciju koja treba da obriše sadržaj datog fajla:

```
void truncate (FCB* fcb);
```

Rešenje

```

void truncate(FCB* fcb) {
    if (fcb == 0) return;
    if (!fcb->head) return; // File already empty
    uint32 last = fcb->head;
    uint32 cnt = 1;
    while (fat[last]) {
        last = fat[last];
        cnt++;
    }
    fat[last] = freeHead;
    freeHead = fcb->head;
    freeCount += cnt;
    fcb->head = 0;
    fcb->size = 0;
}

```

3. zadatak, treći kolokvijum, avgust 2023.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog fajl sistema svaki čvor u hijerarhijskoj strukturi direktorijuma i fajlova predstavljen je objektom klase `Node`. Operacija te klase:

```
Node* Node::getSubnode(const char* pStart, const char* pEnd);
```

vraća podčvor datog čvora `this` koji ima simboličko ime zadato nizom znakova koji počinje znakom na koga ukazuje `pStart`, a završava znakom ispred znaka na koga ukazuje `pEnd` (`pEnd` može ukazivati na `'\0'` ili `'/'`). Ukoliko dati čvor `this` nije direktorijum, ili u njemu ne postoji podčvor sa datim simboličkim imenom, ova funkcija vraća 0. Koreni direktorijum cele hijerarhije dostupan je kao statički pokazivač `Node::root` tipa `Node*`. Znak za razdvajanje (`delimiter`) u stazama, kao i znak za koreni direktorijum je kosa crta `'/'`.

Data je implementacija funkcije `getNodeRel` koja za (sintaksno ispravnu) relativnu putanju datu nizom znakova koji počinje znakom na koga ukazuje `pStart` i završava se znakom ispred znaka na koji ukazuje `pEnd` vraća čvor definisan tom putanjom u odnosu na dati čvor.

Korišćenjem ove funkcije, implementirati funkciju `Node::getNodeAbs` koja za apsolutnu putanju (garantovano je sintaksno ispravna i počinje znakom `'/'`) vraća čvor određen tom putanjom, uz korišćenje keša preslikavanja apsolutnih putanja u čvorove `DentryCache`. Data funkcija `DentryCache::getNode` vraća ranije zapamćen čvor za apsolutnu putanju datu nizom znakova zadatog sa `pStart` i `pEnd`. Ukoliko traženi čvor postoji, treba ažurirati keš unosom ulaza koji preslikava datu apsolutnu putanju u dati čvor pozivom funkcije `DentryCache::storeNode`.

```
static const char delimiter = '/';
```

```
Node* Node::getNodeRel(const char* pStart, const char* pEnd, Node* node) {
    while (node && pStart < pEnd) {
        const char* pE = pStart + 1;
        while (pE < pEnd && *pE != delimiter) pE++;
        node = node->getSubnode(pStart, pE);
        pStart = (pE < pEnd) ? (pE + 1) : pE;
    };
    return node;
}

Node* DentryCache::getNode(const char* pStart, const char* pEnd);
void DentryCache::storeNode(const char* pStart, const char* pEnd, Node*);
Node* Node::getNodeAbs(const char* pStart, const char* pEnd);
```

Rešenje

```
Node* Node::getNodeAbs(const char* pStart, const char* pEnd) {
    const char *pE = pEnd;
    while (pE > pStart) {
        Node* node = DentryCache::getNode(pStart, pE);
        if (node) {
            if (pE < pEnd) node = Node::getNodeRel(pE + 1, pEnd, node);
            if (node) DentryCache::store(pStart, pEnd, node);
            return node;
        }
        do { pE--; } while (pE > pStart && *pE != delimiter);
    }
    Node* node = Node::getNodeRel(pStart + 1, pEnd, Node::root);
    if (node) DentryCache::store(pStart, pEnd, node);
    return node;
}
```

3. zadatak, treći kolokvijum, jun 2022.

- [Postavka](#)
- [Rešenje](#)

Sistemske pozivi za pristup sadržaju binarnog fajla uobičajeno omogućavaju čitanje ili upis size susednih bajtova počev od pozicije *offset* u odnosu na početak sadržaja fajla (prvi bajt je na poziciji 0). Da bi ostvario ovakav pristup, neki kernel koristi klasu `FLogicalAccess` čiji je interfejs dat dole. Ova klasa koristi se tako što se za svaki ovakav pristup instancira objekat ove klase, inicijalizuje se pozivom operacije `reset` sa zadatim parametrima, a onda se iterira sve dok operacija `end` ne vrati 1. U svakoj iteraciji se pristupa odgovarajućim bajtovima jednog logičkog bloka fajla, tako što se može dobiti sledeća informacija:

- `getBlock`: vraća redni broj logičkog bloka u kom se pristupa u toj iteraciji;
- `getRelOffset`: vraća redni broj bajta u tekućem bloku počev od kog se pristupa;
- `getRelSize`: broj bajtova u tekućem bloku kojima se pristupa u toj iteraciji.

```
class FLogicalAccess {
public:
    FLogicalAccess ();
    void reset (size_t offset, size_t size);
    int end() const;
    void next();
    size_t getBlock() const;
    size_t getRelOffset() const;
    size_t getRelSize() const;
};
```

Primer upotrebe ove klase je sledeći:

```
FLogicalAccess fla;
for (fla.reset(offset,size); !fla.end(); fla.next()) {
    // Access fla.getRelSize() bytes
    // starting from the offset fla.getRelOffset()
    // in the logical block fla.getBlock()
}
```

Na primer, ako je veličina bloka 8 bajtova, a inicijalno je zadato: *offset* = 11 i *size* = 15, onda će u prvoj iteraciji biti: `getBlock()` = 1, `getRelOffset()` = 3, `getRelSize()` = 5, u drugoj će ove vrednosti redom biti 2, 0, 8, a u trećoj 3, 0, 2; posle toga će se izaći iz petlje zbog `end()` = 1.

Implementirati u potpunosti klasu `FLogicalAccess`. Veličina bloka je `BLK_SIZE`.

Rešenje

```
class FLogicalAccess {
public:
    FLogicalAccess() {
        reset(0, 0);
    }
    void reset(size_t offset, size_t size);
    int end() const {
        return isEnd;
    }
    void next();
    size_t getBlock() const {
        return blk;
    }
    size_t getRelOffset() const {
        return relOfs;
    }
    size_t getRelSize() const {
```

```

        return relSz;
    }
private:
    size_t size;
    size_t blk, relOfs, relSz;
    int isEnd;
};
inline void FLogicalAccess::reset(size_t offset, size_t sz) {
    blk = offset / BLK_SIZE;
    relOfs = offset % BLK_SIZE;
    relSz = (sz > BLK_SIZE - relOfs) ? (BLK_SIZE - relOfs) : sz;
    isEnd = (sz == 0);
    size -= relSz;
}
inline void FLogicalAccess::next() {
    if (size > 0) {
        blk++;
        relOfs = 0;
        relSz = (size > BLK_SIZE) ? BLK_SIZE : size;
        isEnd = 0;
        size -= relSz;
    } else {
        reset(0, 0);
    }
}
}

```

4. zadatak, kolokvijum, jul 2021.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje kombinovanu tehniku indeksirane alokacije sadržaja fajla. U FCB fajla polje `singleIndex` predstavlja direktni indeks kao niz od `SingleIndexSize` elemenata, pri čemu svaki element sadrži broj fizičkog bloka sa sadržajem fajla (ili 0, ako je neiskorišćen). Ako veličina sadržaja fajla preraste veličinu podržanu ovim indeksom, naredni blokovi sadržaja fajla (preko ove veličine) indeksiraju se indeksom u dva nivoa. Za te potrebe, polje `dblIndex` u FCB sadrži broj bloka na disku u kom je indeks drugog nivoa. Taj blok sa indeksom drugog nivoa sadrži `DbIndexSize` ulaza sa brojevima blokova sa sadržajem fajla, odnosno 0 ako ulaz nije iskorišćen. Polje `size` u FCB sadrži veličinu fajla u bajtovima. Veličina bloka u bajtovima je `BLOCK_SIZE`. Svi ulazi u indeksu prvog nivoa i indeksu drugog nivoa (ako postoji) su sigurno postavljeni na validne vrednosti (broj bloka sa sadržajem ili 0). Realizovati funkciju `extendFile` koja proširuje sadržaj fajla čiji je FCB dat za jedan blok:

```
int extendFile(FCB* fcb);
```

Ova funkcija treba da vrati 0 u slučaju uspeha, a negativnu vrednost u slučaju greške. Ona ne treba da menja polje `size` u FCB (to će raditi pozivalac). Na raspolaganju je funkcija koja vraća pokazivač na sadržaj bloka na disku sa datim brojem, učitanoj u keš:

```
void* getBlock(PBlock blkNo);
```

kao i funkcija koja zauzima jedan slobodan blok i vraća njegov broj ili 0 ako takvog nema:

```
PBlock allocateBlock();
```

Rešenje

```

int extendFile(FCB* fcb) {
    size_t entry = (fcb->size + BLOCK_SIZE - 1)/BLOCK_SIZE;
    if (entry < SingleIndexSize) {
        PBlock newBlock = allocateBlock();
        if (!newBlock) return -1; // Error: no free space for the new block
    }
}

```

```

    fcb->singleIndex[entry] = newBlock;
    return 0;
}
entry -= SingleIndexSize;
if (entry >= DbIndexSize) return -2; // Error: file size limit exceeded
if (!fcb->dblIndex) {
    fcb->dblIndex = allocateBlock();
    if (!fcb->dblIndex) return -3; // Error: no space for index block
}
PBlock newBlock = allocateBlock();
if (!newBlock) return -1; // Error: no free space for the new block
PBlock* dblIx = (PBlock*) getBlock(fcb->dblIndex);
dblIx[entry] = newBlock;
return 0;
}

```

4. zadatak, kolokvijum, avgust 2021.

- [Postavka](#)
- [Rešenje](#)

U nekom FAT fajl sistemu FCB-ovi su smešteni na jednom delu particije, a sadržaj fajlova na drugom, disjunktном delu particije. Za taj fajl sistem pravi se sistemski program koji radi kompakciju prostora na disku tako što premešta blokove sa sadržajem fajla (ne i one sa FCB-om) tako da budu susedni na particiji. FAT tabela je veličine `FAT_SIZE` i njeni ulazi predstavljaju sve blokove na particiji. Ulaz FAT-a sadrži broj sledećeg bloka u ulančanoj listi blokova sa sadržajem fajla, odnosno u ulančanoj listi slobodnih blokova čija je glava u globalnoj promenljivoj `freeBlkHead`; nula u ulazu označava kraj liste blokova sa sadržajem, a negativna vrednost u ulazu označava slobodan blok (sledeći u lancu slobodnih je dat suprotnom vrednošću te negativne, -1 označava kraj liste). FCB svakog fajla smešten je u jedan blok na prvom delu particije, a ulaz FAT-a koji odgovara tom bloku sadrži broj prvog bloka sa sadržajem tog fajla. FAT je cela keširana u memoriji.

Implementirati funkciju `compact` koja za dati fajl radi kompakciju premeštanjem blokova sa sadržajem tog fajla tako da budu smešteni redom počev od bloka sa brojem `startBlk`. Parametar `fcblk` određuje broj bloka u kom je FCB datog fajla. Prostor počev od bloka broj `startBlk` nije oslobođen (u njemu se možda nalaze blokovi sa sadržajem fajlova), ali do kraja particije svakako ima dovoljno blokova za smeštanje sadržaja datog fajla. Na raspolaganju su operacije za čitanje i upis bloka sa datim brojem za dati bafer u memoriji veličine `BLK_SIZE` znakova. Ignorirati greške.

```

int FAT[FAT_SIZE];
void readBlock (int blk, char* buffer);
void writeBlock(int blk, const char* buffer);
void compactFile (int fcblk, int startBlk);

```

Rešenje

```

inline void swap (int* x, int* y) { int temp = *x; *x = *y; *y = temp; }

void moveBlk (int blk) { // Moves the given block to the first free block
    char buffer[BLK_SIZE];
    readBlock(blk,buffer);
    writeBlock(freeBlkHead,buffer);
    swap(&FAT[freeBlkHead],&FAT[blk]);
    for (int i=0; i<FAT_SIZE; i++)
        if (FAT[i]==blk) { FAT[i]=freeBlkHead; break; }
    freeBlkHead = blk;
}

void compactFile (int fcblk, int startBlk) {

```



```

for (int b = FAT[fcbBlk]; b!=0; b=FAT[b]) {
    if (FAT[startBlk]>=0) moveBlk(startBlk);
    moveBlk(b);
    startBlk++;
}
}

```

4. zadatak, kolokvijum, oktobar 2020.

- [Postavka](#)
- [Rešenje](#)

Da bi smanjio internu fragmentaciju, a povećao efikasnost, neki fajl sistem za alokaciju sadržaja fajla koristi klasterne (*cluster*) različite veličine. Klaster uvek sadrži susedne blokove na disku (blokove sa susednim brojevima). Za sadržaj fajla pre njegovog kraja koriste se klasteri veličine `CLUSTER_SIZE` blokova, gde je `CLUSTER_SIZE` predefinisana konstanta, mali prirodan broj veći od 1. Za sam kraj sadržaja fajla koristi se samo onoliko susednih blokova na disku koliko je potrebno, odnosno klaster veličine manje od ili jednake `CLUSTER_SIZE` blokova.

Sistem primenjuje indeksiranu alokaciju sadržaja fajla, pri čemu je indeks u samom FCB. Polje `index` strukture FCB predstavlja indeks kao niz, pri čemu svaki element *i* sadrži broj prvog fizičkog bloka (početak klastera) sa sadržajem fajla (ili 0, ako je neiskorišćen) za klaster sa rednim brojem *i* (počev od 0). Polje `size` u FCB sadrži veličinu sadržaja fajla u bajtovima. Veličina bloka na disku u bajtovima je `BLOCK_SIZE`. FCB je uvek učitao u memoriju za otvoren fajl.

Realizovati funkciju `getPBlock`:

```
size_t getPBlock (FCB* fcb, size_t bt);
```

Ova funkcija treba da vrati broj fizičkog bloka za bajt sa zadatim logičkim rednim brojem `bt` unutar sadržaja fajla (0 u slučaju da `bt` prekoračuje veličinu sadržaja fajla).

Rešenje

```

size_t getPBlock (FCB* fcb, size_t bt) {
    if (bt>=fcb->size) return 0;
    size_t lBlk = bt/BLOCK_SIZE;
    size_t cluster = lBlk/CLUSTER_SIZE;
    size_t pBlk = fcb->index[cluster] + lBlk%CLUSTER_SIZE;
    return pBlk;
}

```

4. zadatak, kolokvijum, jun 2020.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog fajl sistema evidencija slobodnih blokova na particiji vodi se pomoću bit-vektora koji se kešira u memoriji u nizu `blocks` veličine `NumOfBytes` (konstanta `NumOfBlocks` predstavlja broj blokova na particiji). Svaki element ovog niza je veličine jednog bajta, a svaki bit odgovara jednom bloku na disku (1-zauzet, 0-slobodan). Pretpostaviti da je `NumOfBlocks` umnožak broja 8 (`BITS_IN_BYTE`). Date su pomoćne funkcije i funkcija `allocateBlock` koja vrši alokaciju slobodnog bloka tražeći prvi slobodan blok počev od onog datog argumentom. Blok 0 je uvek zauzet.

```

typedef unsigned char byte;
typedef unsigned long ulong;
const byte BITS_IN_BYTE = 8;
const ulong NumOfBlocks = ...;
const ulong NumOfBytes = NumOfBlocks/BITS_IN_BYTE;
byte blocks[NumOfBytes];

void blockToBit(ulong blkNo, ulong& bt, byte& mask) {

```

```

    bt = blkNo/BITS_IN_BYTE;
    mask = 1<<(blkNo%BITS_IN_BYTE);
}

void bitToBlk(ulong& blkNo, ulong bt, byte mask) {
    blkNo = bt*BITS_IN_BYTE;
    for (; !(mask&1); mask>>=1) blkNo++;
}

ulong allocateBlock (ulong startingFrom) {
    ulong bt = 0; byte msk = 0;
    for (ulong blk = startingFrom; blk<NumOfBlocks; blk++) {
        blockToBit(blk, bt, msk);
        if ((blocks[bt]&msk)==0) {
            blocks[bt] |= msk;
            return blk;
        }
    }
    for (ulong blk = 1; blk<startingFrom; blk++) {
        blockToBit(blk, bt, msk);
        if ((blocks[bt]&msk)==0) {
            blocks[bt] |= msk;
            return blk;
        }
    }
    return 0;
}

```

Modifikovati funkciju allocateBlock tako da alocira blok koji je najbliži onom koji je tad argumentom, i to najbliži sa bilo koje strane.

Rešenje

```

#define max(a,b) ((a)>=(b)?(a):(b))

ulong allocateBlock (ulong startingFrom) {
    if (startingFrom>=NumOfBlocks) return 0;
    ulong bt = 0, blk = 0; byte msk = 0;
    ulong limit = max(NumOfBlocks-startingFrom, startingFrom);
    for (ulong i=0; i<limit; i++) {
        if (i<NumOfBlocks-startingFrom) {
            blk = startingFrom+i;
            blockToBit(blk, bt, msk);
            if ((blocks[bt]&msk)==0) {
                blocks[bt] |= msk;
                return blk;
            }
        }
        if (i<startingFrom && i>0) {
            blk = startingFrom-i;
            blockToBit(blk, bt, msk);
            if ((blocks[bt]&msk)==0) {
                blocks[bt] |= msk;
                return blk;
            }
        }
    }
}

```

```

    return 0;
}

```

4. zadatak, kolokvijum, septembar 2020.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje kombinovanu tehniku indeksirane alokacije sadržaja fajla. U FCB fajla polje `singleIndex` predstavlja direktni indeks kao niz od `SingleIndexSize` elemenata, pri čemu svaki element sadrži broj fizičkog bloka sa sadržajem fajla (ili 0, ako je neiskorišćen). Ako veličina sadržaja fajla preraste veličinu podržanu ovim indeksom, naredni blokovi sadržaja fajla (preko ove veličine) indeksiraju se indeksom u dva nivoa. Za te potrebe, polje `dblIndex` u FCB sadrži broj bloka na disku u kom je indeks drugog nivoa. Taj blok sa indeksom drugog nivoa sadrži `DblIndexSize` ulaza sa brojevima blokova sa sadržajem fajla, odnosno 0 ako ulaz nije iskorišćen. Polje `size` u FCB sadrži veličinu fajla u bajtovima. Veličina bloka je `BlockSize`. Svi ulazi u indeksu prvog nivoa i indeksu drugog nivoa (ako postoji) su sigurno postavljeni na validne vrednosti (broj bloka sa sadržajem ili 0).

Realizovati funkciju `truncateFile` koja treba da obriše sadržaj fajla čiji je FCB dat:

```
void truncateFile (FCB* fcb);
```

Na raspolaganju je funkcija koja vraća pokazivač na sadržaj bloka na disku sa datim brojem, učitano u keš:

```
void* getBlock (PBlock blkNo);
```

kao i funkcija koja blok sa datim brojem označava slobodnim:

```
void freeBlock (PBlock blkNo);
```

Rešenje

```

void truncateFile(FCB* fcb) {
    for (size_t i = 0; i < SingleIndexSize; i++)
        if (fcb->singleIndex[i]) {
            freeBlock(fcb->singleIndex[i]);
            fcb->singleIndex[i] = 0;
        }
    if (fcb->dblIndex) {
        PBlock* dblIx = (PBlock*)getBlock(fcb->dblIndex);
        for (size_t i = 0; i < DblIndexSize; i++)
            if (dblIx[i]) freeBlock(dblIx[i]);
        freeBlock(fcb->dblIndex);
        fcb->dblIndex = 0;
    }
    fcb->size = 0;
}

```

3. zadatak, treći kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

Za fajl opisan u zadatku 2. poziva se opisani potprogram za argument $n=12$ i vrednost x koja ne postoji u stablu, a koja je veća od vrednosti u svim čvorovima stabla. Veličina bloka na disku je 512B, a veličina jednog podatka tipa `int` je 32 bita. Fajl se alokira indeksiranim metodom, s tim da je indeks prvog nivoa u FCB koji se smatra učitanim u memoriju (učitava se prilikom otvaranja fajla).

1. Ako je indeks u samo jednom nivou, koliko blokova sa sadržajem ukupno dohvata ovaj program?
2. Ako je indeks u dva nivoa, s tim da je indeks prvog nivoa u FCB i učitani je u memoriju, a jedan indeks drugog nivoa zauzima jedan blok na disku i ima ulaze veličine 64 bita, koliko ukupno blokova dohvata (učitava) ovaj program, računajući i pristupe indeksima drugog nivoa?

Izvesti rezultat i obrazložiti postupak.

Rešenje

Za dati slučaj, program pristupa elementima niza sa sledećim indeksima ($ni := 2ni + 2$):

0, 2, 6, 14, 30, 62, 126, 254, 510, 1022, 2046, 4094.

Svaki element niza zauzima $4=2^2$ bajta, a blok je veličine $512B=2^9B$, pa jedan blok sadrži $2^7=128$ elemenata niza. Zato prvih 7 adresiranih elemenata niza pripada istom bloku sa sadržajem fajla (bloku broj 0), dok su svi ostali elementi u različitim blokovima. Navedeni elementi pripadaju redom sledećim logičkim blokovima sadržaja fajla:

0, 0, 0, 0, 0, 0, 0, 1, 3, 7, 15, 31.

1. Za slučaj indeksa u jednom nivou, dohvata se ukupno $1+5=6$ blokova sa sadržajem (indeksni blok je već učitao).
2. Za slučaj indeksa u dva nivoa, jedan ulaz u indeksu prvog nivoa, odnosno jedan indeksni blok drugog nivoa, pokriva $2^6=64$ blokova sa sadržajem, pa su svi adresirani elementi pokriveni jednim indeksnim blokom drugog nivoa. Zato se ukupno dohvata jedan indeksni blok drugog nivoa i 6 blokova sa sadržajem, ukupno 7.

3. zadatak, treći kolokvijum, jun 2018.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem koristi indeksiranu alokaciju sadržaja fajla sa jednostepenim indeksom u jednom bloku. Struktura FCB keširana je u memoriji, kao i indeksni blok. U strukturi FCB, pored ostalih, postoje sledeća polja:

- `unsigned long size`: veličina sadržaja fajla u bajtovima;
- `unsigned long* index`: pokazivač na (keširan) indeks (ulazi su tipa `unsigned long`).

Osim toga, definisane su i sledeće konstante i funkcija:

- `unsigned long BlockSize`: veličina bloka na disku u bajtovima;
- `unsigned long MaxFileSize`: maksimalna dozvoljena veličina sadržaja fajla u bajtovima; jednaka je maksimalnom broju ulaza u indeksu pomnoženom veličinom bloka `BlockSize`;
- `unsigned long allocateBlock()`: alokira jedan slobodan blok na disku i vraća njegov broj; ukoliko slobodnog bloka nema, vraća 0.

Realizovati funkciju

```
unsigned long extend (FCB* fcb, unsigned extension);
```

koja proširuje sadržaj fajla na čiji FCB ukazuje prvi argument za broj bajtova dat drugim argumentom. Ukoliko traženo proširenje premašuje maksimalnu veličinu fajla ili na disku nema dovoljno slobodnog prostora, sadržaj fajla treba proširiti koliko je moguće. Ova funkcija vraća broj bajtova sa koliko je stvarno uspela da proširi sadržaj (jednako ili manje od traženog).

Rešenje

```
unsigned long extend (FCB* fcb, unsigned extension) {
    if (fcb==0) return 0;

    unsigned long oldSize = fcb->size;
    unsigned oldNumOfBlocks = oldSize/BlockSize + oldSize%BlockSize?1:0;

    unsigned long newSize = oldSize + extension;
    if (newSize>MaxFileSize) newSize = MaxFileSize;
    unsigned newNumOfBlocks = newSize/BlockSize + newSize%BlockSize?1:0;

    unsigned i;
    for (i=oldNumOfBlocks; i<newNumOfBlocks; i++) {
        unsigned long block = allocateBlock();
```

```

    if (block==0) break;
    fcb->index[i] = block;
}

if (i<newNumOfBlocks)
    newSize = i*BlockSize;

fcb->size = newSize;
return newSize-oldSize;

```

3. zadatak, treći kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog FAT fajl sistema cela FAT keširana je u memoriji u strukturi:

```
extern unsigned long fat[];
```

Za ulančavanje se kao *null* vrednost u ulazu u FAT koristi 0 (blok broj 0 je rezervisan). U FCB fajla polje **head** sadrži redni broj prvog bloka sa sadržajem fajla i polje **size** koje sadrži veličinu sadržaja fajla u bajtovima. Slobodni blokovi su označeni ulazima u FAT sa vrednošću -1. Implementirati sledeću funkciju koja treba da obriše sadržaj fajla:

```
void truncate (FCB* fcb);
```

Rešenje

```

void truncate (FCB* fcb) {
    if (fcb==0) return;
    unsigned long cur=fcb->head, next;
    while (cur) {
        next = fat[cur];
        fat[cur] = -1;
        cur = next;
    }
    fcb->head = 0;
    fcb->size = 0;
}

```

3. zadatak, treći kolokvijum, jun 2016.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog FAT fajl sistema cela FAT keširana je u memoriji u strukturi:

```
extern unsigned long fat[];
unsigned long freeHead;
```

Za ulančavanje se kao *null* vrednost u ulazu u FAT koristi 0 (blok broj 0 je rezervisan). U FCB fajla polje **head** sadrži redni broj prvog bloka sa sadržajem fajla. Slobodni blokovi su takođe ulančani u FAT, a broj prvog slobodnog bloka u lancu sadrži globalna promenljiva **freeHead**.

Implementirati sledeću funkciju:

```
unsigned long append (FCB* fcb);
```

koja treba da proširi sadržaj fajla sa datim FCB za jedan blok (da doda jedan blok na kraj). Funkcija vraća 0 ako slobodnih blokova nema ili u slučaju bilo koje druge greške, odnosno broj alociranog bloka ako je operacija uspela. Ne treba vršiti nikakve optimizacije u smislu alokacije najbližeg bloka već alociranim blokovima datog fajla.

Rešenje

```

unsigned long append (FCB* fcb) {
    if (fcb==0 || freeHead==0) return 0;
    unsigned long last=0, next=fcb->head, ret=freeHead;
    while (next) last=next, next=fat[next];
    if (last)
        fat[last] = freeHead;
    else
        fcb->head = freeHead;
    unsigned long oldHead = freeHead;
    freeHead = fat[freeHead];
    fat[oldHead] = 0;
    return ret;
}

```

3. zadatak, treći kolokvijum, septembar 2016.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje indeksiranu alokaciju fajlova, sa indeksima u dva nivoa. U FCB fajla nalazi se polje `index` tipa `unsigned long`, koje predstavlja broj bloka na disku u kome se nalazi indeks prvog nivoa. Indeksi oba nivoa su iste veličine `IndexSize` ulaza tipa `unsigned long`. Nepopunjeni ulazi u indeksima imaju vrednost 0. Operacija `getPBlock` po potrebi u keš učitava blok sa diska sa zadatim brojem i vraća pokazivač na mesto u kešu u koje je taj blok učitao; u slučaju greške vraća 0.

```

typedef unsigned long ulong;
extern const ulong IndexSize;
void* getPBlock(ulong pBlockNo);

```

Realizovati funkciju `getFileBlock()` datu dole, koja se koristi u implementaciji fajl sistema i koja treba da vrati adresu na učitao logički blok fajla sa zadatim (logičkim) brojem; u slučaju greške treba da vrati 0.

```

void* getFileBlock (FCB* fcb, ulong lBlockNo);

```

Rešenje

```

void* getFileBlock (FCB* fcb, ulong lBlockNo) {
    if (fcb==0) return 0; // Exception
    ulong entry0 = lBlockNo/IndexSize;
    if (entry0>=IndexSize) return 0; // Exception
    ulong entry1 = lBlockNo%IndexSize;
    ulong index = fcb->index;
    if (index==0) return 0; // Exception
    ulong* index0 = (ulong*)getPBlock(index);
    if (index0==0) return 0; // Exception
    index = index0[entry0];
    if (index==0) return 0; // Exception
    ulong* index1 = (ulong*)getPBlock(index);
    if (index1==0) return 0; // Exception
    return getBlock(index1[entry1]);
}

```

3. zadatak, treći kolokvijum, jun 2015.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog fajl sistema evidencija slobodnih blokova na disku vodi se pomoću bit-vektora koji se kešira u memoriji u nizu `blocks` veličine `NumOfBlocks/BITS_IN_BYTE` (konstanta `NumOfBlocks` predstavlja broj blokova na disku). Svaki element ovog niza je veličine jednog bajta, a svaki bit odgovara jednom bloku na disku (1-zauzet, 0-slobodan). Pretpostaviti da je `NumOfBlocks` umnožak broja 8 (`BITS_IN_BYTE`).

```
typedef unsigned char byte;
const unsigned int BITS_IN_BYTE = 8;
const unsigned long NumOfBlocks = ...;
byte blocks[NumOfBlocks/BITS_IN_BYTE];
```

1. Implementirati sledeće dve funkcije:

```
void blockToBit(unsigned long blkNo, unsigned long& bt, byte& mask);
void bitToBlk(unsigned long& blkNo, unsigned long bt, byte mask);
```

Funkcija `blockToBit` prima kao ulazni parametar redni broj bloka `blkNo` i na osnovu njega izračunava i upisuje u izlazne parametre broj bajta (`bt`) u bit-vektoru i jednu jedinu jedinicu u onaj razred parametra `msk` koji odgovara bitu u tom bajtu za dati blok. Funkcija `bitToBlock` radi obrnutu konverziju: za ulazni parametar koji je redni broj bajta u bit-vektoru (`bt`) i najniži razred u parametru `msk` koji je postavljen na 1, izračunava i upisuje u izlazni parametar `blk` redni broj bloka koji odgovara tom bajtu i bitu.

2. Korišćenjem funkcija pod a), implementirati funkcije `allocateBlock` i `freeBlock`. Funkcija `allocateBlock` treba da pronade prvi slobodan blok i označi ga zauzetim. Ako takav blok nađe, vraća broj tog bloka, a ako slobodnog bloka nema, vraća 0 (blok broj 0 je rezervisan i nikada se ne koristi u fajl sistemu). Kako bi se optimizovao pristup fajlovima, slobodan blok se traži u blizini bloka sa datim rednim brojem `startingFrom`, na sledeći način:
 - slobodan blok treba tražiti počev od datog bloka naviše (i alocirati prvi slobodan na koji se naide), pri tom može (ali ne mora) da se alocira i bilo koji slobodan blok koji je u istom bajtu bit-vektora kao i dati blok, bez obzira da li je ispred ili iza datog bloka (i on se smatra dovoljno bliskim).
 - ukoliko se na ovaj način ne pronade, počinje se pretraga od prvog bloka (blok broj 0 je rezervisan) pa sve do zadatog bloka `startingFrom`.

Funkcija `freeBlock` označava dati blok slobodnim.

```
unsigned long allocateBlock (unsigned long startingFrom);
void freeBlock (unsigned long blk);
```

Rešenje

- 1.

```
void blockToBit(unsigned long blkNo, unsigned long& bt, byte& mask) {
    bt = blkNo/BITS_IN_BYTE;
    mask = 1<<(blkNo%BITS_IN_BYTE);
}
```

```
void bitToBlk(unsigned long& blkNo, unsigned long bt, byte mask) {
    blkNo = bt*BITS_IN_BYTE;
    for (; !(mask&1); mask>>=1) blkNo++;
}
```

- 2.

```
void freeBlock (unsigned long blk) {
    if (blk>=NumOfBlocks) return;
    unsigned long bt = 0; byte msk = 0;
    blockToBit(blk, bt, msk);
    blocks[bt] &= ~msk;
}

unsigned long allocateBlock (unsigned long startingFrom) {
    unsigned long bt = 0; byte msk = 0;
```

```

for (unsigned long blk = startingFrom; blk<NumOfBlocks; blk++) {
    blockToBit(blk,bt,msk);
    if ((blocks[bt]&msk)==0) {
        blocks[bt] |= msk;
        return blk;
    }
}

for (unsigned long blk =1; blk<startingFrom; blk++) {
    blockToBit(blk,bt,msk);
    if ((blocks[bt]&msk)==0) {
        blocks[bt] |= msk;
        return blk;
    }
}
return 0;
}

```

Nešto efikasnija implementacija:

```

unsigned long allocateBlock (unsigned long startingFrom) {
    if (startingFrom>=NumOfBlocks) return 0;
    unsigned long bt = 0; byte msk = 0;
    blockToBit(startingFrom,bt,msk);
    for (; bt<NumOfBlocks/BITS_IN_BYTE; bt++) {
        if (blocks[bt]==~0) continue; // All blocks in this byte are occupied
        unsigned long blk = 0;
        bitToBlock(blk,bt,~blocks[bt]);
        blockToBit(blk,bt,msk)
        blocks[bt] |= msk;
        return blk;
    }

    blockToBit(1,bt,msk); // blk 0 is reserved
    for (; bt<NumOfBlocks/BITS_IN_BYTE; bt++) {
        if (blocks[bt]==~0) continue; // All blocks in this byte are occupied
        unsigned long blk = 0;
        bitToBlock(blk,bt,~blocks[bt]);
        blockToBit(blk,bt,msk)
        blocks[bt] |= msk;
        return blk;
    }

    return 0;
}

```

3. zadatak, treći kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje FAT za alokaciju sadržaja fajla. FAT je cela keširana u memoriji, na nju ukazuje pokazivač *fat*, i ima FATSIZe ulaza tipa *unsigned*. Prilikom ulančavanja blokova sa sadržajem fajla, *null* vrednost se označava vrednošću 0 u odgovarajućem ulazu u FAT, dok se slobodni blokovi ne ulančavaju posebno, već su njima odgovarajući ulazi u FAT označeni vrednostima *~0U* (sve jedinice binarno); blokovi broj 0 i broj *~0U* se ne koriste u fajl sistemu. U FCB polje *head* tipa *unsigned* sadrži broj prvog bloka sa sadržajem fajla (0 ako je sadržaj prazan).

Realizovati funkciju `extendFile()` datu dole, koja se koristi u implementaciji fajl sistema i koja treba da proširi sadržaj fajla za *by* blokova (da ih alokira i doda na kraj sadržaja fajla). Ova funkcija treba da vrati broj blokova

kojim je stvarno proširen sadržaj fajla i koji može biti jednak `by`, ukoliko je u fajl sistemu bilo dovoljno slobodnog mesta, odnosno manji od te vrednosti (uključujući i 0), ukoliko nije bilo dovoljno slobodnih blokova.

```
unsigned extendFile (FCB* fcb, unsigned by);
```

Rešenje

```
unsigned extendFile (FCB* fcb, unsigned by) {
    if (fcb==0) return 0; // Exception
    // Find the first free block:
    for (unsigned free = 1; free<FATSIZE && (fat[free]!=~0U); free++);
    if (free==FATSIZE) return 0; // No free space
    // Find the file's tail block:
    unsigned tail = fcb->head;
    if (tail)
        while (fat[tail]) tail = fat[tail];
    // Extend the file:
    unsigned extendedBy = 0;
    while (1) {
        if (tail)
            fat[tail] = free;
        else
            fcb->head = free;
        tail = free;
        fat[tail] = 0;
        extendedBy++;
        if (extendedBy==by) return extendedBy;
        // Find the next free block:
        for (free++; free<FATSIZE && (fat[free]!=~0U); free++);
        if (free==FATSIZE) return extendedBy; // No more free space
    }
    return extendedBy;
}
```

3. zadatak, treći kolokvijum, jun 2014.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog fajl sistema evidencija slobodnih blokova na disku vodi se na sledeći način. Indeks (spisak) slobodnih blokova je neograničen i zapisuje se u samim slobodnim blokovima, ulančanim u jednostruku listu. Prema tome, prvi slobodan blok sadrži spisak najviše N slobodnih blokova (*ne* uključujući njega samog, tj. on nije na spisku), dok poslednji ulaz u tom spisku u tom bloku sadrži broj sledećeg bloka u listi, u kome se nalazi nastavak spiska slobodnih blokova itd. Ukoliko je neki slobodni blok bio na spisku, a više nije, njegov ulaz u indeksu slobodnih blokova ima vrednost 0 (blok broj 0 nikada nije slobodan). Kada se zahteva jedan slobodan blok, treba jednostavno uzeti prvi slobodan blok sa spiska. Pri tome, ukoliko je ceo spisak sadržan u prvom bloku u listi ispražnjen, treba alocirati upravo taj prvi blok iz liste i izbaciti ga iz liste.

Na prvi blok u listi ukazuje globalna promenljiva `freeBlocksHead`. Na raspolaganju je i funkcija `getBlock` koja vraća pokazivač na deo memorije u kešu u koju je učitani traženi blok sa diska. (Napomena: broj N je određen veličinom bloka i veličinom tipa `BlockNo`.)

```
typedef ... BlockNo; // Disk block number
extern int blockSize; // Disk block size
void* getBlock (BlockNo block);
extern BlockNo freeBlocksHead;
```

Realizovati funkciju `getFreeBlock()` koja treba da alocira i vrati jedan slobodni blok:

```
BlockNo getFreeBlock ();
```

Rešenje

```

BlockNo getFreeBlock () {
    static const int numOfEntries = blockSize/sizeof(BlockNo);

    BlockNo ret = 0;
    if (freeBlocksHead==0) return ret; // No free blocks!
    BlockNo* index = (BlockNo*)getBlock(freeBlocksHead); // Get first block
    for (int i=0; i<numOfEntries-1; i++) {
        if (index[i]==0) continue;
        ret = index[i]; // Found another block
        index[i]=0;
        return ret;
    }
    // No other blocks found in the first block, return this first block:
    ret = freeBlocksHead;
    freeBlocksHead = index[numOfEntries-1];
    return ret;
}

```

3. zadatak, treći kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje kombinovanu tehniku indeksirane alokacije sadržaja fajla. U FCB fajla polje `singleIndex` predstavlja direktni indeks kao niz od `SingleIndexSize` elemenata, pri čemu svaki element sadrži broj fizičkog bloka sa sadržajem fajla (ili `null`, ako je neiskorišćen). Ako veličina sadržaja fajla preraste veličinu podržanu ovim indeksom, naredni blokovi sadržaja fajla (preko ove veličine) indeksiraju se indeksom u dva nivoa. Za te potrebe, polje `dblIndex` u FCB sadrži `DblIndex0Size` elemenata, pri čemu svaki element sadrži broj bloka sa indeksom drugog nivoa (ili `null`, ako je neiskorišćen). Blokovi sa indeksom drugog nivoa sadrže najviše `DblIndex1Size` ulaza sa brojevima blokova sa sadržajem fajla. Veličina bloka je `BlockSize`.

Realizovati funkciju `getFilePBlockNo()` datu dole, koja se koristi u implementaciji modula za organizaciju fajla i koja treba da vrati broj fizičkog bloka na disku za dati fajl i dati redni broj bajta sadržaja tog fajla (počev od 0). Dati redni broj bajta je veći proveren pre poziva ove funkcije, tako da sigurno adresira bajt unutar stvarne veličine sadržaja fajla (ne prekoračuje ga).

```
PBlock getFilePBlockNo (FCB* fcb, unsigned long byte);
```

Na raspolaganju je funkcija koja vraća pokazivač na sadržaj bloka na disku sa datim brojem, učitano u keš:

```
void* getDiskBlock (PBlock blkNo);
```

Rešenje

```

PBlock getFilePBlockNo (FCB* fcb, unsigned long bt) {
    if (fcb==0) return -1; // Exception
    unsigned long lblk = bt/BlockSize; // Logical block number
    if (lblk<SingleIndexSize) return fcb->singleIndex[lblk];
    lblk -= SingleIndexSize;
    unsigned long dblIndex0Entry = lblk/DblIndex1Size;
    unsigned long dblIndex1Entry = lblk%DblIndex1Size;
    if (dblIndex0Entry>=DblIndex0Size)
        return -1; // Exception: file size overflow
    PBlock dblIndex1PBlkNo = fcb->dblIndex[dblIndex0Entry];
    PBlock* dblIndex1 = (PBlock*)getDiskBlock(dblIndex1PBlkNo);
    if (dblIndex1==0) return -1; // Exception
    return dblIndex1[dblIndex1Entry];
}

```

3. zadatak, treći kolokvijum, jun 2013.

- [Postavka](#)
- [Rešenje](#)

U implementaciji nekog fajl sistema definisani su celobrojni tip `Byte` koji predstavlja bajt, kao i celobrojni tip `BlkNo` koji predstavlja broj logičkog bloka sadržaja fajla (numeracija počev od 0). U strukturi FCB celobrojno polje `cur` predstavlja kurzor za čitanje i upis (u bajtovima, počev od 0), a polje `size` stvarnu veličinu u bajtovima. Konstanta `BLKSIZE` predstavlja veličinu bloka u bajtovima. Na raspolaganju je funkcija koja učitava logički blok datog fajla i vraća pokazivač na taj učitani blok u kešu (vraća 0 u slučaju greške):

```
Byte* readFileBlock (FCB* file, BlkNo blockNo);
```

Realizovati funkciju `fread()` koja za dati fajl učitava `n` bajtova u dati bafer, počev od kurzora, i vraća broj stvarno učitanih bajtova, a kurzor pomera na kraj učitane sekvence. U slučaju prekoračenja veličine sadržaja fajla ili druge greške treba vratiti broj stvarno učitanih bajtova:

```
int fread (FCB* file, Byte* buffer, int n);
```

Rešenje

```
int fread (FCB* f, Byte* buf, int n) {
    if (f==0) return 0;
    if (f->cur+n>f->size) n = f->size - f->cur;
    int read = 0; // Number of bytes read
    while (n>0) {
        BlkNo blkNo = f->cur/BLKSIZE;
        int byte = f->cur%BLKSIZE;
        Byte* block = readFileBlock(f, blkNo);
        if (block==0) return read; // Exception
        while (byte<BLKSIZE && n>0) {
            buf[read++]=block[byte++];
            n--; f->cur++;
        }
    }
    return read;
}
```

3. zadatak, treći kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje kombinovanu tehniku indeksirane alokacije sadržaja fajla. U FCB fajla nalaze se dva ulaza koji predstavljaju indeks nultog nivoa (direktni pokazivači na dva prva bloka sadržaja fajla) i još četiri ulaza koji ukazuju na blokove sa indeksima prvog nivoa.

Na slici je prikazan deo FCB nekog fajla. Blok je veličine 512B, a broj bloka (svaki ulaz u indeksnom bloku) zauzima 4 bajta. Bajtovi sadržaja fajla se broje počev od 0.

1400	NULL
1401	2500
<i>pokazivač na tabelu 2</i>	2501
NULL	2502
NULL	2503
	...

(a) FCB Index

(b) Druga tabela

U kom bloku na disku se nalazi bajt sadržaja ovog fajla sa datim rednim brojem (pored konačnog odgovora, dati i celu računicu):

1. 1000 (decimalno)?
2. 2570 (decimalno)?
3. Kolika je maksimalna veličina fajla koju dozvoljava ovaj sistem?

Rešenje

1. 1401
2. 2503
3. Jedan indeksni blok sadrži najviše $512B:4B = 128$ ulaza. Maksimalna veličina fajla je: $2 \cdot 512B + 4 \cdot 128 \cdot 512B = 257KB$.

3. zadatak, treći kolokvijum, jun 2012.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje kombinovanu tehniku indeksirane alokacije sadržaja fajla. U FCB fajla nalaze se dva ulaza koji predstavljaju indeks nultog nivoa (direktni pokazivači na dva prva bloka sadržaja fajla) i još dva ulaza koji ukazuju na blokove sa indeksima prvog nivoa.

Na slici je prikazan deo FCB nekog fajla. Blok je veličine 1KB, a broj bloka (svaki ulaz u indeksnom bloku) zauzima 4 bajta. Bajtovi sadržaja fajla se broje počev od 0.

1000	4600
1001	4601
<i>pokazivač na tabelu 2</i>	4602
NULL	4603
	...

(c) FCB Index

(d) Druga tabela

U kom bloku na disku se nalazi bajt sadržaja ovog fajla sa datim rednim brojem (pored konačnog odgovora, dati i celu računicu):

1. 2000 (decimalno)?
2. 3570 (decimalno)?
3. Kolika je maksimalna veličina fajla koju dozvoljava ovaj sistem?

Rešenje

1. 1001
2. 4601
3. Jedan indeksni blok sadrži najviše $1024B:4B = 256$ ulaza. Maksimalna veličina fajla je: $2 \cdot 1KB + 2 \cdot 256 \cdot 1KB = 514KB$.

3. zadatak, treći kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje indeksirani pristup alokaciji prostora za sadržaj fajla, s tim da je indeks neograničen i organizovan kao jednostruko ulančana lista indeksnih blokova. Na prvi indeksni blok u listi ukazuje polje `index` u FCB. Svaki indeksni blok sadrži `NumOfEntries` ulaza tipa `BlkNo` koji ukazuju na blokove sa sadržajem fajla i još jedan ulaz (iza ovih) istog tipa koji ukazuje na sledeći indeksni blok u listi (vrednost 0 označava *null* ulaz). Na raspolaganju je funkcija za pristup blokovima diska kroz keš, koja vraća pokazivač na deo memorije u kome se nalazi traženi blok diska učitani u keš (vraća 0 u slučaju greške):

```
Byte* getDiskBlock (BlkNo block);
```

Realizovati funkciju `getFileBlock()` koja za dati fajl dohvata logički blok sa datim brojem. U slučaju prekoračenja veličine sadržaja fajla ili druge greške treba vratiti 0.

```
Byte* getFileBlock (FCB* file, unsigned int block);
```

Rešenje

```
Byte* getFileBlock (FCB* file, unsigned int block) {
    if (file==0 || file->index==0) return 0;
    BlkNo* index = (BlkNo*)getDiskBlock(file->index);
    if (index==0) return 0;
    while (block>=NumOfEntries) {
        block -= NumOfEntries;
        if (index[NumOfEntries]==0) return 0;
        index = (BlkNo*)getDiskBlock(index[NumOfEntries]);
        if (index==0) return 0;
    }
    if (index[block]==0) return 0;
    return getDiskBlock(index[block]);
}
```

3. zadatak, treći kolokvijum, jun 2011.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem koristi FAT, uz dodatni mehanizam detekcije i oporavka od korupcije ulančanih lista na sledeći način. Kada se FAT kešira u memoriji, vidi se kao niz struktura tipa `FATEntry`. Ova struktura ima dva celobrojna polja. Polje `next` je broj ulaza u FAT u kome se nalazi sledeći element u ulančanoj listi; vrednost 0 označava kraj liste. Polje `fid` ove strukture sadrži identifikator fajla kome pripada taj element liste. U strukturi FCB celobrojno polje `id` predstavlja identifikator datog fajla, a polje `head` sadrži redni broj ulaza u FAT koji je prvi element u ulančanoj listi datog fajla.

Implementirati funkciju čiji je potpis dat dole. Ona treba da proveri konzistentnost ulančane liste datog fajla, proverom da li svi elementi liste pripadaju baš tom fajlu. Ukoliko je sve u redu, ova funkcija treba da vrati 1. Ukoliko naiđe na element u listi koji je pogrešno ulančan, odnosno ne pripada tom fajlu (tako što polje `fid` ne odgovara identifikatoru tog fajla), taj pogrešno ulančani ostatak liste treba da „odseče“ postavljanjem terminatora liste (vrednost 0 u polje `next`) u poslednji ispravan element liste (ili glavu liste, ako je prvi element pogrešan) i da vrati 0.

```
FATEntry FAT[...];
```

```
int check_consistency (FCB* file);
```

Rešenje

```
int check_consistency (FCB* file) {
    if (file==0) return 0;
    FID fid = file->id;
    int cur=file->head, prev=0;
    for (; cur!=0; prev=cur, cur=FAT[cur].next)
        if (FAT[cur].fid!=fid) {
            if (prev) FAT[prev].next=0;
            else file->head=0;
            return 0;
        }
    return 1;
}
```

3. zadatak, treći kolokvijum, septembar 2011.

- Postavka
- Rešenje

Neki fajl sistem koristi indeksirani pristup alokaciji fajlova sa indeksima u dva nivoa. Polje `index` u strukturi FCB sadrži broj indeksnog bloka prvog nivoa. Broj bloka 0 u indeksu označava *null* - nealociran ulaz (fizički blok broj 0 se nikada ne koristi za fajlove). Konstanta `INDEXSIZE` definiše broj ulaza u jednom indeksu (u jednom bloku), a tip `BLKNO` predstavlja broj bloka (logičkog ili fizičkog). Realizovati funkciju koja se koristi kod direktnog pristupa fajlu:

```
void* f_getblk(FCB* file, BLKNO logical);
```

koja vraća pokazivač na dohvaćeni i keširani blok sa podacima fajla sa datim logičkim brojem. Na raspolaganju je funkcija:

```
void* f_getblk(BLKNO physical);
```

koja vraća pokazivač na keširani blok sa datim fizičkim brojem i učitava ga u keš ako je potrebno.

Rešenje

```
void* f_getblk(FCB* file, BLKNO lb) {
    if (file==0) return 0; // Exception: null FCB

    unsigned long int entry = lb/INDEXSIZE;
    if (entry>=INDEXSIZE) return 0; // Exception: lb too large!

    BLKNO* index = (BLKNO*)f_getblk(file->index);
    if (index==0) return 0; // Exception: error accessing index!

    BLKNO index2blk = index[entry];
    if (index2blk==0) return 0; // Access over the file's boundary

    index = (BLKNO*)f_getblk(index2blk);
    if (index==0) return 0; // Exception: error accessing index!

    entry = lb%INDEXSIZE;
    return f_getblk(index[entry]);
}
```

Uvod u operativne sisteme

1. zadatak, prvi kolokvijum, april 2009.

- [Postavka](#)
- [Rešenje](#)

Pored svake dole navedene tvrdnje napisati da li je ona tačna ili netačna:

1. Multiprogramski sistem bez deljenja vremena stvara privid da svaki korisnik ima svoj računar uspešnije nego sistem sa deljenim vremenom.
2. Poslovi koji rade isključivo izračunavanje (koriste samo CPU), brže će biti završeni (počev od trenutka kada su aktivirani) na sistemu sa deljenim vremenom nego na sistemu sa paketnom obradom.
3. Smanjenje kvanta vremena koje se dodeljuje procesima u sistemu sa deljenim vremenom može da poveća ukupno vreme zadržavanja procesa u sistemu.
4. U monoprogramskom multiprocesnom sistemu, u jednom trenutku u memoriji ne može biti više od jednog procesa.

Rešenje

1. Netačno.
2. Netačno.
3. Tačno.
4. Netačno.

1. zadatak, prvi kolokvijum, april 2008.

- [Postavka](#)
- [Rešenje](#)

Posmatra se izvršavanje nekoliko neinteraktivnih poslova sa ulazno/izlaznim operacijama na paketnom operativnom sistemu i na operativnom sistemu sa multiprogramiranjem, pod pretpostavkom identičnih hardverskih platformi i uporednom izvršavanju na multiprogramskom sistemu. Pored svake dole navedene tvrdnje napisati da li je ona tačna ili netačna:

1. Vreme od početka do završetka izvršavanja posla kraće je na multiprogramskom sistemu.
2. Iskorišćenje procesora bolje je na multiprogramskom sistemu.
3. Propusnost sistema (broj završenih poslova u jedinici vremena, *throughput*) bolja je na multiprogramskom sistemu.
4. Režijsko vreme (beskorisno za poslove) manje je na multiprogramskom sistemu.

Rešenje

1. Netačno.
2. Tačno.
3. Tačno.
4. Netačno.

1. zadatak, prvi kolokvijum, april 2006.

- [Postavka](#)
- [Rešenje](#)

Posmatra se izvršavanje nekoliko neinteraktivnih poslova sa ulazno/izlaznim operacijama na paketnom operativnom sistemu i na operativnom sistemu sa multiprogramiranjem, pod pretpostavkom identičnih hardverskih platformi i uporednom izvršavanju na multiprogramskom sistemu. Pored svake dole navedene tvrdnje napisati da li je ona tačna ili netačna:

1. Vreme od početka do završetka izvršavanja posla kraće je na multiprogramskom sistemu.
2. Iskorišćenje procesora bolje je na multiprogramskom sistemu.

3. Propusnost sistema (broj završenih poslova u jedinici vremena, *throughput*) bolja je na multiprogramskom sistemu.
4. Režijsko vreme (beskorisno za poslove) manje je na multiprogramskom sistemu.

Rešenje

1. NE - Objašnjenje: kod multiprogramskog okruženja POJEDINAČNI posao će trajati duže zbog konkurentnog izvršavanja i režijskih operacija koje se događaju u toku izvršenja posla.
2. DA - Objašnjenje: Dok jedan posao čeka na I/O, drugi se izvršava, tj. koristi processor, tako da se iskorišćenje procesora povećava.
3. DA - Objašnjenje: Pošto se poslovi izvršavaju u paraleli (dok jedan čeka na I/O, a drugi se izvršava), onda se ukupno vreme odziva skraćuje, pa se samim tim broj završenih poslova u jedinici vremena (*throughput*) povećava.
4. NE - Objašnjenje: Jedino režijsko vreme kod paketnog Osa je dovlačenje posla u radnu memoriju, dok kod multiprogramskog Osa postoje režijske operacije i prilikom svake zamene poslova.